Soft-Constraint Quadratic Programming: Methods, Contracts, and Negative Timing Results

Felipe Santibáñez-Leal
CAOS Research program, github.com/fsantibanezleal/CAOS_SoftQP

2 November 2026

Technical report · v0.01.002, 2 November 2026 · Licence CC BY 4.0
CAOS Research programme · soft-constraint quadratic programming
Version DOI 10.5281/zenodo.23899079
Concept DOI (always latest) 10.5281/zenodo.23851659
Not peer reviewed. Source code, data and computational records:
github.com/fsantibanezleal/CAOS_SoftQP

Abstract

Soft constraints price departures from selected operating preferences while preserving immutable physical limits. We describe a reproducible convex quadratic optimization system that makes the original objective, physical units, method compatibility and numerical evidence inspectable. Seven implemented paths cover explicit slack lifting, published generalized projection, side-specific DAQP weights, a fixed FlexQP reconstruction, ElastiQP, a side-specific interval proximal update and a directional disjoint group-peak update. Eighteen versioned contracts connect synthetic input generation, native execution, independent original-space evaluation, local request/result exchange and verified static presentation. Fifteen authored or source-faithful model cases exercise scalar semantics, mixed hard/soft policy, sparse structure, predictive control and building energy. A separate registered computation contains 456 seeded instances, retaining 11592 accepted and 7560 incompatible measured observations. Its 168-pair held-out median relative cold end-to-end reduction is 0.499839%, with a 95% seed-cluster interval from -3.291967% to 3.944835%. A second computation retains all 210 grouped repeated- forecast inputs and 7770 raw observations. Its complete 144-member held-out primary score is -212.449875%, with interval limits -225.336618% and -192.873759%, including 9 failed pairs. Neither registered ten-percent improvement criterion is established. Equivalent formulations, physical covariance, objective/dual checks and retained failure states distinguish mathematical agreement from elapsed-time advantage. The system supports transparent research and development; the synthetic evidence does not establish operational safety, measured field benefit or algorithmic priority.

1 Introduction

An optimization model can be numerically feasible and operationally unacceptable. In a predictive controller, relaxing an actuator bound changes what the plant is permitted to do; relaxing a preferred room temperature changes the service being requested. Treating both departures as interchangeable slack conceals a policy decision. The application studied here makes that decision

explicit: physical limits remain hard, while selected interval preferences receive specified lower- and upper-side prices. The intended use is the construction and inspection of small, reproducible optimization models before a practitioner embeds one in a larger controller. The examples are synthetic. They do not measure occupant satisfaction, building savings, industrial reliability or operational safety.

The numerical foundation is a convex quadratic program with sparse affine rows. That foundation appears in constrained model predictive control, energy scheduling and repeated allocation problems [26, 19, 19]. A soft constraint adds a convex price to selected departures rather than changing the definition of physical feasibility. The important modeling questions are therefore which rows may yield, which direction matters, which units a departure has, and which cost is actually optimized. Exact penalty theory explains why an adequately large linear price can sometimes recover a hard-constrained optimum; it does not say that an arbitrary finite price makes every returned elastic optimum acceptable for a physical system [16, 22].

The implementation compares explicit lifting, generalized slack projection, side-specific projection, DAQP, a fixed-parameter FlexQP reconstruction and ElastiQP on their genuine compatibility domains. A separate directional group peak implementation couples several interval departures while retaining the original interval rows in its principal system. Neither unequal side prices nor the ability to express a directional peak establishes a new modeling capability. DAQP already supports unequal side weights; directional row splitting and a compact epigraph express the grouped objective exactly [1, 29, 21]. The computational question is narrower: does a particular original-row proximal implementation reduce the cost of solving the same mathematical problem?

Two completed studies answer that question negatively for their registered primary criteria. The first contains 456 seeded instances and retains 11592 accepted and 7560 incompatible measured observations. Its 168 accepted held-out pairs yield a median relative cold end-to-end reduction of 0.499839%, with a 95% seed-cluster interval from -3.291967% to 3.944835%. The second contains 210 repeated-forecast instances. Its complete 144-instance held-out endpoint yields -212.449875%, with corresponding limits -225.336618% and -192.873759%, including 9 failed pairs. Neither result establishes the predicted 10% improvement. These are distinct studies with different native packages, populations and timing boundaries.

The technical contribution is an inspectable implementation and evidence chain, not a claim of uniform superiority. Specifically, the system provides an original- space model with physical row metadata; exact adapters with explicit incompatibility states; independent objective, hard-row and primal and dual checks; repeated-operation measurements with retained failures; and versioned producer/consumer boundaries. The distinction between numerical evidence and its presentation is implemented: native code computes solutions, local tools export validated results, and the static reader displays or imports matching evidence. A dormant API and unavailable coupled browser path remain visibly unavailable.

The remainder gives the mathematical formulation and notation, details every data exchange and processing stage, derives each executable method, and states the experimental protocols. The results include model agreement, formulation size, conditioning, repeated control and complete negative primary inference. Validation and discussion distinguish a correct optimum from a fast computation and from an operationally useful decision. The accompanying research paper develops the two related proximal proposals and their separately attributed experiments; this technical account also describes the full software system and all of its contracts.

2 Problem formulation

Let $x \in \mathbb{R}^n$ be the original decision vector. The symmetric matrix $P \in \mathbb{R}^{n\times n}$ is positive semidefinite, $q \in \mathbb{R}^n$ is finite, and $A \in \mathbb{R}^{m\times n}$ is sparse. Row i describes the physical or normalized quantity $A_i x$ with endpoints $\ell_i \le u_i$. An absent lower or upper endpoint is represented by an explicit missing-side value in the serialized model, not by an arbitrary large finite number. Let $\mathcal{H}$ be the hard row set and $\mathcal{S}$ its soft complement. The base objective is

$$f(x) = \tfrac{1}{2} x^\top P x + q^\top x. \tag{1}$$

The physical interpretation of a coefficient is determined by the decision's units and the chosen objective unit. If x_j is a position in metres, its linear coefficient has objective units per metre, while the corresponding diagonal quadratic coefficient has objective units per square metre. A coefficient is not intrinsically dimensionless merely because it is stored as a floating-point value.

For a scalar row value $z_i = A_i x$, define the nonnegative departures

$$v_{L,i}(z_i) = (\ell_i - z_i)_+, \qquad v_{U,i}(z_i) = (z_i - u_i)_+. \tag{2}$$

An absent side contributes zero and has no associated finite-side penalty term. The sign convention for a corrective slack is

$$s_i = v_{L,i} - v_{U,i}, \qquad z_i + s_i \in [\ell_i, u_i]. \tag{3}$$

Thus an upper overshoot has negative corrective slack. Plots display the two nonnegative departures separately so opposite violations cannot cancel visually. The separable side price is

$$g_i(z_i) = a_{L,i} v_{L,i} + a_{U,i} v_{U,i} + \tfrac{1}{2} b_{L,i} v_{L,i}^2 + \tfrac{1}{2} b_{U,i} v_{U,i}^2, \qquad a_{L,i}, a_{U,i}, b_{L,i}, b_{U,i} \ge 0. \tag{4}$$

These nonnegative coefficients define a proper closed convex function for a nonempty interval. A zero coefficient removes that portion of the price. Setting every soft coefficient to zero makes that preference free; it does not convert the row into a hard constraint. Hardness is a separate policy field.

The complete separable problem is

$$\min_x f(x) + \sum_{i\in\mathcal{S}} g_i(A_i x) \quad \text{subject to} \quad \ell_i \le A_i x \le u_i \quad (i \in \mathcal{H}). \tag{5}$$

Mixed published examples additionally permit Huber and grouped penalties through their explicitly compatible adapters. They are not silently replaced by (4). The grouped extension uses disjoint sets I_g and strictly positive directional scales $c_{U,i}, c_{L,i}$:

$$M_g(z) = \max\{0, \max_{i\in I_g} c_{U,i}(z_i - u_i)_+, \max_{i\in I_g} c_{L,i}(\ell_i - z_i)_+\}. \tag{6}$$

The corresponding price is $a_g M_g + \frac{1}{2} b_g M_g^2$, with nonnegative a_g, b_g. If group rows use different physical quantities, the directional scales must convert their departures to a common peak unit. In the teaching example, inverse- metre scales make the peak dimensionless. Adding raw metres to degrees or energy would define no coherent physical maximum.

The implementation refuses negative quadratic curvature, invalid sparse shapes, inconsistent vector lengths, reversed finite endpoints, nonpositive directional scales, and overlapping or empty groups. Positive semidefinite base curvature alone does not guarantee existence or uniqueness. Hard constraints can be infeasible, and zero-curvature directions can be unbounded. A valid

input schema therefore establishes an admissible mathematical representation, not existence of an optimizer. The evaluator retains numerical termination and original-space acceptance separately. This distinction is important when an algorithm returns an iteration-limited candidate: a vector may be useful diagnostically without being counted as an accepted solution under the experimental rule.

All algorithm comparisons preserve (5) or its explicitly named grouped counterpart. A method that cannot represent the selected penalty is incompatible. Changing a quadratic price to a linear price, adding a Hessian regularizer, or relaxing a hard inequality would define another optimization problem. Such changes cannot serve as unannounced numerical repairs in a matched comparison, even if they make a backend return a successful status.

3 Notation and physical units

Table 1 collects the mathematical quantities used in the model, algorithms and acceptance calculations. A single scalar objective unit is assumed within one optimization instance. This permits comparison of objective values between equivalent formulations of that instance, but does not permit adding objectives from different physical models or independently normalized examples. The sparse matrix entries inherit row-unit divided by decision-unit dimensions. Their numerical magnitude may change under unit conversion even when the physical problem and its optimizer remain identical.

Table 1: Notation and dimensions for the original-space soft quadratic model. Row and decision units belong to the physical metadata of each synthetic case rather than to a universal unit convention.

Symbol	Meaning	Units
x_j	Original decision component	Declared decision unit
$A_i x, \ell_i, u_i$	Row value and interval endpoints	Declared row unit
$v_{L,i}, v_{U,i}, s_i$	Lower/upper departure and corrective slack	Row unit
f, F, g_i	Base, total and interval penalty objective	Objective unit
$a_{L,i}, a_{U,i}$	Linear departure prices	Objective unit per row unit
$b_{L,i}, b_{U,i}$	Quadratic departure prices	Objective unit per squared row unit
$c_{L,i}, c_{U,i}$	Directional peak scales	Peak unit per row unit
y_i	Original row multiplier	Objective unit per row unit
H_{ii}	Positive proximal metric entry	Objective unit per squared row unit
r_{stat}	Stationarity residual	Coordinatewise objective gradient unit
$T, \Delta t$	Measured duration and physical time step	Seconds, or explicit model hours
R_i	Paired relative timing reduction	Dimensionless; displayed as percent

A row transformation $z'_i = d_i z_i$ with $d_i > 0$ transforms both endpoints by the same factor. To preserve a separable objective, the corresponding prices transform as $a'_i = a_i/d_i$ and $b'_i = b_i/d_i^2$. A directional peak scale transforms as $c'_i = c_i/d_i$. The row dual transforms reciprocally, $y'_i = y_i/d_i$, so its contribution to stationarity remains invariant. These rules distinguish a genuine unit change from a policy change. Altering only a directional scale changes the penalty's relative preference and can legitimately move the physical optimizer.

The control examples use position in metres, velocity in metres per second and acceleration in metres per squared second. A metre-to-centimetre transformation must also transform position-sensitive objective and row coefficients. A seconds- to-milliseconds transformation changes

discrete dynamic coefficients and the numerical representation of velocity and acceleration; changing labels alone is not a test of covariance. The hourly thermal grouped examples use a different time convention from the quarter-hour published building example. Their heater energy and comfort scales are interpreted within their own model, not transferred by matching variable names.

Timing quantities are wall-clock durations with explicit measurement boundaries. They are not physical prediction times. A thirty-two-frame forecast operation can have one measured duration even though its model horizon spans hours or seconds. Likewise, a plotted input index is a population ordering, not a measured time axis. Keeping these meanings distinct prevents a computational scatter plot from being read as a physical control trajectory or an operational latency guarantee.

The unit-aware reader distinguishes a zero value from an absent physical quantity. If the selected row unit contains no hard row, no maximum hard-row error in that unit exists. Reporting zero would incorrectly suggest a measured feasibility check. The same convention applies to unsupported method outputs: their objective and elapsed duration remain unavailable rather than receiving a numerical value that could enter a later sum or comparison.

4 Data and data contracts

Every numerical input in the evaluated product is synthetic or adapted from an identified open synthetic model. There are no private building measurements, customer records or operational performance labels. The fifteen-case catalog uses a fixed authored seed and separates scalar semantics, hard-row integrity, sparse structure, predictive control and energy policy. The registered original and grouped studies introduce their own prospective instance grids and seed splits; neither is inferred from the teaching catalog. The code and model attribution retain their separate upstream terms, while explanatory material and the archived scientific datasets carry their stated distribution licences.

The C13 building source follows the published two-zone model from OSQPSlack, with a 20 kWh battery and 96 quarter-hour intervals. It preserves mixed quadratic, grouped, elastic and Huber prices. C14 and C15 reuse the physical structure with explicitly changed separable side-price policies, so their optima are not reproductions of the mixed objective. The grouped study's hourly thermal inputs use a separate synthetic RC forecast model with hard dynamics, initial conditions and heater-energy limits. A forecast input is not a measured trajectory or a closed-loop controller experiment. These distinctions constrain every comparison and figure interpretation.

The eighteen versioned boundaries below specify how those scientific objects move between producers, numerical tools and readers. Tables are generated from the actual JSON schemas; the accompanying prose explains the physical semantics and cross-field checks that a type validator cannot prove. Unknown fields, malformed dimensions and invalid states are refused according to the respective schema and consumer. Missing data remain missing rather than becoming an empty successful result. A schema example establishes shape, not deployed availability. Physical units are explicit row/decision metadata where they apply; identifiers, hashes, enum states and counts have no physical dimension.

4.1 Deployment case inventory

The case-index contract schema 3.1.0 is produced by the manifest component and consumed by the static artifact loader and optional file-serving API. Its schema, engine version, case count, default case, case list and file inventory are shown in Table 2. Each listed artifact has an identity that the reader checks before using its contents. Counts are nonnegative integers; paths and hashes are identifiers rather than physical quantities. A default selection must refer to a listed case, and a file's checksum must match its actual bytes. Schema validity alone cannot establish either condition. Absence of this inventory stops the site build because a reader cannot safely discover its scientific evidence. An older pipeline version is stale and is displayed as such. The index never proves that every possible solver request has run; it names the finite exported distribution. Replacing the source with a fixture inventory exercises the same consumer boundary without pretending the fixture is the complete production case population.

Table 2: Schema-derived top-level fields, dimensions and literal structural restrictions for case-index schema 3.1.0. Complete nested numerical definitions are expanded in the contract appendix; physical maps remain those of the original model.

Field	Type	Constraint	Unit / role
schema	string	const=softqp.artifact-index/v1	structural metadata / nested units
engine_version	string	See defined object / value type	structural metadata / nested units
n_cases	integer	minimum=1	count / integer identifier
default_case	string	pattern=^C[0-9]{2}$	structural metadata / nested units
cases	array	minItems=1	structural metadata / nested units
files	array	uniqueItems=True	structural metadata / nested units

4.2 Individual deployment manifest

The case-manifest contract schema 5.0.0 connects one scientific case to the shared deployment shell. The manifest producer assigns the case identifier, bilingual title/category, engine identity, product version, seed, source, artifact descriptors, lane and acceptance information. The consumer uses that envelope to select available evidence without embedding numerical assumptions in navigation code. Table 3 lists the actual fields. A seed is a reproducibility identifier; it is not a physical measurement. Each artifact retains its own unit-bearing scientific payload. If an indexed manifest is absent, readiness is partial and the corresponding case is unavailable, while other cases remain accessible. Unknown cases receive a missing-file response. A stale engine/product identity cannot be hidden by preserving an old checksum. The producer and consumer tests also exercise source substitution, preventing a handwritten browser fixture from becoming a special unvalidated path. This contract describes presentation availability, not a certificate of every mathematical optimum contained in the referenced payload.

Table 3: Schema-derived top-level fields, dimensions and literal structural restrictions for case-manifest schema 5.0.0. Complete nested numerical definitions are expanded in the contract appendix; physical maps remain those of the original model.

Field	Type	Constraint	Unit / role
schema	string	const=softqp.manifest/v1	structural metadata / nested units
case_id	string	pattern=^C[0-9]{2}$	structural metadata / nested units
title	object	additionalProperties=False	structural metadata / nested units
category	object	additionalProperties=False	structural metadata / nested units
engine	object	additionalProperties=False	structural metadata / nested units
product_version	string	See defined object / value type	structural metadata / nested units
seed	integer	See defined object / value type	count / integer identifier
source	string	See defined object / value type	structural metadata / nested units
artifact	object	additionalProperties=False	structural metadata / nested units
lane	string	const=precompute	structural metadata / nested units
gate	object	additionalProperties=False	structural metadata / nested units

4.3 Readiness and catalog identity

The readiness contract schema 2.0.1 is computed from the actual contract catalog, product identity and distributed inventory. Its schema tag, product version, catalog digest and per-contract states are shown in Table 4. It is consumed by the shared reader and optional API to distinguish available, provider, missing, partial and stale evidence within the shared state vocabulary. These are availability states with explanatory reasons, not accuracy labels. The producer does not infer a trained model from an empty model directory, and the readiness document excludes itself from recursively proving its own presence. An absent file prevents a complete build; a runtime load failure is reported as unknown readiness while other views retain their own evidence. The catalog digest prevents a changed boundary inventory from being paired silently with an old readiness result. Four grouped-study boundaries remain unavailable to the browser because no validated replay adapter is installed. Their valid schema examples and complete external study data do not change that missing state.

4.4 Dormant file-serving API

The API contract schema 2.1.0 is an OpenAPI boundary owned by the optional app service. In the current static product this service is dormant. Its unavailable body has a detail field, while the full OpenAPI description also specifies case, artifact and readiness routes and response states. Table 5 describes the unavailable body rather than inventing a live inference response. The producer serves precomputed files when explicitly activated; it never executes an optimizer as a consequence of an HTTP request. The static reader uses committed artifacts directly and therefore does not depend on a resident API. A missing routed document returns the documented

Table 4: Schema-derived top-level fields, dimensions and literal structural restrictions for readiness schema 2.0.1. Complete nested numerical definitions are expanded in the contract appendix; physical maps remain those of the original model.

Field	Type	Constraint	Unit / role
schema	string	const=softqp.readiness/v2	structural metadata / nested units
product_version	string	pattern=^\d+\.\d{2}\.\d{3}$	structural metadata / nested units
catalog_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
contracts	array	minItems=1	structural metadata / nested units

unavailable response with its reason, and an unknown case or artifact is missing. An enabled service's own readiness is distinct from the static distribution's explicit dormant-API entry. This distinction avoids interpreting an example response, a schema-valid object or a local development server as evidence of a deployed numerical computation service.

Table 5: Schema-derived top-level fields, dimensions and literal structural restrictions for api schema 2.1.0. Complete nested numerical definitions are expanded in the contract appendix; physical maps remain those of the original model.

Field	Type	Constraint	Unit / role
detail	object	additionalProperties=False	structural metadata / nested units

4.5 Scientific case inventory

The softqp-index contract schema 1.1.0 is produced by the scientific catalog runner and consumed by the workbench's case-data loader. It carries its schema version, overall state, default case, case descriptions, environment and limitations; Table 6 gives the field inventory. It names actual scientific records separately from shell deployment manifests. The case count and identifiers describe a finite synthetic catalog, not a sample of field deployments. Environment strings identify how the exported objects were computed; they do not make an arbitrary user's local runtime equivalent. A missing inventory leaves the scientific workbench unavailable with a path-specific explanation. Development/provider evidence is identified as such. Consumers compare listed paths and byte identities before interpreting case records, and source-swap tests exercise the same code using a real alternative fixture tree. This boundary prevents a visually populated case selector from being confused with loaded, current and scientifically compatible data for the selected method.

4.6 Original-space case result

The softqp-case-result contract schema 2.3.0 contains a case identifier, bilingual descriptive fields, seed/source, numerical settings, method outcomes, original row metadata and, where supplied, the sparse problem. The scientific runner produces it; the workbench validates it and computes its physical displays. Table 7 lists its fields. Each run retains state and reason, original decision/dual

Table 6: Schema-derived top-level fields, dimensions and literal structural restrictions for softqp-index schema 1.1.0. Complete nested numerical definitions are expanded in the contract appendix; physical maps remain those of the original model.

Field	Type	Constraint	Unit / role
schema_version	string	const=softqp.index/v1	structural metadata / nested units
state	string	const=development	structural metadata / nested units
default_case	union / composition	See defined object / value type	structural metadata / nested units
cases	array	See defined object / value type	structural metadata / nested units
environment	object	additionalProperties=False	structural metadata / nested units
limitations	array	See defined object / value type	structural metadata / nested units

vectors, dimensions, termination and objective/violation metrics when available. Row units attach to original departures, not auxiliary lifted coordinates. Method incompatibility and numerical failure do not satisfy the available-result branch of the schema. Cross-field consumers check vector lengths, original objectives, hard policy and lower/upper violation consistency. A missing case leaves that case unavailable while other records remain usable. A zero numerical hard error is reported only when the selected unit actually contains hard rows; absence of such rows is unavailable, not a fabricated zero. This preserves both scientific semantics and unit-aware interpretation.

Table 7: Schema-derived top-level fields, dimensions and literal structural restrictions for softqp-case-result schema 2.3.0. Complete nested numerical definitions are expanded in the contract appendix; physical maps remain those of the original model.

Field	Type	Constraint	Unit / role
schema_version	string	const=softqp.case-result/v2	structural metadata / nested units
case_id	string	pattern=^C[0-9]{2}$	structural metadata / nested units
title	object	additionalProperties=False	structural metadata / nested units
category	object	additionalProperties=False	structural metadata / nested units
seed	integer	See defined object / value type	count / integer identifier
source	string	minLength=1	structural metadata / nested units
settings	object	additionalProperties=False	structural metadata / nested units
runs	array	minItems=1	structural metadata / nested units
rows	array	See defined object / value type	count / integer identifier
problem	object	additionalProperties=False	structural metadata / nested units

4.7 Sparse scientific problem specification

The softqp-case-spec contract schema 1.0.0 is the original-space input exchanged by local request execution. Its matrices P, A, vector q, lower/upper bounds and penalty object appear in Table 8. Sparse matrices use explicit compressed-column arrays: data, row indices, column pointers and shape. Pointer monotonicity, index range, finite coefficients and dimension agreement are numerical cross-field requirements beyond a list type. Bounds may explicitly omit a side; missing endpoints are not replaced by arbitrary large finite constants. The penalty object preserves hard/soft policy and side coefficients. The producer is the local I/O component, and the native executor and browser import validator are its consumers. A complete matching specification is required before local execution can be accepted; its absence does not disable canonical replay. Physical units remain supplied by the original case's row/decision context, so a structurally valid matrix cannot independently redefine metres as centimetres or change a hard physical row into a finite-cost preference.

Table 8: Schema-derived top-level fields, dimensions and literal structural restrictions for softqp-case-spec schema 1.0.0. Complete nested numerical definitions are expanded in the contract appendix; physical maps remain those of the original model.

Field	Type	Constraint	Unit / role
P	object	additionalProperties=False	structural metadata / nested units
q	array	maxItems=20000	cost / decision unit
A	object	additionalProperties=False	structural metadata / nested units
lower	array	maxItems=20000	original row unit
upper	array	maxItems=20000	original row unit
penalty	object	additionalProperties=False	structural metadata / nested units

4.8 Bounded local cost-edit request

The softqp-run-request contract schema 1.1.0 carries the case identity, immutable base digest, schema version and three edit factors; its fields are in Table 9. The browser creates the request, and the local scientific request runner consumes it. Lower and upper factors range from 0.01 to 100, while the additional quadratic factor ranges from 0 to 100. They are dimensionless multipliers on existing side prices, not replacement physical units or arbitrary sparse matrices. The base digest binds the edit to the exact canonical model. Cross-field validation preserves matrices, linear term, bounds and hard-row coefficients; grouped/Huber models outside the separable editor are refused. If no matching complete request exists, the local path is unavailable while stored replay remains valid. A changed digest, extra field or unsupported factor cannot be repaired by accepting an unrelated result. This narrowly defined exchange makes a local what-if policy change inspectable without implying a hosted solver or browser-native execution.

4.9 Local native result envelope

The softqp-local-export contract schema 1.1.0 wraps the original request, result, schema identity and optional runtime information. The local request runner produces it, and the browser verifies it before changing the selected workbench result. Table 10 gives the top-level fields; nested request

Table 9: Schema-derived top-level fields, dimensions and literal structural restrictions for softqp-run-request schema 1.1.0. Complete nested numerical definitions are expanded in the contract appendix; physical maps remain those of the original model.

Field	Type	Constraint	Unit / role
schema_version	string	const=softqp.run-request/v1	structural metadata / nested units
case_id	string	pattern=^C[0-9]{2}$	structural metadata / nested units
base_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
lower_multiplier	number	minimum=0.01; maximum=100	dimensionless factor
upper_multiplier	number	minimum=0.01; maximum=100	dimensionless factor
quadratic_multiplier	number	minimum=0; maximum=100	dimensionless factor

and result retain their own complete definitions. Runtime fingerprints improve provenance but do not replace original-space numerical checks. The consumer matches case digest and edit factors, checks unchanged hard policy, recomputes objective components and departures, and rejects altered imported objectives. Missing or mismatched exports leave local execution unavailable, preserving canonical replay. Accepted imports are labelled local-native and can be restored to the original record; changing the selected case discards an incompatible local import. No upload is sent to a hosted service. The exported file is therefore evidence of a particular external numerical execution under a matching request, not proof that the reader itself computed an optimizer or that every result from a user machine has the historical experiment's source identity.

Table 10: Schema-derived top-level fields, dimensions and literal structural restrictions for softqp-local-export schema 1.1.0. Complete nested numerical definitions are expanded in the contract appendix; physical maps remain those of the original model.

Field	Type	Constraint	Unit / role
schema_version	string	const=softqp.local-export/v1	structural metadata / nested units
request	object	additionalProperties=False	structural metadata / nested units
result	object	additionalProperties=False	structural metadata / nested units
environment	object	additionalProperties=False	structural metadata / nested units

4.10 Paired predictive-control stream

The softqp-stream contract schema 1.0.0 is produced by the experiment exporter and consumed by the fixed C12 stream reader. It carries case/protocol/source identities, environment, stream specification, frames, counts and limitations, as shown in Table 11. Frames retain physical time, the common state, oracle action, disturbances and method/mode observations. Position, velocity and acceleration use metres, metres per second and metres per squared second; elapsed numerical durations use seconds. Thirty-two frames share one independently driven trajectory, so comparing their solver outputs does not compare separate closed-loop controllers. Warm and cold workspaces have explicit setup/update boundaries. A missing stream is shown as missing

with its path without erasing other evidence. The stream remains unavailable after an edited local model because its recorded policy no longer matches that request. Full frame ordering and mode identities are checked; overlapping predicted horizon costs cannot be summed as realized operational costs, and incompatible generalized projection states remain visible.

Table 11: Schema-derived top-level fields, dimensions and literal structural restrictions for softqp-stream schema 1.0.0. Complete nested numerical definitions are expanded in the contract appendix; physical maps remain those of the original model.

Field	Type	Constraint	Unit / role
schema_version	string	const=softqp.stream/v1	structural metadata / nested units
case_id	string	const=C12	structural metadata / nested units
protocol_id	string	const=softqp-experiments-v1	structural metadata / nested units
engine_source	string	pattern=^[0-9a-f]{40}$	structural metadata / nested units
environment	object	additionalProperties=False	structural metadata / nested units
specification	object	additionalProperties=False	structural metadata / nested units
frames	array	minItems=32; maxItems=32	count / integer identifier
counts	object	additionalProperties=False	structural metadata / nested units
limitations	array	See defined object / value type	structural metadata / nested units

4.11 Original experiment summary

The softqp-campaign contract schema 1.0.0 summarizes the original registered study. The exporter supplies the protocol, environment, source/digest maps, observed process duration, peak worker memory, restart count, planned/completed instance counts, cell summaries, primary endpoint and linked artifacts. Table 12 lists these fields. Durations are in seconds and resident memory in bytes; objective discrepancies and relative reductions are dimensionless. The browser's experimental reader consumes the summary after byte verification. A completed count must agree with the actual instance inventory and retained trial states. Its primary endpoint is the stored original 168-pair analysis, not a value recomputed from whichever descriptive plot is currently filtered. Missing evidence is explicitly unavailable while other case data remain present. Runtime measurements describe the specified supervised process and are not process-wide maxima inferred from CPU activity. This contract connects numerical observations to their registered uncertainty without treating a schema-valid positive replacement or a newly written checksum as independent scientific acceptance.

4.12 Seeded instance inventory

The softqp-datasets contract schema 1.0.0 governs the original study's array of generated-instance descriptors. Each entry identifies its split, dimension, density, diagonal condition, penalty family, seed, generation duration, problem digest and independent oracle state/reason/objective. Its item fields are shown in Table 13. This is a list contract, not an object whose missing top-level

Table 12: Schema-derived top-level fields, dimensions and literal structural restrictions for softqp-campaign schema 1.0.0. Complete nested numerical definitions are expanded in the contract appendix; physical maps remain those of the original model.

Field	Type	Constraint	Unit / role
schema_version	string	const=softqp.campaign/v1	structural metadata / nested units
protocol	object	additionalProperties=False	structural metadata / nested units
environment	object	additionalProperties=False	structural metadata / nested units
protocol_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
source_sha256	object	additionalProperties={'type': 'string', 'pattern': '^[0-9a-f]{64}$'}	structural metadata / nested units
observed_wall_seconds	number	minimum=0	s
worker_peak_rss_bytes	integer	minimum=0	bytes
worker_restarts	integer	minimum=0	count / integer identifier
planned_instances	integer	minimum=0	count / integer identifier
completed_instances	integer	minimum=0	count / integer identifier
cells	array	See defined object / value type	structural metadata / nested units
primary_endpoint	object	additionalProperties=False	structural metadata / nested units
counts	object	additionalProperties=False	structural metadata / nested units
artifacts	array	See defined object / value type	structural metadata / nested units

properties imply an empty dataset. The experiment exporter produces it; the benchmark reader and replay validator consume it. Seeds and digests identify exact generated models. Density and condition are dimensionless controlled inputs; generation duration is a reported process interval distinct from solver time. A missing list makes experiment evidence unavailable with its specific path. Duplicate or omitted instances are inconsistent with the prospective population even if every remaining entry validates separately. Oracle failure is retained rather than replacing its model with an easier seed. This inventory therefore supports denominator and compatibility checks without turning an authored synthetic case grid into an operational sampling claim.

Table 13: Schema-derived top-level fields, dimensions and literal structural restrictions for softqp-datasets schema 1.0.0. Complete nested numerical definitions are expanded in the contract appendix; physical maps remain those of the original model.

Field	Type	Constraint	Unit / role
id	string	pattern=^(development|heldout|robustness)-[0-9]{4}$	structural metadata / nested units
split	enumerated value	enum=['development', 'heldout', 'robustness']	structural metadata / nested units
n	integer	minimum=0	count / integer identifier
density	number	minimum=0	dimensionless normalized diagnostic / factor
condition	number	minimum=0	dimensionless normalized diagnostic / factor
family	enumerated value	enum=['symmetric-elastic', 'asymmetric-elastic', 'asymmetric-l1']	structural metadata / nested units
seed	integer	minimum=0	count / integer identifier
generation_seconds	number	minimum=0	s
problem_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
oracle_state	enumerated value	enum=['accepted', 'oracle-failed']	structural metadata / nested units
oracle_reason	union / composition	See defined object / value type	structural metadata / nested units
oracle_objective	union / composition	See defined object / value type	model cost unit

4.13 Original per-trial observation

The softqp-trial contract schema 1.0.0 governs each JSONL observation and its CSV projection. It retains the instance/split/family controls, method, repetition and warm-up flag, state/reason, original metrics, discrepancy, elapsed/update/setup durations, numerical result information, iterations, dimensions and backend status fields. Table 14 lists the actual field inventory. The experiment exporter produces these rows; replay and benchmark consumers preserve their identity and type. Numerical durations are seconds, matrix sizes are counts and acceptance metrics have their stated normalized or physical interpretation. An incompatible observation can carry no solved objective; absence is not encoded as a successful zero. Warm-up flags prevent initialization samples from entering the measured primary reduction. Missing raw evidence remains missing even if a summary CSV exists. Full JSONL retains diagnostics that cannot be reconstructed from selected plot points. Repetition, method ordering and reason fields are retained so a fast unsuccessful or unsupported call cannot silently enter an accuracy-qualified timing distribution.

Table 14: Schema-derived top-level fields, dimensions and literal structural restrictions for softqp-trial schema 1.0.0. Complete nested numerical definitions are expanded in the contract appendix; physical maps remain those of the original model.

Field	Type	Constraint	Unit / role
id	string	pattern=^(development|heldout|robustness)-[0-9]{4}$	structural metadata / nested units
split	enumerated value	enum=['development', 'heldout', 'robustness']	structural metadata / nested units
n	integer	minimum=0	count / integer identifier
density	number	minimum=0	dimensionless normalized diagnostic / factor
condition	number	minimum=0	dimensionless normalized diagnostic / factor
family	enumerated value	enum=['symmetric-elastic', 'asymmetric-elastic', 'asymmetric-l1']	structural metadata / nested units
seed	integer	minimum=0	count / integer identifier
method	enumerated value	enum=['lifted', 'generalized', 'asymmetric', 'daqp', 'flex', 'elastiqp']	structural metadata / nested units
repetition	integer	minimum=-1	count / integer identifier
warmup	boolean	See defined object / value type	structural metadata / nested units
state	enumerated value	enum=['accepted', 'incompatible', 'failed', 'accuracy-rejected', 'timeout', 'resource-limit', 'oracle-failed']	structural metadata / nested units
reason	union / composition	See defined object / value type	structural metadata / nested units
metrics	union / composition	See defined object / value type	structural metadata / nested units
normalized_gap	union / composition	See defined object / value type	dimensionless normalized diagnostic / factor
elapsed_seconds	union / composition	See defined object / value type	s
update_seconds	union / composition	See defined object / value type	s
workspace_setup_seconds	union / composition	See defined object / value type	s
result	union / composition	See defined object / value type	structural metadata / nested units
iterations	union / composition	See defined object / value type	count / integer identifier
dimensions	union / composition	See defined object / value type	structural metadata / nested units
native_setup_seconds	union / composition	See defined object / value type	s
native_solve_seconds	union / composition	See defined object / value type	s
status	union / composition	See defined object / value type	structural metadata / nested units
status_code	union / composition	See defined object / value type	count / integer identifier

4.14 Experimental artifact inventory

The softqp-experiment-index contract schema 1.0.0 associates the protocol identity and experiment state with the exported artifact list. The producer is the experiment export module; the static experimental loader is its consumer. Table 15 lists its schema version, protocol identifier, artifacts and state. Each artifact is a named file with a declared representation and checksum rather than an inferred availability flag. Counts and byte identities are dimensionless metadata; physical units remain in the scientific records. Missing inventory leaves experiment evidence unavailable but does not disable independent teaching cases. The browser verifies actual bytes before reading JSON or plotting the selected projection. A valid inventory cannot certify the underlying mathematical acceptance on its own: replay must also verify complete populations, numerical states and source identities. Conversely, an existing large external archive does not automatically become a declared browser artifact. This boundary keeps the small public reader's exact distributed files separate from external raw study storage and prevents accidental claims that an unimplemented adapter has loaded an archived computation.

Table 15: Schema-derived top-level fields, dimensions and literal structural restrictions for softqp-experiment-index schema 1.0.0. Complete nested numerical definitions are expanded in the contract appendix; physical maps remain those of the original model.

Field	Type	Constraint	Unit / role
schema_version	string	const=softqp.experiments/v1	structural metadata / nested units
protocol_id	string	const=softqp-experiments-v1	structural metadata / nested units
artifacts	array	See defined object / value type	structural metadata / nested units
state	enumerated value	enum=['development', 'measured']	structural metadata / nested units

4.15 Directional repeated-forecast input batch

The softqp-coupled-batch contract schema 1.0.0 is an offline research input boundary produced and consumed by the coupled contract/worker components. Its schema version, kind, description, original problem, vector updates and unit arrays appear in Table 16. The problem retains sparse matrices, hard policy, disjoint groups, positive directional scales and original physical metadata. Updates change only the admitted repeated-operation vectors while preserving the fixed structural objects. Units distinguish each decision, interval row and group peak, so an inverse-row-unit directional scale can be checked against the chosen dimensionless peak. The batch is hashed before worker execution and cross-referenced by its observations. A valid batch example does not establish an available browser replay path: that consumer is absent and readiness remains missing for this boundary. Invalid group overlap, nonpositive scales, changed hard policy or mismatched update dimensions are refused rather than coerced into a superficially feasible model. The complete external archive contains the actual synthetic batches separately from these shape examples.

Table 16: Schema-derived top-level fields, dimensions and literal structural restrictions for softqp-coupled-batch schema 1.0.0. Complete nested numerical definitions are expanded in the contract appendix; physical maps remain those of the original model.

Field	Type	Constraint	Unit / role
schema_version	string	const=1.0.0	structural metadata / nested units
kind	string	const=directional-coupled-peak-batch	structural metadata / nested units
description	string	minLength=1	structural metadata / nested units
problem	object	additionalProperties=False	structural metadata / nested units
updates	array	minItems=1; maxItems=512	structural metadata / nested units
units	object	additionalProperties={'type': 'string', 'minLength': 1}	structural metadata / nested units

4.16 Directional method observation

The softqp-coupled-observation contract schema 1.0.0 represents one oracle or method operation in the grouped study. It retains role, instance/split/seed, family/curvature/quadratic controls, method, repetition, warm-up, state/reason, complete elapsed duration, input batch digest, compatibility status, frame records, backend results, objectives and structural dimensions. The fields are shown in Table 17. The supervised worker produces the observation; the replay validator checks every returned frame in original coordinates. An elapsed duration measures preparation plus all 32 updates, solves, recovery and required acceptance calculations, not one forecast frame. An inaccurate candidate, dependent oracle failure and timeout remain distinct states. Missing primal/dual vectors are not fabricated from an objective value. No browser adapter is installed for these observations, so schema examples do not make the lane available. The batch digest, fixed method settings and complete frame count prevent transplanting an accepted solution from another instance or counting only successful frames from an otherwise failed repeated operation.

Table 17: Schema-derived top-level fields, dimensions and literal structural restrictions for softqp-coupled-observation schema 1.0.0. Complete nested numerical definitions are expanded in the contract appendix; physical maps remain those of the original model.

Field	Type	Constraint	Unit / role
schema_version	string	const=1.0.0	structural metadata / nested units
kind	string	const=directional-coupled-observation	structural metadata / nested units
role	enumerated value	enum=['oracle', 'method']	structural metadata / nested units
id	string	pattern=^(development|heldout|robustness)-[0-9]{4}$	structural metadata / nested units
split	enumerated value	enum=['development', 'heldout', 'robustness']	structural metadata / nested units
seed	integer	minimum=0	count / integer identifier
family	enumerated value	enum=['sparse', 'thermal']	structural metadata / nested units
curvature	enumerated value	enum=['positive-definite', 'semidefinite', 'zero-hessian']	structural metadata / nested units

Field	Type	Constraint	Unit / role
quadratic	number	minimum=0	cost / peak unit squared
method	enumerated value	enum=['___oracle___', 'native', 'epigraph-default', 'epigraph-fixed', 'row-split-default', 'row-split-fixed', 'daqp-direct', 'daqp-prox-auto', 'daqp-prox-forced', 'daqp-lp-model']	structural metadata / nested units
repetition	integer / null	minimum=-1; maximum=2	count / integer identifier
warmup	boolean	See defined object / value type	structural metadata / nested units
state	enumerated value	enum=['accepted', 'failed', 'accuracy-rejected', 'incompatible', 'oracle-failed', 'resource-limit', 'timeout']	structural metadata / nested units
reason	string / null	See defined object / value type	structural metadata / nested units
elapsed_seconds	number / null	exclusiveMinimum=0	s
batch_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
predeclared_ incompatible	boolean	See defined object / value type	structural metadata / nested units
frames	array	maxItems=32	count / integer identifier
backend_results	array	maxItems=32	structural metadata / nested units
objectives	array / null	maxItems=32	model cost unit
n	integer	minimum=1	count / integer identifier
group_size	integer	minimum=1	count / integer identifier
condition	number	minimum=1	dimensionless normalized diagnostic / factor
horizon	integer	minimum=1	count / integer identifier
zones	integer	minimum=1	count / integer identifier

4.17 Directional split summary

The softqp-coupled-campaign contract schema 1.0.0 summarizes each registered grouped-study split. Its environment/source maps, full instance inventory, dataset paths, measured state counts, primary fields, peak worker memory, restart count and raw/CSV hashes are listed in Table 18. The offline exporter produces the summary, and independent replay consumes it. Development, held-out and robustness populations remain separate; they cannot be pooled to replace the 144-member primary endpoint. Elapsed quantities in nested observations use seconds, resident memory uses bytes and structural sizes are counts. Split names and digests identify populations rather than statistical weights selected after timing. A browser consumer is explicitly absent, and readiness remains missing. The external validation checks all measured states, inventories and artifact hashes before accepting the summary. Rewriting a checksum alongside a changed primary number therefore does not recreate the independent evidence authority. This contract supports archival reproducibility and denominator accounting without declaring a hosted research execution service or a successful computational hypothesis.

Table 18: Schema-derived top-level fields, dimensions and literal structural restrictions for softqp-coupled-campaign schema 1.0.0. Complete nested numerical definitions are expanded in the contract appendix; physical maps remain those of the original model.

Field	Type	Constraint	Unit / role
schema_version	string	const=1.0.0	structural metadata / nested units
kind	string	const=directional-coupled-campaign	structural metadata / nested units
protocol_id	string	const=softqp-directional-coupled-v1	structural metadata / nested units
split	enumerated value	enum=['development', 'heldout', 'robustness']	structural metadata / nested units
environment	object	See defined object / value type	structural metadata / nested units
source_sha256	object	additionalProperties={'$ref': '#/$defs/hash'}	structural metadata / nested units
inventory	array	See defined object / value type	structural metadata / nested units
datasets	array	See defined object / value type	structural metadata / nested units
counts	object	additionalProperties={'type': 'integer', 'minimum': 0}	structural metadata / nested units
primary	object	See defined object / value type	structural metadata / nested units
peak_worker_rss	integer	minimum=0	bytes
worker_restarts	integer	minimum=0	count / integer identifier
raw_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
csv_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units

4.18 Isolated directional teaching exchange

The softqp-coupled-lesson contract schema 1.0.0 connects a small physical teaching input to actual native, compact-epigraph and directional row-split answers. It carries schema/SDK identity, worker fingerprint, input digest, the original input and method records; Table 19 lists these fields. The separately configured direct-CPython worker produces the result, and the teaching consumer verifies it before interpreting objectives, physical decisions, group peaks, hard errors and original KKT diagnostics. Inputs specify linear/quadratic group prices, row scaling and a directional policy scale. Row conversion is distinct from changing that policy. The source fingerprint and installed public SDK manifest prevent an unrelated local module from silently replacing the tested implementation. The exchange has bounded process time, output size and cleanup behavior. It remains unavailable to the browser because the teaching consumer is offline. Missing or mismatched results are refused, and a local shape example never serves as proof of a deployed numerical lane. This small illustration is separate from the full registered timing population and contributes no additional inference to that endpoint.

Table 19: Schema-derived top-level fields, dimensions and literal structural restrictions for softqp-coupled-lesson schema 1.0.0. Complete nested numerical definitions are expanded in the contract appendix; physical maps remain those of the original model.

Field	Type	Constraint	Unit / role
schema_version	string	const=softqp.coupled-lesson/v1	structural metadata / nested units
sdk_version	string	const=0.2.0	structural metadata / nested units
worker_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
input_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
input	object	additionalProperties=False	structural metadata / nested units
records	array	minItems=3; maxItems=3	structural metadata / nested units

5 System architecture

The system separates scientific computation from evidence presentation. Figure 1 shows the implemented flow: an original physical model enters a native numerical method, its output is evaluated independently, and a versioned export is consumed by a static reader. The reader does not compute a new optimizer by displaying an existing result. This separation keeps numerical source identity, local user execution and web distribution independently inspectable. A successful web build establishes neither the installability of a native wheel nor the scientific acceptance of every numerical result referenced by that build.

The first processing stage constructs or ingests a canonical original-space problem and physical row descriptions. Authored scalar and control cases come from deterministic generators. The source-faithful building adapter preserves its upstream mixed penalties, while separate constructors identify authored policy variants. Structural validation checks sparse dimensions, finite numeric values, interval order, units and hard/soft policy before a backend is selected.

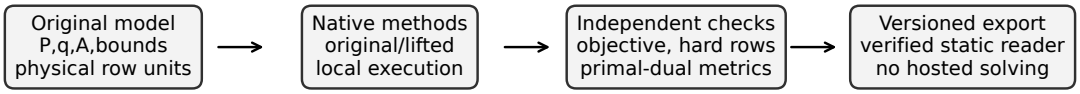


Figure 1: Implemented data flow from original physical model through native solving and independent acceptance to a versioned static reader. This is an architecture diagram, not a measured speed or reliability result.

Native method packages are owned by a separate public engine repository; the product scripts do not hide an unpublished internal solver distribution.

The method stage applies an explicit compatibility rule and runs an admitted objective using fixed settings. It records available, incompatible and failed outcomes with reasons. Prepared workspaces retain only the structure their interface permits, including matrix patterns, coefficients and hard masks. The evaluation stage recovers original coordinates, objective components, lower/upper departures and hard error; grouped repeated operations additionally retain framewise primal and dual checks. This order prevents a library success flag from being exported as scientific acceptance before original quantities have been checked.

The export stage validates the scientific documents against their schemas, writes exact finite UTF-8 data, and produces checksummed inventories and deployment manifests. Readiness is calculated from actual files and contract states. JSONL preserves nested diagnostics and unsuccessful observations; CSV provides a projection for inspection. The static frontend verifies byte identities and schema/cross-field constraints before rendering a selected case or an experimental comparison. Its unit filter avoids combining physically incommensurable departures in one raw numerical axis.

The local cost editor creates a bounded request against a canonical digest. The user explicitly runs the native scientific command outside the browser and imports its result envelope. The reader checks that the request, model, prices, hard policy and recovered metrics agree. Canonical replay and local native imports receive different provenance labels. A selected C12 stream cannot be reused after a local policy edit because its stored inputs no longer match the edited model. The optional API remains a dormant file-serving component rather than an optimization endpoint.

The grouped research and teaching profiles are separate from the ordinary public-SDK profile. Their batches, observations and split summaries have schemas and complete external evidence, but no installed browser replay adapter. The teaching worker uses direct external CPython, a public SDK fingerprint and bounded process cleanup. These boundaries permit actual native methods to be taught without claiming that their external execution has become a browser capability. No training or fitted model stage exists: the implemented methods are deterministic optimization procedures, and the unavailable learned FlexQP policy is not replaced by a fabricated checkpoint.

The distributed application has six scientific routes, but route count is not a proxy for numerical depth. Scientific authority resides in the exact inputs, implementations, protocols and accepted outputs. The deployment workflow builds committed evidence and checks its own commit identity; it does not regenerate an experimental study to refresh a version badge. This architecture enables reproducible inspection while retaining the limits of synthetic data and externally executed numerical work.

6 Methods

The executable set contains seven mathematical or implementation paths. Each description states the objective that the path preserves, its principal numerical operation, its supported domain and the evidence needed to accept its output. The comparisons use the same original-space evaluator. Backend status integers are translated individually; they are not treated as interchangeable proof of optimality. Algorithmic complexity descriptions refer to identified operations, not to a measured asymptotic law inferred from a small runtime plot.

6.1 Explicit slack lifting with OSQP

Explicit lifting converts a priced interval problem into an ordinary quadratic program by exposing finite-side departures as optimization variables. For each soft lower side introduce $t_{L,i} \geq 0$, and for each soft upper side introduce $t_{U,i} \geq 0$. Retain original hard rows without slack. The transformed bounds are

$$A_i x + t_{L,i} \geq \ell_i, \qquad A_i x - t_{U,i} \leq u_i, \qquad t_{L,i}, t_{U,i} \geq 0. \tag{7}$$

The lifted objective is

$$\tfrac{1}{2} x^\top P x + q^\top x + \sum_i \left(a_{L,i} t_{L,i} + \tfrac{1}{2} b_{L,i} t_{L,i}^2 + a_{U,i} t_{U,i} + \tfrac{1}{2} b_{U,i} t_{U,i}^2 \right). \tag{8}$$

Monotonicity of the nonnegative side prices permits an optimum whose slacks equal the actual departures. Zero prices can leave auxiliary variables nonunique, but recovering departure directly from Ax still gives the correct original objective. The evaluator therefore does not infer original violation by trusting an arbitrarily chosen unpriced lifted slack.

The sparse model construction is explicit. Original variables occupy the first block, followed by only the finite-side auxiliaries actually required. Original curvature and linear coefficients remain in their block; auxiliary diagonal curvature and linear prices populate the new entries. Every added nonnegativity bound and relaxed side has a documented position. This construction supports side-asymmetric prices without changing their meaning. Compact special helpers are used for the mixed published building objective, where treating a shared peak as independent row slacks would instead change the objective.

OSQP applies an operator-splitting iteration to the resulting convex QP and solves a regularized quasi-definite linear system [30]. Its numerical regularization is part of the solver subproblem, not an added scientific penalty in (8). Positive semidefinite problems remain admissible within the backend's convex domain. Existence of a saddle point and the relevant fixed or controlled parameter assumptions govern convergence; a finite iteration limit still produces only the method-specific termination outcome. Numerical infeasibility certificates and post-solve accuracy checks are separate operations.

Algorithm 1 describes the adapter's procedure. After solving, upper- side lifted multipliers contribute positively to the original row dual and lower- side multipliers negatively. Hard equalities retain their original dual position. The resulting x, y are passed to the original objective and feasibility routines. This recovery makes it possible to compare an enlarged problem to an original-row implementation without comparing unrelated residual definitions. A small residual in the lifted coordinates alone would not prove that the right physical model had been assembled.

Algorithm 1 Construct an equivalent finite-side lift, solve its ordinary quadratic program and recover the original-space decision and multiplier for independent evaluation.

Require: Original P, q, A, ℓ, u, hard mask and side prices
1: Allocate one nonnegative auxiliary for each present soft side
2: Assemble (7) and (8); retain hard rows
3: Solve with the pinned OSQP implementation and explicit settings
4: Recover x and $y_i = y_{U,i} - y_{L,i}$, preserving hard-row multipliers
5: Recompute original objective, departures and raw hard violation
6: Return the backend status and independent metrics, including failure reasons

The formulation cost depends on the number of finite soft sides and the sparsity of their incidence. It increases variable and row counts, but that increase does not determine factorization fill or end-to-end runtime. Sparse ordering, diagonal structure and reuse of prepared workspaces can make an enlarged problem highly competitive. For C01 the stored assembled lift has two variables, three rows and seven nonzeros; the original-row representation has one variable, one row and two nonzeros. These counts describe assembled matrices, not the nonzeros of a numeric factor or a guarantee of a particular runtime ratio.

Two uses of lifting must be distinguished. The timed original comparison uses the pinned native backend for both ordinary lifting and the candidate so backend implementation differences do not automatically favor one path. The independent high-accuracy reference uses stock OSQP with tighter tolerances. That reference is an accuracy oracle, not a timed participant substituted after seeing results. The coupled comparison separately uses the exact compact peak epigraph. Its different auxiliary structure is described with the directional method rather than conflated with independent per-side lifting.

The lift's auxiliary uniqueness deserves particular care in validation. A strictly positive quadratic departure price determines a unique optimal slack for a fixed original decision. A zero-price side can instead permit many auxiliary values with the same scientific cost. Comparing those auxiliaries componentwise to one reference would reject mathematically equivalent answers. The evaluator therefore reconstructs the smallest physical departures from the original row values, retains hard feasibility, and compares the original objective. This convention also prevents a backend's slack-storage sign from being mistaken for a physical lower or upper violation in the user-facing record.

6.2 Published generalized slack projection

Generalized slack projection removes certain explicit slack variables from an OSQP-style split by replacing the box projection with a proximal operation. The published derivation and native implementation are due to Schwan, Lew and Plancher, building on slack elimination in ADMM [29, 18]. The implementation here preserves the authors' source identity and supported penalty families. It does not equate a citation to an executed method: compatible synthetic

cases actually call the native backend and recover original-space objectives and duals.

Let C be the row interval box, Π_C its projection, and Φ the supported slack penalty. The key residual/projection construction is

$$R_C(v) = v - \Pi_C(v), \qquad T(v) = \Pi_C(v) + \mathrm{prox}_{\Phi/\rho}(R_C(v)), \quad \rho > 0, \tag{9}$$

with the corrective-slack versus violation-residual convention fixed consistently. The proximal operation itself is

$$\mathrm{prox}_{\Phi/\rho}(r) = \mathrm{argmin}_w \left\{ \Phi(w) + \tfrac{\rho}{2}\|w - r\|_2^2 \right\}. \tag{10}$$

For the published unconditional penalties, sign symmetry and contraction behavior permit the generalized projection to represent the eliminated slack problem. These are mathematical assumptions, not implementation options that can be ignored when a requested lower price differs from an upper price.

The supported published paths include symmetric quadratic, linear, elastic and selected Huber or grouped penalties. In C13 the mixed objective exercises that broader native penalty representation. By contrast, the authored side-asymmetric energy variants cannot be sent to this backend as if their coefficients were symmetric. The adapter returns an incompatible state with a reason rather than averaging the coefficients or solving a symmetric surrogate. This refusal is visible in both the method inventory and the cross-case displays.

The modified proximal operation changes the row update without necessarily enlarging the original x, z principal system. That property is useful when a problem repeatedly updates its vectors while retaining matrices and penalty structure. It does not remove the sparse system solve, guarantee invariant fill, or imply that every numerical parameter change is free. An adaptive ADMM penalty can require refactorization even when the matrix dimensions remain identical. The computational comparison consequently measures the complete specified operation rather than counting auxiliaries as a substitute for elapsed time.

Algorithm 2 separates compatibility, numerical execution and original acceptance. A prepared workspace may retain fixed matrices, a fixed hard mask, the finite-side pattern and the admitted penalty family. Only permitted vector changes are applied during repeated execution. Passing a different group layout or a different coefficient structure is not a valid vector-only update. The same restrictions apply to references and the candidate, avoiding a comparison that grants reuse only to one selected implementation.

Algorithm 2 Execute the published generalized projection only on supported penalty semantics, retaining the original row system and independent objective and feasibility recovery.

Require: Original problem, published penalty family and pinned backend
1: Check symmetry, grouping and finite-side assumptions
2: **if** the requested objective is unsupported **then**
3: Return incompatible with the precise semantic reason
4: **end if**
5: Prepare the original-row linear system and admitted penalty descriptors
6: Iterate the native solve and generalized row operation (9)
7: Recover original x, y; evaluate the requested original objective
8: Report termination, hard violation, disagreement and numerical diagnostics

Convergence follows the underlying convex splitting result only within its stated assumptions, including appropriate solution existence and numerical parameter conditions [3, 10]. The

published proximal formula is not a blanket theorem for a sign-asymmetric, variable-metric or learned-parameter extension. The separate candidate derivations state precisely which broader penalties and fixed metrics are considered. Their accuracy and convergence claims do not retroactively enlarge the domain of the authors' code.

Source attribution is substantive. The adapted native code is distributed by the separate engine; nested OSQP and QDLDL terms are retained alongside the OSQPSlack notice. The building source is reproduced with its original mixed penalties before authored variants are introduced. Preserving those distinctions allows a reader to identify what is reproduced, what is an adapter, and what is a mathematical extension. A plot that omits incompatible cells must retain their reasons elsewhere; otherwise absence can be mistaken for a successful zero-cost solution or evidence that a supported reference was never executed.

The common projection notation does not erase differences between separable and shared-group costs. With independent quadratic prices, each row operation can be evaluated separately. A shared infinity-norm price requires a joint operation because changing one row's largest departure can alter the group's active coordinates. The published native descriptors encode that grouping. The source-faithful building case exercises precisely this distinction. A consumer that merely checks the number of rows, while dropping the group descriptor, could accept a well-formed numerical vector for the wrong optimization problem. Original objective reconstruction is therefore part of the reference comparison, not an optional display calculation.

6.3 DAQP with side-specific elastic weights

DAQP uses a dual active-set strategy for convex quadratic programs and exposes side-specific soft weights through its public interface [1, 13]. This capability is important to the comparison: asymmetry alone cannot be claimed as a new modeling feature of an original-row projection method. A fair adapter must reproduce the requested finite objective rather than select the backend's default softness convention and call the result equivalent.

For a compatible elastic side with departure $t \geq 0$, the requested price is

$$\varphi(t) = at + \tfrac{1}{2}bt^2, \qquad a \geq 0, \quad b > 0. \tag{11}$$

The adapter maps the quadratic convention reciprocally where the public backend defines a softness parameter through its inverse curvature. The precise mapping is applied per side, with lower and upper coefficients kept separate. It is validated by evaluating (11) in original coordinates after the solve, rather than assuming that similarly named interface fields have the same units or normalization. Dividing by a zero quadratic price is not a valid way to request a pure linear objective.

An active set identifies constraints whose multipliers may be nonzero. For a positive-definite original Hessian, eliminating the primal vector yields a dual quadratic structure involving

$$x = -P^{-1}(q + A^\top y), \qquad AP^{-1}A^\top, \tag{12}$$

with the finite-side and elastic conventions handled by the backend. This identity explains why positive-definite curvature matters to the direct path; it is not a prescription to form dense inverses explicitly. The tested adapter uses the library interface and its maintained factors. It does not repair a semidefinite scientific model by adding an undisclosed Hessian shift.

The ordinary comparison admits DAQP when the base Hessian and finite soft-side quadratic coefficients meet the implemented positive-definite elastic domain. Pure L1 and grouped/Huber

objectives are incompatible with that particular mapping. The coupled study has separately registered direct epigraph and proximal DAQP policies, including their stated linear-programming cases. Those policies are not silently substituted into the earlier experiment. Distinct protocols make their different acceptance and termination conditions inspectable.

Algorithm 3 describes the direct side-weight adapter. The conversion has explicit size bounds of 2048 variables and 8192 finite sides or rows where applicable. These are implementation resource limits, not mathematical restrictions of convex quadratic programming. A refusal at this boundary is retained as an unavailable operation; it does not justify reporting a successful answer to a smaller or differently regularized model. Inputs that pass the size check still require semantic compatibility and original-space evaluation.

Algorithm 3 Map compatible side-specific elastic prices into DAQP, retain its active workspace for allowed vector updates and recover original-space objective and multipliers.

Require: Positive-definite admitted QP and finite-side elastic coefficients

1: Check finite dimensions, hard mask, side pattern and strictly positive quadratic prices
2: Convert each lower and upper softness convention with its physical scale
3: Prepare DAQP; retain factor and active state for compatible vector updates
4: Solve; interpret the backend's own status convention
5: Recover original dual signs and independently evaluate all original rows
6: Return accepted only if the external accuracy requirements also hold

Repeated-operation fairness requires using DAQP's documented update operation. The prepared workspace preserves its active state and updates only the allowed linear term and bounds. It does not resend a complete sense vector on every frame when the public API already permits a cheaper vector update. Conversely, changing penalty structure invalidates that reuse. The measured timing includes the selected preparation/update boundary consistently, so the comparison is not an active-set warm start against a deliberately rebuilt reference.

Backend status interpretation is method-specific. DAQP's status indicating a soft optimum cannot be read using OSQP's integer convention, where a numerically equal value may denote an inaccurate result. The adapter translates the status, then recomputes objective discrepancy and maximum raw hard violation. In the coupled study, candidate vectors also face the separate primal and dual checks. An active-set result that is fast but solves the wrong softness scaling is therefore rejected; a correct result outside the supported direct domain is not manufactured by altering the objective. These controls preserve the scientific comparison even when different backends make different engineering tradeoffs. The separately registered proximal DAQP policies address additional epigraph regimes through a proximal-point interpretation rather than a permanent scientific Hessian shift [2, 27]. Their original objective is checked after numerical recovery. This distinction matters for a zero- Hessian linear-price case: a positive quadratic term introduced only to make a direct solver callable would change the requested model. The protocol instead admits only its explicitly specified policies and retains unsupported or unsuccessful outcomes. These later grouped policies do not retroactively expand the direct adapter's compatibility in the original separable timing study, whose method settings and excluded cells remain fixed.

6.4 Fixed-parameter FlexQP reconstruction

FlexQP addresses elastic quadratic optimization with a splitting formulation; Deep FlexQP additionally uses a learned numerical-parameter policy [23]. The implemented reference recon-

structs the fixed equations and Algorithm 1 from the paper. It is neither the authors' released implementation nor a reproduction of a learned policy. Trained weights, training data and a reproducible policy implementation were unavailable within the comparison. The fixed reconstruction is nevertheless executable on its stated domain and is evaluated against the same independent lifted original objective.

The admitted model has positive finite-side linear prices and no finite hard rows or quadratic soft costs. After converting finite sides to inequalities, the elastic problem can be written as

$$\min_{x,s\geq 0} \ \tfrac{1}{2}x^\top Px + q^\top x + w^\top s \quad \text{subject to} \quad Gx - s \leq h, \qquad w > 0. \tag{13}$$

Lower interval sides become signed inequalities with their own prices. This conversion preserves directionality, but it does not justify giving a hard physical row a finite weight. The adapter consequently refuses hard rows rather than choosing a numerically large weight and calling it exact hardness.

The fixed numerical parameters are $\sigma_x = 10^{-6}$, $\sigma_s = 1$, $\rho = 1$ and relaxation $\alpha = 1.6$. Eliminating the slack-related block produces the Schur system

$$K = P + \sigma_x I + cG^\top G, \qquad c = (\sigma_s^{-1} + \rho^{-1})^{-1}. \tag{14}$$

The positive proximal terms stabilize the numerical subproblem. They do not modify the scientific objective in (13); treating σ_x as an objective regularizer would change the target optimizer. This distinction is checked through original objective evaluation and stationarity, not through a comparison of the stabilized subproblem's objective.

The implementation uses Python and SciPy iteration around its prepared factors. It is less mature and has different language-level overhead from the compiled C backends. Observed runtime differences therefore measure these concrete implementations under the protocol, rather than an implementation-independent lower bound on the FlexQP formulation. The reconstruction is not retuned after examining the held-out results. In particular, the comparison does not choose a different proximal parameter for each unfavorable test instance.

Algorithm 4 states the executable boundary without conflating fixed and learned variants. The original dual of (13) has finite-side bounds determined by the linear prices. The implementation checks stationarity and the projected dual-box residual rather than accepting only a small change between successive iterates. A stable iteration can converge to the wrong fixed point if the sign or price mapping is incorrect; original-space conditions are needed to distinguish that defect from a genuinely small step.

Algorithm 4 Execute the fixed FlexQP equations on an admitted pure linear elastic problem, preserving original prices and independently testing the returned original-space optimum.

Require: Compatible pure linear elastic inequalities and fixed numerical parameters

1: Refuse finite hard rows, quadratic prices and nonpositive admitted side prices
2: Convert finite interval sides into $Gx - s \leq h$ with original linear prices
3: Prepare (14) using the fixed proximal parameters
4: Iterate the paper's fixed splitting and relaxation updates
5: Check original stationarity and projected dual-box residuals
6: Recover interval dual signs and compare the original objective to lifting

The relationship between a finite elastic optimum and hard feasibility requires care. If the original hard inequalities are feasible and suitable multiplier bounds are known, a sufficiently

large exact linear price can recover a hard solution. That result does not make every finite price exact, nor does it make the output safe for a plant whose immutable limits have been softened. The application explicitly describes the objective being priced and separates it from the original hard mask. This is a modeling and acceptance requirement, not a criticism unique to one elastic algorithm.

Compatibility exclusions remain part of the scientific denominator. Symmetric and asymmetric elastic families with nonzero quadratic side prices are outside this pure linear reconstruction. They are marked incompatible before timing, not counted as inaccurate solves or assigned zero durations. Conversely, admitted asymmetric linear cases are genuinely executed and checked, so an unavailable learned policy is not used to excuse omission of the fixed published reference. The complete six-path original comparison can therefore expose both semantic limits and implementation costs without pretending that all methods solve every penalty family. Within a prepared fixed-policy operation, the signed inequality matrix and its numerical factor can be retained while admitted vectors change. A new hard mask, finite-side pattern or price structure is not such a vector change. The reconstruction's boundary checks distinguish these cases before reuse. This is also why the complete-call timing is more informative than an isolated triangular solve: it includes the work required to admit and recover the actual scientific problem. The theoretical proximal regularizers and the practical implementation overhead must both remain visible when interpreting a comparison with a mature compiled backend.

6.5 ElastiQP for pure linear elastic inequalities

ElastiQP provides a published dual active-set solver for elastic quadratic problems motivated by constrained robot control [21]. The product consumes its pinned public version 0.2.0 implementation as a separate comparison path. Its supported domain is not identified merely by the word elastic. The adapter admits positive-definite base curvature, pure linear finite-side prices, and hard equalities. It refuses quadratic side prices and hard inequalities that the implemented interface cannot preserve exactly.

After finite-side conversion, the mathematical problem is

$$\min_{x} \ \tfrac{1}{2}x^\top Px + q^\top x + \sum_j w_j(G_j x - h_j)_+ \quad \text{subject to} \quad Ex = e, \qquad P \succ 0, \quad w_j > 0. \tag{15}$$

A lower interval side becomes the signed row $-A_i x \le -\ell_i$ with its lower-side price. An upper side retains $A_i x \le u_i$ and its upper-side price. Equality rows are placed in E, rather than replaced by two finite linear penalties. This construction makes the physical hard/soft distinction recoverable after solving.

The elastic inequality multipliers lie in a bounded dual box. Conditional on an active dual configuration, the primal/equality calculation satisfies

$$\begin{bmatrix} P & E^\top \\ E & 0 \end{bmatrix} \begin{bmatrix} x \\ \nu \end{bmatrix} = \begin{bmatrix} -q - G^\top \mu \\ e \end{bmatrix}, \qquad 0 \le \mu \le w. \tag{16}$$

The expression explains the distinction between capped elastic multipliers and equality multipliers; it is not an instruction to build or invert this matrix independently of the library. The backend maintains the numerical structures appropriate to its algorithm. The adapter's task is to preserve the input model and recover the original coordinates and signs.

Algorithm 5 describes that boundary. In particular, a backend termination flag is interpreted using ElastiQP's own convention before the external evaluator recomputes the requested objective.

Positive-definite curvature is checked rather than created through an undisclosed Hessian shift. A problem whose optimum is changed by a small regularizer is not a matched reference, even if the regularizer seems numerically harmless in a particular example. Likewise, a hard inequality is not silently converted to a costly elastic preference just because the original hard model is feasible.

Algorithm 5 Preserve admitted pure linear interval prices and hard equalities in ElastiQP, then recover original row multipliers and independently evaluate the scientific objective.

Require: Positive-definite original model with admitted pure linear soft sides

1: Refuse quadratic prices, unsupported hard inequalities and conversion limits
2: Form signed finite-side inequalities and retain hard equalities separately
3: Call the pinned ElastiQP implementation with explicit numerical settings
4: Recover x, finite-side multipliers and hard-equality multipliers
5: Combine lower and upper multipliers in original row order
6: Evaluate original objective, raw hard violation and oracle discrepancy

The conversion shares explicit implementation limits with the other dense reference adapters: admitted dimensions are bounded before numerical allocation. These limits are recorded as resource restrictions and do not imply that larger mathematical problems are ill-defined. Memory behavior depends on the converted finite-side count, equality structure and maintained active-set data. A runtime comparison must therefore preserve the conversion cost and the chosen timing boundary. Comparing only a selected inner iteration to another backend's full validation/export call would answer a different question.

The published always-feasible elastic formulation concerns feasibility of the relaxed optimization representation. It does not mean every original operating inequality holds at a finite-price optimum. For example, if satisfying a comfort limit costs more in the base objective than the declared linear departure price, the optimizer may purchase a violation. Hard equality satisfaction remains a separate requirement. The application displays raw departure and hard-error quantities so an elastic status is not mistaken for a certificate of physical admissibility.

In the original synthetic timing study, the pure linear family is the natural compatible domain for this reference. The quadratic elastic families remain incompatible rather than being rerun with their quadratic coefficient removed. This produces a heterogeneous but explicit method-case matrix. The comparison does not infer that one reference is universally stronger from a count of compatible cells; each compatibility rule follows the actual represented objective. The data retain excluded cells, backend failures and accuracy checks as different states, enabling a reader to determine whether an apparent runtime difference is numerical or simply a change in the problem being solved. The bounded elastic dual also explains a common modeling mistake. Reaching the upper multiplier price cap can be consistent with a positive departure at the optimum; it is not necessarily a failed active-set calculation. The price cap belongs to the chosen objective. Equality multipliers are not capped by that same finite-side price, because their constraints remain hard. Recovering those two multiplier classes separately prevents an apparently small aggregate residual from concealing softened dynamics. The exported original-space state therefore records policy, objective and feasibility together rather than presenting one generic elastic success label.

6.6 Side-specific interval proximal update

The side-specific projection path evaluates the separable penalty (4) inside an original-row OSQP-style iteration. Its narrow implementation property is that lower and upper linear/quadratic coefficients enter the row operation without adding independent departure variables to the principal system. Unequal side costs already have established formulations, including the DAQP interface. The question tested by the original experiment is computational cost against equivalent lifting, not whether unequal prices can be modeled for the first time.

For a temporary scalar value v and positive numerical metric ρ, the row operation solves

$$z^+ = \operatorname{argmin}_z\{g_i(z) + \tfrac{\rho}{2}(z-v)^2\}. \tag{17}$$

It has the closed form

$$z^+ = \begin{cases} \ell_i - \dfrac{[\rho(\ell_i - v) - a_{L,i}]_+}{\rho + b_{L,i}}, & v < \ell_i, \\ v, & \ell_i \le v \le u_i, \\ u_i + \dfrac{[\rho(v - u_i) - a_{U,i}]_+}{\rho + b_{U,i}}, & v > u_i. \end{cases} \tag{18}$$

An absent side is omitted. A hard row uses exact interval projection. The positive denominator follows from $\rho > 0$ and nonnegative quadratic price; the formula remains defined for a pure linear or a zero-price soft side.

To derive the upper branch, write $d = z - u_i > 0$. Differentiating the exterior quadratic yields $(\rho + b_{U,i})d = \rho(v - u_i) - a_{U,i}$. If its right-hand side is nonpositive, the exterior optimum is inadmissible and the endpoint satisfies the subgradient condition. Otherwise division gives the positive departure. The lower branch follows by reflecting the displacement and using the lower coefficients. Inside the interval the price is zero and $z = v$ minimizes the proximal displacement. Strong convexity of (17) establishes uniqueness, including at the interval endpoints.

This scalar derivation is an application of established proximal convex analysis [24, 20, 27]. It is not a proof of arbitrary adaptive ADMM behavior, a finite-precision error theorem or a global priority claim. Within a fixed positive metric and an attained primal and dual solution, the underlying splitting framework supplies the relevant convergence argument. Unit conversion must transform the side prices, row coefficients and dual coordinates consistently. A positive absolute tolerance in physical departure units cannot by itself establish covariance across metres and centimetres.

Algorithm 6 identifies the actual native procedure. The separate engine owns the C/C++ update, Python bindings, scaling and objective recovery. The product consumes the public SDK in an isolated profile. Grouped and Huber costs do not become supported merely because their scalar notation resembles an elastic price. The separable adapter refuses them; the mixed C13 model uses the published generalized projection or exact lifting instead.

The implementation must preserve more than the scalar minimizer. Original objective evaluation, the dual conjugate, scaling, recession behavior and fixed-shape repeated updates have independent controls. A stationary vector can still violate the soft subgradient relation. The diagnostic proximal test $z = \operatorname{prox}_g^{H_0}(z + H_0^{-1}y)$ checks that relation at the original row value rather than selecting a derivative arbitrarily at a kink. Here H_0 is an explicitly positive diagonal diagnostic metric; its numeric identity applies only in canonical diagnostic coordinates. Signed slack cancellation is prevented by separate lower and upper departure arrays.

Algorithm 6 Apply side-specific scalar proximal updates in the original interval rows while preserving hard policy and independently recovering the requested objective and dual conditions.

Require: Admitted separable original problem, side prices and positive row metric
1: Prepare the fixed-shape original-row native workspace
2: **for** each row-update operation **do**
3: Project hard rows; apply (18) to each admitted soft row
4: Continue the native primal/dual splitting update
5: **end for**
6: Recover original x, y and physical lower/upper departures
7: Evaluate the original objective and method-specific termination criteria
8: Compare accepted outputs to the independent equivalent lift

The original held-out endpoint does not establish the registered computational gain. Its median relative reduction is 0.499839%, with a confidence interval crossing zero. Correctness and retained original dimensions therefore coexist with a negative timing conclusion. A favorable selected family, one small scalar matrix or a quick warm update cannot replace that primary result. The implementation remains a complete executable method within its supported domain; its scientific limitations are stated through the full comparison, not hidden by relabeling the failed prediction as a different acceptance rule. Endpoint behavior supplies useful concrete controls. A sufficiently large linear price relative to one temporary proximal displacement maps that displacement exactly to the preferred endpoint. This is a statement about one proximal subproblem, not a guarantee that a finite price enforces the original row for every possible model or iteration. Setting the quadratic price to zero leaves a genuine one-sided soft-threshold operation. Setting both prices to zero leaves a free preference, while an explicit hard mask still projects exactly. Confusing these three cases changes the optimization policy even though each resulting program can remain convex and solvable. The scalar analytic cases and independently recovered original dual signs test these distinctions without relying on matching backend status integers or comparing an auxiliary representation to itself.

6.7 Directional disjoint group-peak proximal update

The second proximal path prices the largest weighted departure within each disjoint group, using (6). It permits unequal upper and lower scales and keeps the original interval-row representation in its principal system. Existing directional row splitting and compact epigraphs express the same objective, so row preservation is a representation property rather than an expressibility or priority result. Coordinatewise orthant monotonicity and weighted threshold calculations have established antecedents [5, 25].

For a box C and residual penalty Ψ, assume Ψ is proper, closed, convex, nonnegative, minimized at zero, and nonincreasing under coordinate contractions $D = \operatorname{diag}(d_i)$ with $0 \le d_i \le 1$. For fixed diagonal $H \succ 0$, the exact identity is

$$\operatorname{prox}^H_{g_C}(v) = \Pi_C(v) + \operatorname{prox}^H_{\Psi}(v - \Pi_C(v)), \qquad g_C(z) = \inf_{s \in C} \Psi(z - s). \tag{19}$$

The proof clips each candidate to the coordinate segments between $\Pi_C(v)$ and v. Both diagonal quadratic distance and the contraction-monotone penalty cannot increase. On those segments the problem is exactly the residual proximal problem. A dense positive metric does not support that coordinatewise argument; the theorem is not silently extended to arbitrary metric coupling.

For one directional group let $r = v - \Pi_C(v)$, choosing c_i from the active sign of each residual. A fixed candidate peak $t \geq 0$ gives

$$w_i(t) = \operatorname{sign}(r_i)\min\{|r_i|, t/c_i\}, \qquad F(t) = at + \tfrac{1}{2}bt^2 + \tfrac{1}{2}\sum_i h_i(|r_i| - t/c_i)_+^2. \tag{20}$$

Sort the breakpoints $c_i|r_i|$ in descending order and examine a consistent active prefix S. Within that interval, a stationary threshold satisfies

$$t = \frac{\sum_{i\in S} h_i|r_i|/c_i - a}{b + \sum_{i\in S} h_i/c_i^2}. \tag{21}$$

The zero endpoint and empty active set are handled explicitly. If both prices vanish, the residual is returned unchanged; no zero denominator is evaluated. Sorting costs $O(k \log k)$ for a group of k rows, followed by a prefix scan. This extra proximal work can outweigh savings from a smaller principal system.

Algorithm 7 states the native operation. The local code refuses nonrepresentable arithmetic and uses local floating-point spacing at threshold boundaries. That treatment is important for row-scale covariance: a fixed absolute floor in departure units would change its meaning under a reciprocal scale transformation. Group membership remains disjoint in the implemented interface. Overlapping groups would require a different proximal calculation and are rejected rather than approximated as independent groups.

Algorithm 7 Compute an exact directional group proximal operation in a fixed positive diagonal metric, with explicit free-price, threshold-endpoint and representability handling.

Require: Nonempty admitted interval box, disjoint groups and positive directional scales
1: Compute $p = \Pi_C(v)$ and $r = v - p$; retain hard rows separately
2: **for** each soft group **do**
3: **if** $a = b = 0$ **then**
4: Keep the residual unchanged
5: **else**
6: Sort the finite breakpoints $c_i|r_i|$ in descending order
7: Find the consistent threshold interval using (21) and $t = 0$
8: Set the residual coordinates using (20)
9: **end if**
10: **end for**
11: Return $p + w$ and continue fixed-metric relaxed splitting

The joint dual condition differs from independent coordinate price caps. The directional dual quantity is a sum of positive and negative multiplier parts divided by their corresponding scales. Its conjugate yields one group budget. The evaluator therefore checks every group's normalized Fenchel relation and every hard row's complementarity separately, alongside stationarity and bounded dual repair. A signed aggregate gap is retained diagnostically but cannot allow an erroneous soft subgradient to cancel a negative hard-row gap.

The fixed-metric relaxation scope is $0 < \alpha < 2$ with the stated convexity and attained-saddle-point assumptions. No variable-metric or arbitrary adaptive- relaxation theorem is claimed. The complete repeated-forecast comparison uses public SDK 0.2 and includes matched row-split, compact epigraph and direct/proximal DAQP policies. Its primary score is -212.449875%,

including all failed pairs under the prespecified rule. That result refutes the registered timing prediction on the tested population. It cannot be converted to a uniform slowdown multiplier, nor rescued by omitting oracle limits, unsuccessful native operations or semantically incompatible reference cells. The physical scale controls also separate two superficially similar edits. Multiplying an original row and its bounds by a positive conversion factor, while dividing both directional scales by that factor, preserves the peak and physical optimizer. Multiplying only the upper directional scale changes the relative policy price. The teaching example intentionally exercises both operations and compares actual native, row-split and epigraph answers. Its hard coupling equality remains exact while the selected physical allocation can change under the policy edit. This demonstrates the model's semantics; it is not an additional performance experiment. The same distinction is enforced in the research batch's unit arrays and source digests so that a recorded scale conversion cannot silently become a new optimization objective. Keeping those arrays attached to the original rows is essential when the reference expands their representation into signed one-sided constraints. The expanded reference retains a recoverable mapping to each original lower or upper side, so its dual comparison tests the same physical quantity.

7 Experimental protocol

The teaching catalog, original timing study, repeated-control stream and grouped repeated-forecast study answer different questions. They are not pooled into one statistical sample. Catalog cases establish semantic and numerical examples; the two registered studies test separately specified computational predictions; the stream describes reuse on a common physical trajectory. Benchmark methodology requires an explicit problem population, consistent success criteria and comparable timing boundaries [8, 14]. A solver ranking obtained after removing unsupported or unsuccessful cells would otherwise mix representability, numerical reliability and speed.

The original main grid uses dimensions 32,64,128 and 256, twice as many rows as variables, densities 0.05 and 0.2, and diagonal Hessian conditions 1 and 1000. Its three families are symmetric elastic, asymmetric elastic and asymmetric linear. Development seeds 1000–1002 and evaluation seeds 9000–9005 are distinct. The 24-instance robustness calculation uses dimension 32, density 0.1, six additional seeds and conditions 1,100,10000 and 1000000 while keeping the other generated quantities fixed within each seed. Together these populations contain 456 instances. Rows are normalized before the declared cost/target distribution is applied; independent unit and scaling examples are separate from a claim that this grid covers arbitrary physical row scaling.

Each original backend has one warm-up and seven measured repetitions. Method order is randomized by the fixed ordering seed within an instance and repetition. The cold end-to-end boundary includes input validation, formulation construction, setup, solve and original evaluation. It excludes generation, module import and export. The primary population is the asymmetric elastic family with dimension at least 128 and density 0.05. Its 24 distinct problems and seven repetitions produce 168 paired observations across six evaluation seed clusters. Calling those 168 observations independent generated instances would be incorrect. The point estimate is the median paired reduction

$$R_j = 1 - \frac{T_{\mathrm{candidate},j}}{\min(T_{\mathrm{lift},j}, T_{\mathrm{DAQP},j})}, \qquad \widehat{R} = \mathrm{median}_j\, R_j. \tag{22}$$

The stored 95% interval resamples seed clusters with the fixed 2000-resample procedure. It is not recomputed for the figures. Cluster resampling preserves dependence among repeated

observations from the same generated seed; bootstrap accuracy with few clusters remains a limitation [11, 7, 4]. The prediction requires a reduction of at least 10% and a lower interval endpoint above zero, together with complete original-space acceptance.

Original numerical settings are absolute/relative solver tolerances of 10^{-7}, an iteration limit of 200000, and adaptive penalty checks at interval 25. The separate stock lifting oracle uses tolerances of10^{-9}. External acceptance requires normalized objective discrepancy and maximum raw hard-row departure no larger than10^{-6}. Stationarity remains diagnostic in that protocol. Threading is limited to one BLAS thread; one worker is supervised with a 10 second trial deadline, a 7200 second total limit and a 4 GiB process-memory limit. These are protocol settings, not observed per-method resource costs.

The C12 stream contains 32 frames of a horizon 20 double-integrator model with a 0.2 second physical step. An independent stock-lift oracle drives the common plant, with velocity impulses at frames 8 and 20. Every method receives the same physical state. Cold calls include preparation; warm calls record initial workspace setup separately from validation, vector update, solve and evaluation. There is one documented observation per frame. The stream supports descriptive reuse comparisons, not a statistical superiority claim or a comparison of independently controlled plants.

The grouped study retains 48 development,144 held-out and 18 robustness instances. Each operation includes preparation/factorization and 32 updates, solves, physical recovery and independent accuracy checks. Library/process startup and serialization are excluded symmetrically from the inner timer; all compiled competitors are preloaded. The outer supervisor still retains startup, deadlines, process memory and cleanup outcomes. Nine method policies have one warm-up and three measured repetitions per instance, plus an oracle observation, yielding 7770 raw and 5670 measured method observations. The primary population contains all 144 held-out instances and six seed clusters. Its per-instance score compares the native median with the fastest compatible registered reference; failed pairs retain the specified score of-1. An absent native median or failed oracle cannot be replaced with a favorable successful subset. The registered minimum improvement and interval condition remain false regardless of which descriptive formulation plot is displayed.

8 Results

The results distinguish exact formulation agreement, availability, numerical acceptance and elapsed-time inference. Tables 20 and 21 report every stored case/method outcome, including incompatible cells. Objectives are compared within one unchanged model; their numerical values are not ranked across different physical objectives. Hard errors are evaluated on original rows, and N/A denotes the absence of a compatible returned objective. The complete six-path timing comparison and separate nine-policy grouped study have their own larger inventories.

Figure 2 isolates the scalar lower-side example. The actual exported optima agree on a purchased lower departure rather than on the preferred interval itself. Its piecewise-quadratic objective curve is an evaluation of the stored mathematical objective, not an extra numerical experiment. The shaded interval identifies preference, not a physical hard bound. This small example illustrates why an elastic optimum can be correct while violating the preferred row. Figure 3 shows the exported variable counts across all fifteen cases. Larger or smaller principal formulations do not directly measure factorization fill, cache behavior or complete-operation cost.

Table 20: Complete stored catalog outcomes for the indicated cases and every catalog method. N/A means no compatible returned objective, and hard error is evaluated in original row coordinates.

Case	Method	State	Objective	Hard error	Gap
C01	lifted	available	-2.98167	0	0
C01	generalized	available	-2.98167	0	0
C01	asymmetric	available	-2.98167	0	0
C01	daqp	available	-2.98167	0	4.44089e-16
C02	lifted	available	-2.98167	0	0
C02	generalized	available	-2.98167	0	0
C02	asymmetric	available	-2.98167	0	0
C02	daqp	available	-2.98167	0	4.44089e-16
C03	lifted	available	-2.564	0	0
C03	generalized	incompatible	N/A	N/A	N/A
C03	asymmetric	available	-2.564	0	4.44089e-16
C03	daqp	available	-2.564	0	4.44089e-16
C04	lifted	available	-4.14062	7.33857e-13	0
C04	generalized	incompatible	N/A	N/A	N/A
C04	asymmetric	available	-4.14063	6.95773e-10	9.57696e-11
C04	daqp	available	-4.14062	0	1.01252e-13
C05	lifted	available	-4.5	0	0
C05	generalized	available	-4.5	0	2.66454e-15
C05	asymmetric	incompatible	N/A	N/A	N/A
C05	daqp	incompatible	N/A	N/A	N/A
C06	lifted	available	-30.6139	0	0
C06	generalized	incompatible	N/A	N/A	N/A
C06	asymmetric	available	-30.6139	0	3.66003e-07
C06	daqp	available	-30.6139	0	8.1416e-07
C07	lifted	available	-34.5757	0	0
C07	generalized	incompatible	N/A	N/A	N/A
C07	asymmetric	available	-34.5757	0	1.09511e-07
C07	daqp	incompatible	N/A	N/A	N/A
C08	lifted	available	-2.64164	0	0
C08	generalized	incompatible	N/A	N/A	N/A
C08	asymmetric	available	-2.64164	0	1.24144e-08
C08	daqp	available	-2.64164	0	1.32642e-08

Table 21: Complete stored catalog outcomes for the indicated cases and every catalog method. N/A means no compatible returned objective, and hard error is evaluated in original row coordinates.

Case	Method	State	Objective	Hard error	Gap
C09	lifted	available	-34.0702	0	0
C09	generalized	available	-34.0702	9.42717e-08	2.83169e-07
C09	asymmetric	available	-34.0702	9.42717e-08	2.83169e-07
C09	daqp	available	-34.0702	8.88178e-16	6.51837e-09
C10	lifted	available	-31.3247	0	0
C10	generalized	available	-31.3247	1.82809e-07	8.24346e-07
C10	asymmetric	available	-31.3247	1.82809e-07	8.24346e-07
C10	daqp	available	-31.3247	1.77636e-15	1.03659e-08
C11	lifted	available	3.89929	0	0
C11	generalized	incompatible	N/A	N/A	N/A
C11	asymmetric	available	3.89929	0	4.44089e-16
C11	daqp	available	3.89929	0	0
C12	lifted	available	-41.204	1.90152e-10	0
C12	generalized	incompatible	N/A	N/A	N/A
C12	asymmetric	available	-41.204	4.21994e-10	4.61511e-08
C12	daqp	available	-41.204	8.88178e-16	6.16883e-08
C13	lifted	available	36.6605	6.53606e-10	0
C13	generalized	available	36.6605	7.52559e-09	1.61722e-08
C13	asymmetric	incompatible	N/A	N/A	N/A
C13	daqp	incompatible	N/A	N/A	N/A
C14	lifted	available	48.403	2.37126e-09	0
C14	generalized	incompatible	N/A	N/A	N/A
C14	asymmetric	available	48.403	2.09591e-08	2.54563e-06
C14	daqp	incompatible	N/A	N/A	N/A
C15	lifted	available	42.6822	1.43467e-09	0
C15	generalized	incompatible	N/A	N/A	N/A
C15	asymmetric	available	42.6822	2.72591e-09	2.17369e-06
C15	daqp	incompatible	N/A	N/A	N/A

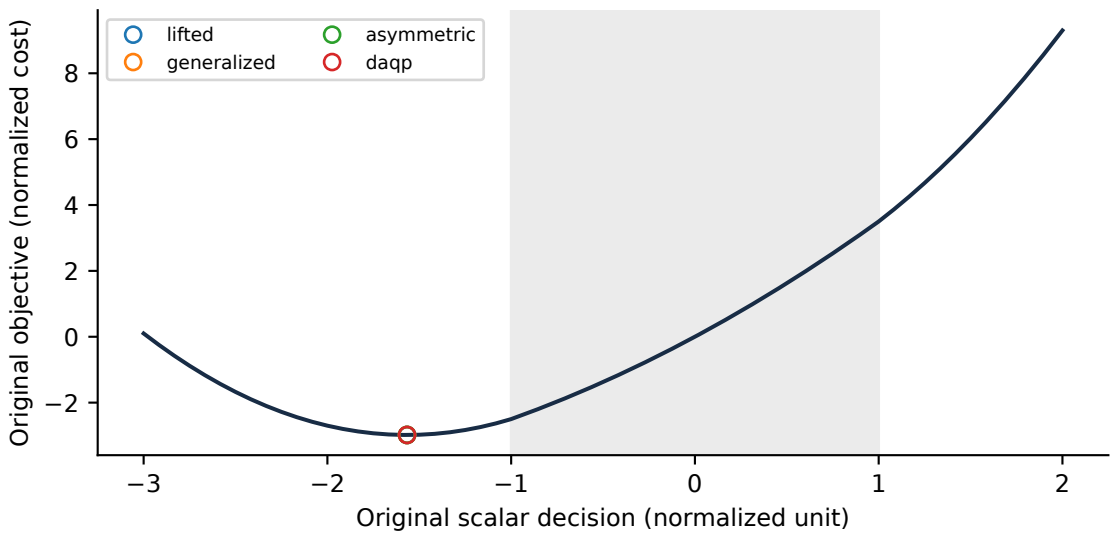


Figure 2: Analytic evaluation of the stored C01 objective and actual exported method optima. The shaded interval is the preferred band, and the curve is a mathematical illustration rather than a new timed observation.

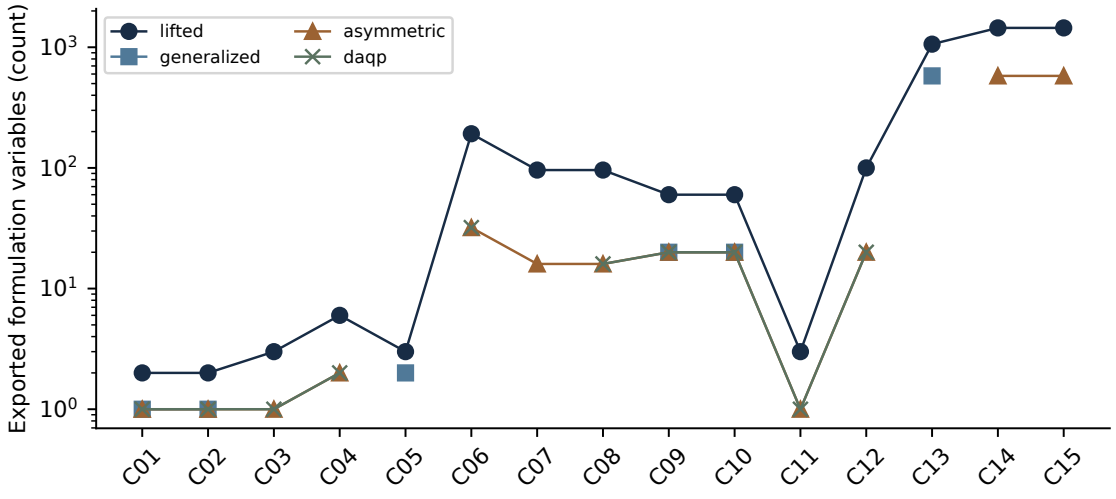


Figure 3: Exported variable counts for all fifteen stored catalog cases and four catalog paths. Missing points denote unsupported objectives; the counts describe formulation size and do not measure factorization fill or computational advantage.

Figure 4 makes the catalog's semantic exclusions explicit. The source-faithful building objective admits lifting and published generalized projection while refusing unsupported separable substitutions. The asymmetric energy variants instead define different prices, so their stored objectives cannot be read as algorithmic improvements over the mixed-policy optimum. Likewise, an incompatible symmetric projection on the asymmetric C12 stream is not a failed numerical solve of the same admitted objective. The matrix distinguishes these states before any runtime reading is attempted.

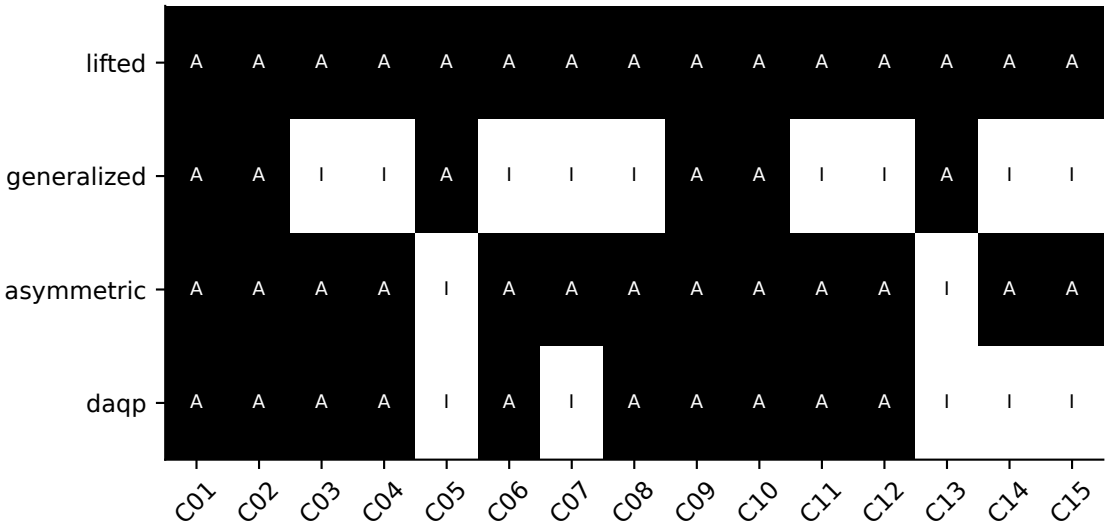


Figure 4: Full catalog method-case compatibility: A denotes an available exported solution and I an incompatible requested objective. This four-path catalog is distinct from the six-path original timing study and the separate grouped study.

The original computation completed all 456 planned instances with 11592 accepted and 7560 incompatible measured observations and no numerical or external-accuracy rejection. Table 22 preserves its primary 168 paired observations and the separate grouped endpoint. The original reduction 0.499839% has a 95% interval crossing zero. Figure 5 therefore does not establish the predicted 10% gain. The plotted zero and ten-percent reference lines are inferential context, not replacement cutoffs chosen after observing the estimate. Figure 6 shows stored cell medians and interquartile ranges for one prespecified family/density slice; that descriptive slice does not replace the full primary analysis.

Table 22: Distinct primary populations, stored effect estimates and uncertainty for the two completed timing studies. Failed-pair rules are not transferred between experiments and neither registered ten percent improvement criterion is established.

Study	Population	Pairs	Effect (%)	95% limits
Separable	456	168	0.499839	-3.29197, 3.94484
Grouped	210	144	-212.45	-225.337, -192.874

The controlled conditioning results in Figure 7 retain maximum objective discrepancy for each compatible robustness cell. The finite grid extends to diagonal condition10^6, and its accepted

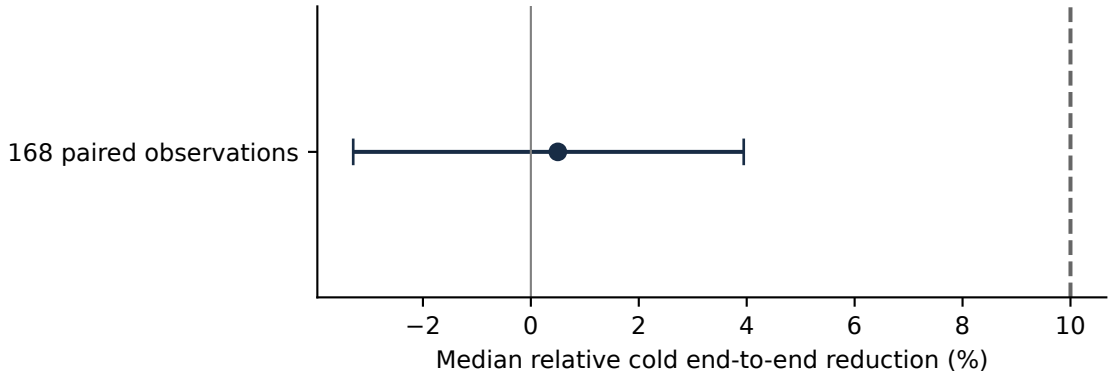


Figure 5: Original held-out timing estimate with its stored 95 percent seed-cluster confidence interval and registered ten percent reference. The interval crosses zero; neither interval nor population is recomputed for this figure.

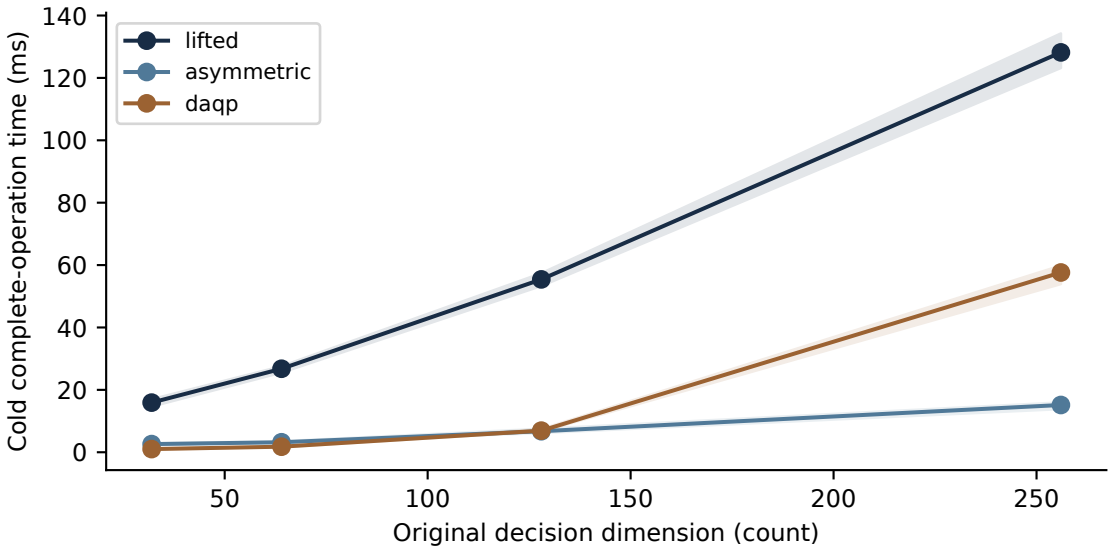


Figure 6: Stored held-out asymmetric-elastic timing cells at density 0.05 and unit diagonal condition. Lines are stored medians and bands stored interquartile ranges; this descriptive subset does not replace the complete primary endpoint.

observations remain within the original objective and hard-row limits. The plot does not establish stability for arbitrary nearly dependent rows, every semidefinite problem, or an untested physical scaling. A missing method remains absent, not an invented zero-error point. Relative objective agreement also does not prove an identical dual vector when the mathematical dual is nonunique.

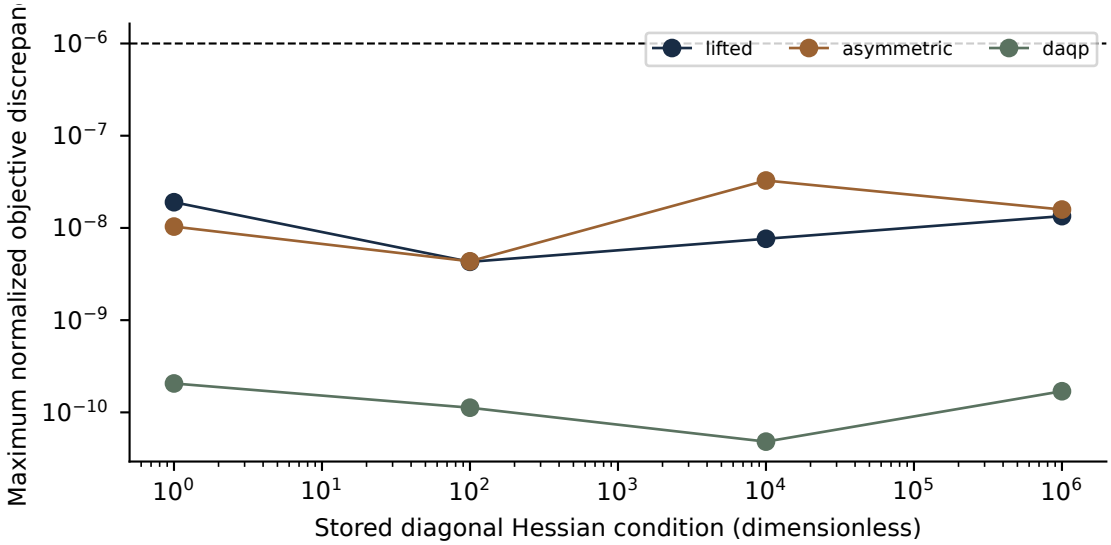


Figure 7: Maximum stored objective discrepancy in each compatible robustness cell by diagonal condition. The dashed line is the original external objective criterion; absent methods remain absent rather than receiving an invented zero discrepancy.

The common trajectory in Figure 8 shows the actual stored position, velocity and bounded applied acceleration. It is driven once by the independent oracle. Figure 9 compares stored cold and warm operation durations along those same inputs. Workspace reuse can reduce preparation work, but the stream alone cannot attribute every timing difference solely to an active set, initial iterate or cached factor. The plotted box summaries describe the retained observations; they are not a new cluster-bootstrap experiment. Overlapping predicted horizon costs remain distinct from realized plant cost.

The grouped computation retains all 210 inputs. Table 23 and Figure 10 account for all 5670 measured method observations:3667 accepted,12 accuracy-rejected,1719 incompatible,270 dependent oracle-failed and two timeouts. The larger raw inventory additionally retains warm-ups and oracle observations. All 163390 returned candidate frames pass their applicable original-space checks;44 noncandidate backend-limit frames remain distinct. Those frame checks do not turn an iteration-limited batch into an accepted primary operation.

The accuracy-rejected category must not be read as twelve returned vectors violating the independent KKT calculation. The retained batches include backend/iteration-limit or absent-vector outcomes; in particular, six held-out row-split batches returned status seven without a candidate vector. The two timeouts occurred in native and forced-proximal DAQP operations. Six robustness oracle limits account for the dependent 144 measured oracle-failed rows in that split. These are operation and dependency states, not additional independent KKT violations. Every returned candidate frame still passed the applicable original-space checks.

Figure 11 retains all 144 primary members in their stored order. Nine failed pairs keep

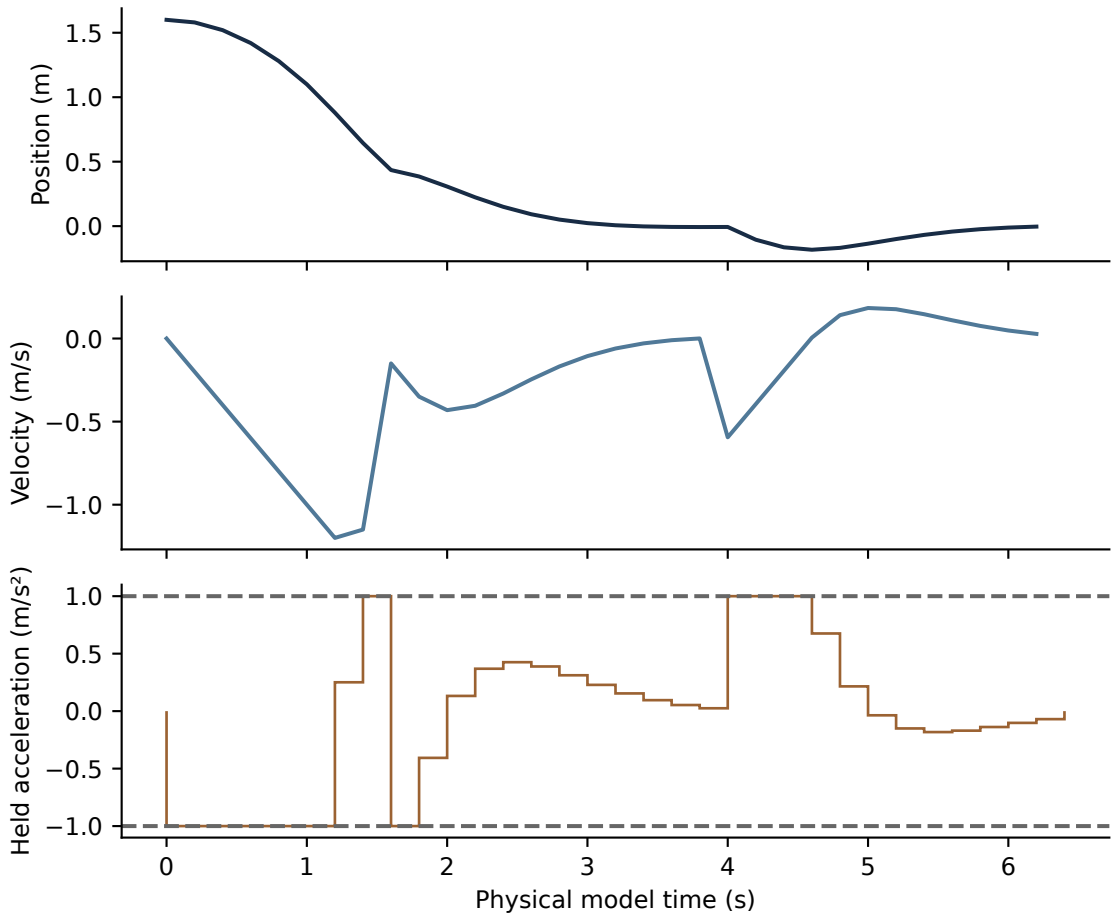


Figure 8: The stored common C12 plant trajectory and interval-held oracle acceleration across thirty-two frames. Steps retain each complete physical sampling interval; these curves are not separate closed-loop controller performance measurements.

Table 23: Measured grouped-study method states by split, including dependent oracle failures and supervised timeouts. Counts cover all measured method rows, while warm-up and oracle observations remain in the larger raw archive.

Split	Accepted	Accuracy	Incompat.	Oracle	Timeout
development	879	0	396	21	0
heldout	2587	6	1188	105	2
robustness	201	6	135	144	0

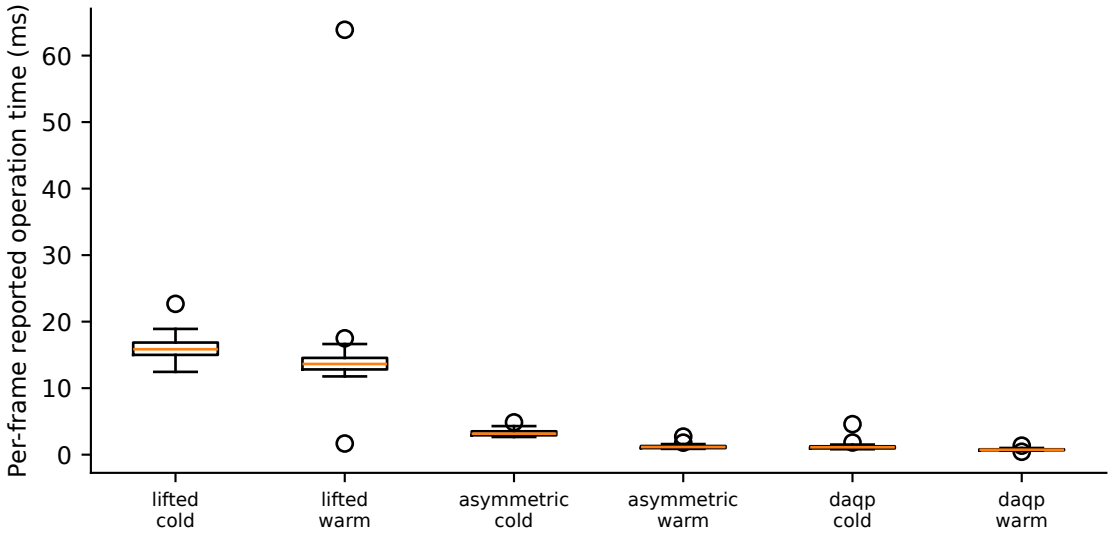


Figure 9: Stored cold and warm C12 operation durations for every accepted frame of each compatible method. The box summaries are descriptive paired-path evidence, not a randomized causal isolation of warm starting from all other implementation choices.

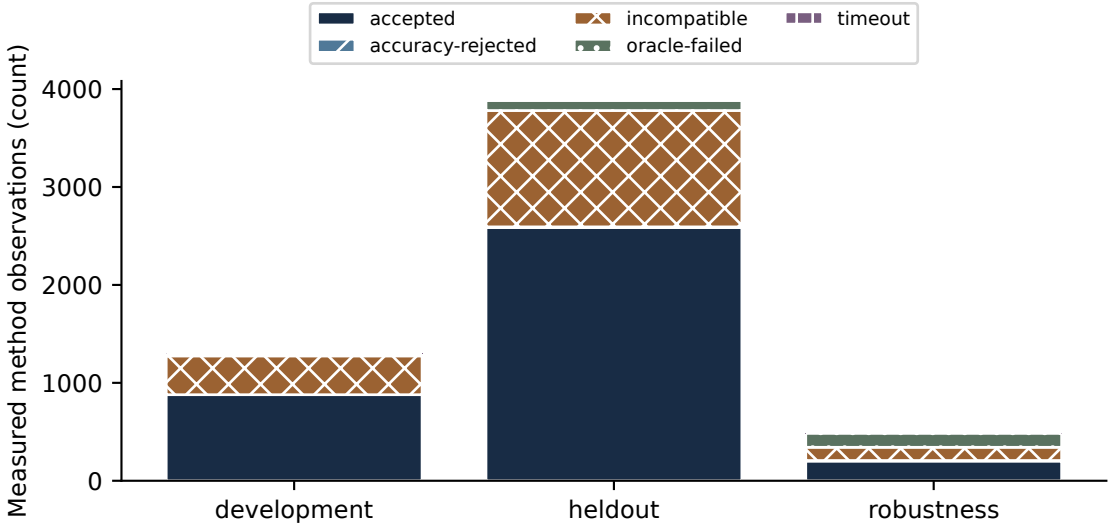


Figure 10: All 5670 measured method observations of the grouped study, separated by their original development, held-out and robustness splits. Acceptance, accuracy rejection, incompatibility, dependent oracle failure and supervised timeout remain different categories.

their fixed score, and six missing native medians remain gaps. No selected baseline is imputed where the registered rule selected none. The primary median reduction is -212.449875%, with 95% limits -225.336618% and -192.873759%. These values do not define a uniform per-instance slowdown multiplier. Figure 12 displays returned method medians with their unequal sample counts. It is a descriptive formulation comparison, not an alternative matched estimator that removes failures or compatibility differences. The original primary prediction is not rescued by those plots.

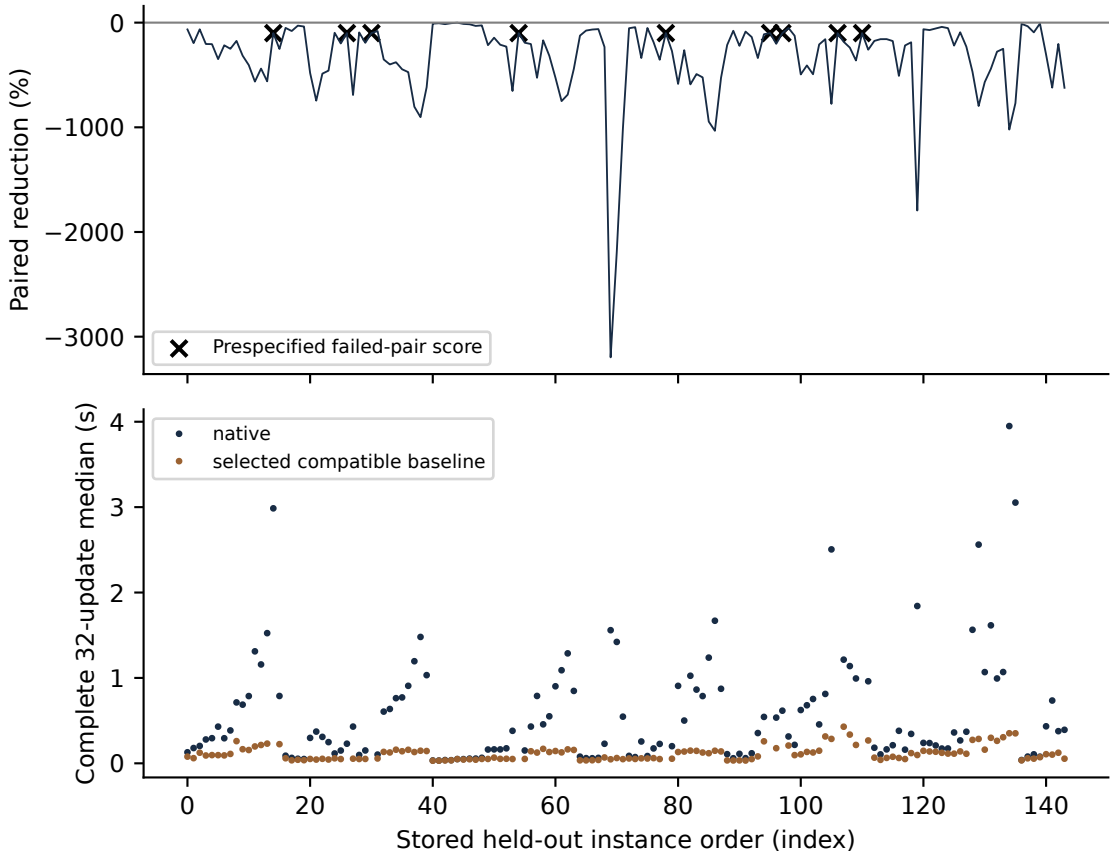


Figure 11: All 144 stored grouped-study primary members in original order, with nine failed scores retained. Complete-operation medians preserve missing values as gaps and do not impute a comparator where the primary rule selected none.

Finally, Figure 13 gives the stored native, epigraph and directional row-split answers for the same two-variable physical teaching input. Their objective and decision agreement illustrates representation equivalence on a transparent example. It contributes no new timing sample to either registered study. The dimensional covariance checks and policy scale changes described in the teaching material distinguish preserving physical meaning from intentionally changing a directional preference. The coexistence of that numerical agreement with the complete negative timing endpoint is the principal practical result: mathematical equivalence and smaller principal dimensions do not guarantee a cheaper complete operation.

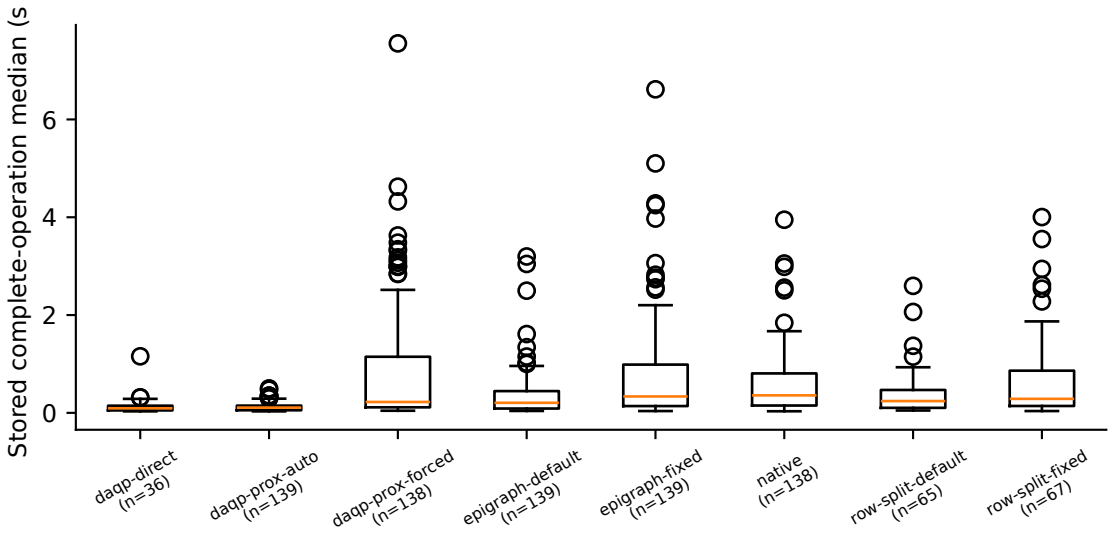


Figure 12: Distributions of stored held-out per-instance medians for each returned compatible formulation. Sample counts differ because compatibility and failures differ; these unmatched descriptive distributions are not the registered paired comparison or a causal ablation.

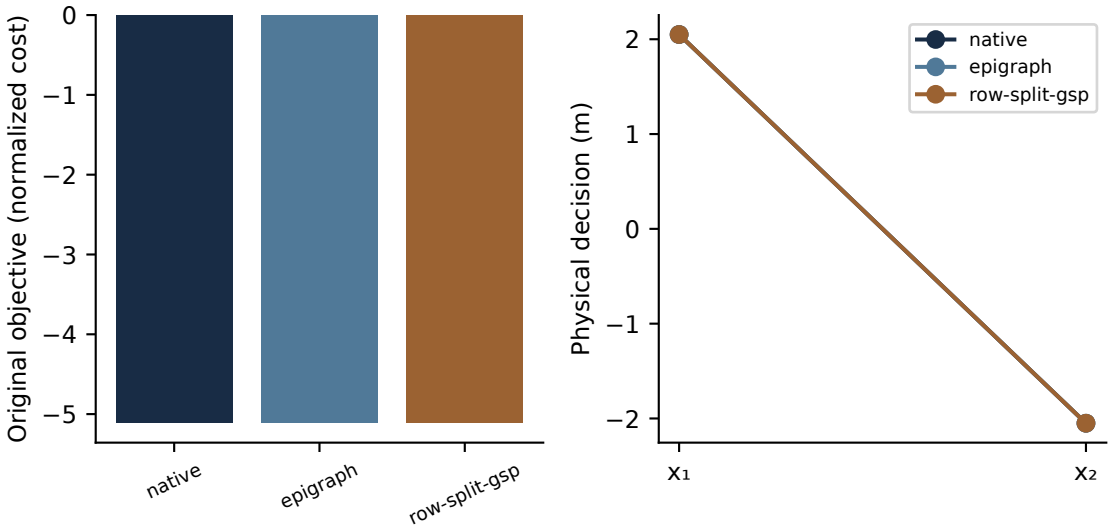


Figure 13: Stored native, epigraph and directional row-split answers for the same two-variable physical teaching input. Objective and decision agreement are a small numerical illustration, not additional timing evidence or an independent experiment.

9 Validation

Validation starts in original coordinates. For a returned decision x, the evaluator computes Ax, both departure arrays, the original base objective and the exact requested penalty. Hard rows are tested without borrowing the backend's internally scaled residual. The original timing criterion uses

$$\delta_F = \frac{|F(x) - F(x_\star)|}{\max\{1, |F(x_\star)|\}}, \qquad h(x) = \max_{i \in \mathcal{H}} \max\{(\ell_i - A_i x)_+, (A_i x - u_i)_+\}. \tag{23}$$

The independent lifted value $x_\star$ uses the matching objective, not a solution of a symmetric or regularized surrogate. Both quantities must satisfy their registered limits. The denominator in δ_F is a numerical normalization, not a claim that objective scales from unrelated physical models are interchangeable. If the hard-row set is empty, its aggregate violation is defined as $h(x) = 0$. This global acceptance convention is distinct from a reader's unit-filtered display, where no hard rows in the selected unit means that field is unavailable.

Primal–dual conditions add information beyond objective agreement. With original multiplier y, stationarity and the soft subgradient condition are

$$Px + q + A^\top y = 0, \qquad y \in \partial g(Ax). \tag{24}$$

For separable costs, a proximal fixed-point check evaluates the second relation without choosing an arbitrary derivative at a kink. The grouped study uses the interval support and joint directional conjugate. For one group define

$$L_g(y) = \sum_{i \in I_g} \left(\frac{(y_i)_+}{c_{U,i}} + \frac{(-y_i)_+}{c_{L,i}} \right). \tag{25}$$

Its residual-penalty conjugate is

$$\Psi_g^*(y) = \begin{cases} (L_g(y) - a_g)_+^2 / (2b_g), & b_g > 0, \\ 0, & b_g = 0,\ L_g(y) \le a_g, \\ +\infty, & b_g = 0,\ L_g(y) > a_g. \end{cases} \tag{26}$$

The complete interval conjugate also contains the box support. Thus the grouped dual condition is a joint price budget, not independent per-coordinate caps. Every soft group's absolute normalized Fenchel gap and every hard row's complementarity are checked separately. A signed aggregate gap is retained diagnostically and cannot permit cross-row cancellation.

Raw and repaired dual quantities are distinguished. A bounded repair can project small numerical defects onto an admissible dual domain, but its size and effect on stationarity remain visible. An arbitrarily large repair would erase the evidence that the returned dual was wrong. Candidate vectors are checked frame by frame; backend-limit observations without vectors remain noncandidates rather than acquiring synthetic coordinates. The 163390 returned candidate frames and 44 noncandidate limit frames consequently have different roles in the grouped validation record.

Physical covariance tests transform rows, bounds, prices, directions and duals consistently. They exercise metre-to-centimetre and second-to-millisecond maps in the predictive-control model and reciprocal directional row scaling in the grouped teaching example. Negative controls omit or mismatch units, change hard policy, reverse side signs and alter imported objectives. These checks establish concrete semantic invariants; they do not prove every floating-point computation

invariant under arbitrary scaling. General floating-point error and equilibration considerations remain relevant background [15, 17].

The data boundaries have separate producer and consumer tests, including fixture substitution, malformed arrays, unknown fields, cross-field mismatch, stale source identity and rehashed tampering. Exact artifact hashes prove identity only relative to a trusted inventory. They do not by themselves prove numerical correctness, so source-bound independent acceptance is retained separately. The same principle applies to a notebook execution or a web screenshot: successful execution/rendering does not substitute for inspecting what the method solved.

Additional solver literature informs the numerical reference choices without becoming unexecuted performance evidence. Interior-point linear programming and conic/QP implementations illustrate alternative handling of structure and regularization [31, 28]; those citations are not extra timed competitors in the registered populations. Reproducible test infrastructure and benchmark definitions are useful for future experiments [14], but the present conclusions remain restricted to the exact synthetic inputs, adapters, tolerances and complete operations actually recorded here.

10 Discussion

The two negative timing outcomes are consistent with the distinction between algebraic size and numerical work. An original-row formulation can save auxiliaries while adding a more expensive proximal operation, a threshold sort, a source conversion or an acceptance calculation. Sparse factorization also depends on fill, ordering, numeric scaling and reuse. Variable counts alone do not measure these effects. The small C01 formulation comparison therefore explains representation but cannot establish why a complete repeated-forecast operation is faster or slower.

The strongest practical comparison is an equivalent objective with a capable reference used according to its documented interface. Directional row splitting is especially important because it reuses a published grouped projection method to express unequal directions. Compact epigraphs offer another exact route. Removing those references would test a weaker question. Likewise, allowing only the candidate to preserve its workspace would build an artificial advantage into repeated execution. Online active-set methods have long exploited parametric structure [12]; a reuse mechanism is not in itself evidence of a new algorithmic contribution.

The grouped threshold calculation has clear mathematical value even when its runtime prediction fails. It exposes the actual directional metric, zero-price behavior and joint dual budget. Those details make a correctness check possible and clarify why a dense metric or overlapping groups require additional analysis. Existing weighted projection algorithms and simplex/ norm-ball work provide relevant algorithmic background [25, 9, 6]. The implemented threshold is not presented as the first use of sorting, orthant behavior or scalar active-prefix equations.

Numerical policy and physical policy must also remain distinct. A stronger temperature price can reduce a comfort departure by changing the optimized tradeoff, not by improving numerical accuracy. An oracle-driven predictive trajectory can isolate solver agreement on common inputs, not demonstrate that separately optimized controllers have equal field behavior. For energy examples, comparing a mixed group/Huber optimum with an authored separable variant describes a policy change. These distinctions limit what a practitioner should infer from cross-case objective tables and interactive cost edits.

The results suggest choosing an admitted exact formulation based on the actual problem regime and measured complete-operation requirements, rather than adopting the smallest principal matrix by default. Where hard policy is immutable, a method that cannot preserve it should be refused. Where uncertainty about physical coefficients dominates numerical error, tighter solver tolerances do not certify a better operational model. A future computational prediction should specify a new mechanism and evaluation prospectively; the existing held-out data cannot be retuned into evidence for a revised threshold, population or favorable subset.

The system's inspectability remains useful despite the failed performance criteria. Original-space quantities, exact compatibility and retained unsuccessful outcomes let a reader distinguish an implementation defect from an objective mismatch or a negative computational finding. They do not establish adoption, realized savings or safety. Those questions require different data, deployment conditions and evidence than the synthetic optimization studies described here.

11 Limitations

The inputs are transparent synthetic optimization models. The source-faithful building example preserves an open model rather than observations from an occupied building. Its battery and thermal parameters are not identified from a particular installation. The grouped thermal study uses a separate hourly RC forecast model. Neither model supports a claim about actual energy savings, occupant comfort, component ageing or operational safety. Those outcomes require measured disturbances, identification uncertainty, actuator behavior and a separately validated control policy.

The timing studies were executed on one nonreserved host with explicit software identities and resource boundaries. Fixed seeds and randomized method order improve reproducibility but do not eliminate operating-system interference, thermal throttling, library maturity or language-level overhead. The Python/SciPy FlexQP reconstruction and native C backends are concrete implementations with different engineering histories. Their runtime difference is not a formulation-independent theorem. Backend setup includes linear-system preparation; isolated numeric factor time was not instrumented and cannot be inferred from a setup field.

Statistical generalization is limited by the prospective populations and few seed clusters. Original repeated observations are dependent within generated problems and seeds. The cluster procedure respects a selected dependence structure, but six clusters do not guarantee accurate coverage under every disturbance or sampling process [4]. The grouped score includes the prescribed failed-pair treatment. Its negative median is not a universal latency multiplier. Unmatched method distributions and selected descriptive slices cannot replace either registered paired endpoint.

The mathematical scope is also explicit. The group proximal identity uses nonempty closed boxes, contraction-monotone convex prices and a fixed positive diagonal metric. The native grouped interface requires disjoint groups and positive finite directional scales. The convergence statement assumes an attained saddle point and the specified relaxation range; it does not establish arbitrary adaptive metrics, overlapping groups, nonconvex curvature or every finite-precision infeasibility certificate. The synthetic robustness grid is finite, so passing it does not certify all nearly singular or physically scaled problems.

Finally, the browser verifies and presents existing results and matching local-native imports. It has no browser-native optimizer or hosted optimization service, and the grouped research exchanges have no installed replay adapter. A published package, a schema example and a

successful render are different kinds of availability evidence. Deep FlexQP's trained policy was unavailable and is not reproduced. The complete negative findings remain scientifically informative within these restrictions, without establishing algorithmic priority, superiority, practical adoption or a qualified production control system.

12 Reproducibility

The public source repository is https://github.com/fsantibanezleal/CAOS_SoftQP; the separate native engine is https://github.com/fsantibanezleal/CAOS_SoftQP_Engine. The original study uses its recorded native source identity, which must not be equated silently with a later public wheel. The grouped study uses the separate pinned SDK 0.2 profile and registered protocol/source map. Replaying archived observations does not require rerunning timing measurements.

The grouped evidence dataset is available at https://doi.org/10.5281/zenodo.23875194. The stored original summaries and common control trajectory are `data/derived/experiments/campaign.json` and `data/derived/experiments/stream.json` in the product source at https://github.com/fsantibanezleal/CAOS_SoftQP/tree/2d3f299d80edad49c2599e61bb6e3c875c119043. The original computation used native engine source `d0c69ed974d577ba02adbd4806332270d9425651`; the grouped registered implementation used source `246bf87e690f4d010494c556e2021d311d587422`. The grouped archive contains the protocol and complete observations; its summary is also `manuscripts/coupled-peak-study/results/summary/summary.json` in that frozen product tree. These identifiers locate the numerical evidence independently of later manuscript presentation changes.

The committed manuscript asset generator reads immutable scientific inputs, checks their Git identities and emits vector PDF figures, literal result tables and source manifests. Running `python manuscripts/shared/generate_assets.py` rebuilds those assets without solving a model or recomputing a confidence interval. Each document's `build.py` verifies numeric sources and compiles native LaTeX with BibTeX, refusing unresolved or clipped output. The manuscript rendering requirements are pinned separately from numerical experiment environments.

The full grouped data include canonical batches, complete JSONL/CSV, protocol snapshots, environment identities, unsuccessful observations and replay validation. Reproduction of a new timing study requires an exact historical source/environment checkout and a new external output directory; such a rerun is not needed to inspect the preserved results. Large study outputs, temporary files and scientific environments use external role roots. No secret token or private operational dataset is required to read the scientific inputs or reconstruct the published figures and tables.

Declaration of generative AI use. Generative AI tools assisted source synthesis, mathematical checking, implementation and drafting. Numerical and editorial claims were checked against independent formulations and preserved computational evidence. Responsibility for the work rests with the named author.

13 Conclusions

An explicit hard/soft policy, original-space objective and versioned exchange boundaries make a convex optimization result inspectable. The implemented system compares seven method

paths across their actual compatibility domains, preserves physical row units and supports native execution, independent evaluation, verified replay and matching local result import. Equivalent lifting and directional reformulations show that representation choices must be assessed on the same objective rather than on similarly named soft constraints.

The complete original 456-instance study does not establish its 10% timing prediction: the stored median reduction is 0.499839%, with a 95% interval from -3.291967% to 3.944835%. The separate grouped study likewise does not establish improvement: its 144-instance primary score is -212.449875%, with limits -225.336618% and -192.873759% and 9 retained failed pairs. These are separate inferential results with different measurement scopes, not interchangeable runtime rankings or a uniform slowdown statement.

Original-row proximal calculations can be mathematically exact and still fail to reduce complete-operation cost. Future work should identify a specific mechanism, preserve the strongest compatible references and evaluate a prospective population. The current synthetic evidence supports reproducible modeling and numerical investigation; it does not establish field benefit, safety or algorithmic priority.

A Complete nested contract fields

The following schema-derived tables expand nested sparse matrices, original row metadata, method outputs, update vectors and numerical diagnostics. Their field paths retain array markers and alternative branches so an object-level summary cannot hide the type or bound of a numerical child. Structural restrictions are copied literally from the identified schema. Paths marked if identify a predicate; then and else specify its respective consequences. A not branch forbids its displayed condition; contains with minContains specifies required matching array members. These branch markers preserve conditional meaning rather than asserting every alternative simultaneously. Physical dimensions are interpreted through the original model and its explicit unit maps; a metadata container has no single physical unit merely because it contains numerical values.

The schemas establish structural admission rather than convexity, physical adequacy or scientific optimality. Cross-field requirements described in the data section still apply: sparse pointer consistency, matching vector lengths, unchanged hard policy, exact request identity, group membership and complete experimental populations cannot be inferred by counting table rows. Likewise, a unit symbol is not a numerical conversion and an absent result is not zero. This appendix complements the numerical derivations and producer/consumer descriptions; it does not replace original-space evaluation. Each table names the exact contract whose complete fields it expands, keeping the parameter meaning inspectable without treating a schema example as available browser data or a proof of successful solver execution.

Nested fields of case-index are enumerated in Table 24.

Table 24: Complete nested fields, value types, literal schema restrictions and physical or structural dimensions for case-index. Required and optional structures retain the original schema and model interpretation.

Field path	Type	Restrictions	Unit / role
schema	string	const=softqp.artifact-index/v1	structural metadata / nested units
engine_version	string	See defined object / value type	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
n_cases	integer	minimum=1	count / integer identifier
default_case	string	pattern=^C[0-9]{2}$	structural metadata / nested units
cases	array	minItems=1	structural metadata / nested units
cases[]	object	additionalProperties=False; required=['case_id', 'title', 'category', 'manifest_path']	structural metadata / nested units
cases[].case_id	string	pattern=^C[0-9]{2}$	structural metadata / nested units
cases[].title	object	additionalProperties=False; required=['en', 'es']	structural metadata / nested units
cases[].title.en	string	minLength=1	structural metadata / nested units
cases[].title.es	string	minLength=1	structural metadata / nested units
cases[].category	object	additionalProperties=False; required=['en', 'es']	structural metadata / nested units
cases[].category.en	string	minLength=1	structural metadata / nested units
cases[].category.es	string	minLength=1	structural metadata / nested units
cases[].manifest_path	string	pattern=^(?!.*\.\.)[A-Za-z0-9_./-]+\.json$	structural metadata / nested units
files	array	uniqueItems=True	structural metadata / nested units
files[]	string	pattern=^(?!.*\.\.)[A-Za-z0-9_./-]+\.(json|csv|jsonl)$	structural metadata / nested units

Nested fields of case-manifest are enumerated in Table 25.

Table 25: Complete nested fields, value types, literal schema restrictions and physical or structural dimensions for case-manifest. Required and optional structures retain the original schema and model interpretation.

Field path	Type	Restrictions	Unit / role
schema	string	const=softqp.manifest/v1	structural metadata / nested units
case_id	string	pattern=^C[0-9]{2}$	structural metadata / nested units
title	object	additionalProperties=False; required=['en', 'es']	structural metadata / nested units
title.en	string	minLength=1	structural metadata / nested units
title.es	string	minLength=1	structural metadata / nested units
category	object	additionalProperties=False; required=['en', 'es']	structural metadata / nested units
category.en	string	minLength=1	structural metadata / nested units
category.es	string	minLength=1	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
engine	object	additionalProperties=False; required=['package', 'version']	structural metadata / nested units
engine.package	string	const=softqp	structural metadata / nested units
engine.version	string	See defined object / value type	structural metadata / nested units
product_version	string	See defined object / value type	structural metadata / nested units
seed	integer	See defined object / value type	count / integer identifier
source	string	See defined object / value type	structural metadata / nested units
artifact	object	additionalProperties=False; required=['path', 'bytes', 'sha256', 'format']	structural metadata / nested units
artifact.path	string	pattern=^(?!.*\.\.)[A-Za-z0-9_./-]+\.json$	structural metadata / nested units
artifact.bytes	integer	minimum=1	bytes
artifact.sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
artifact.format	string	const=json	structural metadata / nested units
lane	string	const=precompute	structural metadata / nested units
gate	object	additionalProperties=False; required=['lane', 'reasons']	structural metadata / nested units
gate.lane	string	const=precompute	structural metadata / nested units
gate.reasons	array	minItems=1	structural metadata / nested units
gate.reasons[]	object	additionalProperties=False; required=['en', 'es']	structural metadata / nested units
gate.reasons[].en	string	minLength=1	structural metadata / nested units
gate.reasons[].es	string	minLength=1	structural metadata / nested units

Nested fields of readiness are enumerated in Table 26.

Table 26: Complete nested fields, value types, literal schema restrictions and physical or structural dimensions for readiness. Required and optional structures retain the original schema and model interpretation.

Field path	Type	Restrictions	Unit / role
schema	string	const=softqp.readiness/v2	structural metadata / nested units
product_version	string	pattern=^\d+\.\d{2}\.\d{3}$	structural metadata / nested units
catalog_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
contracts	array	minItems=1	structural metadata / nested units
contracts[]	object	additionalProperties=False; required=['id', 'version', 'status', 'provenance', 'state', 'reason', 'missing']	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
contracts[].id	string	pattern=^[a-z0-9][a-z0-9-]*$	structural metadata / nested units
contracts[].version	string	pattern=^\d+\.\d+\.\d+$	structural metadata / nested units
contracts[].status	enumerated value	enum=['real', 'active']	structural metadata / nested units
contracts[].provenance	enumerated value	enum=['real', 'stub', 'simulator', 'fixture', 'baseline', None]	structural metadata / nested units
contracts[].state	enumerated value	enum=['available', 'partial', 'stale', 'provider', 'training', 'missing']	structural metadata / nested units
contracts[].reason	union / composition	See defined object / value type	structural metadata / nested units
contracts[].reason (oneOf1)	object	additionalProperties=False; required=['en', 'es']	structural metadata / nested units
contracts[].reason (oneOf1).en	string	minLength=1	structural metadata / nested units
contracts[].reason (oneOf1).es	string	minLength=1	structural metadata / nested units
contracts[].reason (oneOf2)	null	See defined object / value type	structural metadata / nested units
contracts[].missing	array	See defined object / value type	structural metadata / nested units
contracts[].missing[]	string	See defined object / value type	structural metadata / nested units
contracts[] (allOf1)	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf1) (if)	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf1) (if).state	string	const=available	structural metadata / nested units
contracts[] (allOf1) (then)	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf1) (then).reason	null	See defined object / value type	structural metadata / nested units
contracts[] (allOf1) (then).provenance	string	const=real	structural metadata / nested units
contracts[] (allOf1) (else)	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf1) (else).reason	object	additionalProperties=False; required=['en', 'es']	structural metadata / nested units
contracts[] (allOf1) (else).reason.en	string	minLength=1	structural metadata / nested units
contracts[] (allOf1) (else).reason.es	string	minLength=1	structural metadata / nested units
contracts[] (allOf2)	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf2) (if)	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf2) (if).state	string	const=partial	structural metadata / nested units
contracts[] (allOf2) (then)	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf2) (then).missing	array	minItems=1	structural metadata / nested units
contracts[] (allOf2) (else)	object	See defined object / value type	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
contracts[] (allOf2) (else).missing	array	maxItems=0	structural metadata / nested units
contracts[] (allOf3)	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf3) (if)	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf3) (if).state	string	const=missing	structural metadata / nested units
contracts[] (allOf3) (then)	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf3) (then).provenance	null	See defined object / value type	structural metadata / nested units
contracts[] (allOf3) (else)	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf3) (else).provenance	string	See defined object / value type	structural metadata / nested units
contracts[] (allOf4)	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf4) (if)	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf4) (if).state	string	const=provider	structural metadata / nested units
contracts[] (allOf4) (then)	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf4) (then).provenance	enumerated value	enum=['stub', 'simulator', 'fixture', 'baseline']	structural metadata / nested units
contracts[] (allOf5)	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf5) (if)	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf5) (if).state	string	const=training	structural metadata / nested units
contracts[] (allOf5) (then)	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf5) (then).provenance	enumerated value	enum=['real', 'baseline']	structural metadata / nested units
contracts[] (allOf6)	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf6) (if)	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf6) (if).status	string	const=active	structural metadata / nested units
contracts[] (allOf6) (then)	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf6) (then).provenance	object	See defined object / value type	structural metadata / nested units
contracts[] (allOf6) (then).provenance (not)	string	const=real	structural metadata / nested units

Nested fields of api are enumerated in Table 27.

Table 27: Complete nested fields, value types, literal schema restrictions and physical or structural dimensions for api. Required and optional structures retain the original schema and model interpretation.

Field path	Type	Restrictions	Unit / role
detail	object	additionalProperties=False; required=['state', 'reason']	structural metadata / nested units
detail.state	string	const=missing	structural metadata / nested units
detail.reason	object	additionalProperties=False; required=['en', 'es']	structural metadata / nested units
detail.reason.en	string	minLength=1	structural metadata / nested units
detail.reason.es	string	minLength=1	structural metadata / nested units

Nested fields of softqp-index are enumerated in Table 28.

Table 28: Complete nested fields, value types, literal schema restrictions and physical or structural dimensions for softqp-index. Required and optional structures retain the original schema and model interpretation.

Field path	Type	Restrictions	Unit / role
schema_version	string	const=softqp.index/v1	structural metadata / nested units
state	string	const=development	structural metadata / nested units
default_case	union / composition	See defined object / value type	structural metadata / nested units
default_case (anyOf1)	string	pattern=^C[0-9]{2}$	structural metadata / nested units
default_case (anyOf2)	null	See defined object / value type	structural metadata / nested units
cases	array	See defined object / value type	structural metadata / nested units
cases[]	object	additionalProperties=False; required=['case_id', 'title', 'category', 'path', 'sha256', 'failed']	structural metadata / nested units
cases[].case_id	string	pattern=^C[0-9]{2}$	structural metadata / nested units
cases[].title	object	additionalProperties=False; required=['en', 'es']	structural metadata / nested units
cases[].title.en	string	minLength=1	structural metadata / nested units
cases[].title.es	string	minLength=1	structural metadata / nested units
cases[].category	object	additionalProperties=False; required=['en', 'es']	structural metadata / nested units
cases[].category.en	string	minLength=1	structural metadata / nested units
cases[].category.es	string	minLength=1	structural metadata / nested units
cases[].path	string	pattern=^C[0-9]{2}\.json$	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
cases[].sha256	string	pattern=^[a-f0-9]{64}$	structural metadata / nested units
cases[].failed	boolean	See defined object / value type	structural metadata / nested units
environment	object	additionalProperties=False; required=['python', 'platform', 'packages']	structural metadata / nested units
environment.python	string	See defined object / value type	structural metadata / nested units
environment.platform	string	See defined object / value type	structural metadata / nested units
environment.packages	object	additionalProperties={'type': 'string'}	structural metadata / nested units
environment.engine_revision	string / null	pattern=^[0-9a-f]{40}$	structural metadata / nested units
environment.engine_source_dirty	boolean / null	See defined object / value type	structural metadata / nested units
environment.native_binary_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
limitations	array	See defined object / value type	structural metadata / nested units
limitations[]	string	See defined object / value type	structural metadata / nested units

Nested fields of softqp-case-result are enumerated in Table 29.

Table 29: Complete nested fields, value types, literal schema restrictions and physical or structural dimensions for softqp-case-result. Required and optional structures retain the original schema and model interpretation.

Field path	Type	Restrictions	Unit / role
schema_version	string	const=softqp.case-result/v2	structural metadata / nested units
case_id	string	pattern=^C[0-9]{2}$	structural metadata / nested units
title	object	additionalProperties=False; required=['en', 'es']	structural metadata / nested units
title.en	string	minLength=1	structural metadata / nested units
title.es	string	minLength=1	structural metadata / nested units
category	object	additionalProperties=False; required=['en', 'es']	structural metadata / nested units
category.en	string	minLength=1	structural metadata / nested units
category.es	string	minLength=1	structural metadata / nested units
seed	integer	See defined object / value type	count / integer identifier
source	string	minLength=1	structural metadata / nested units
settings	object	additionalProperties=False; required=['eps_abs', 'eps_rel', 'max_iter', 'adaptive_rho_interval']	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
settings.eps_abs	number	exclusiveMinimum=0	solver coordinate tolerance; model scaling applies
settings.eps_rel	number	exclusiveMinimum=0	dimensionless normalized diagnostic / factor
settings.max_iter	integer	minimum=1	count / integer identifier
settings.adaptive_rho_interval	integer	minimum=1	count / integer identifier
runs	array	minItems=1	structural metadata / nested units
runs[]	object	additionalProperties=False; required=['method', 'state', 'reason', 'result']	structural metadata / nested units
runs[].method	enumerated value	enum=['lifted', 'generalized', 'asymmetric', 'daqp', 'flex', 'elastiqp']	structural metadata / nested units
runs[].state	enumerated value	enum=['available', 'failed', 'incompatible']	structural metadata / nested units
runs[].reason	union / composition	See defined object / value type	structural metadata / nested units
runs[].reason (anyOf1)	string	minLength=1	structural metadata / nested units
runs[].reason (anyOf2)	null	See defined object / value type	structural metadata / nested units
runs[].result	union / composition	See defined object / value type	structural metadata / nested units
runs[].result (anyOf1)	object	additionalProperties=False; required=['method', 'status', 'status_code', 'x', 'metrics', 'dimensions', 'iterations', 'setup_seconds', 'solve_seconds', 'elapsed_seconds']	structural metadata / nested units
runs[].result (anyOf1).method	string	See defined object / value type	structural metadata / nested units
runs[].result (anyOf1).status	string	See defined object / value type	structural metadata / nested units
runs[].result (anyOf1).status_code	integer	See defined object / value type	count / integer identifier
runs[].result (anyOf1).x	union / composition	See defined object / value type	decision unit
runs[].result (anyOf1).x (anyOf1)	array	See defined object / value type	decision unit
runs[].result (anyOf1).x (anyOf1)[]	number	See defined object / value type	decision unit
runs[].result (anyOf1).x (anyOf2)	null	See defined object / value type	decision unit
runs[].result (anyOf1).y	union / composition	See defined object / value type	cost / original row unit
runs[].result (anyOf1).y (anyOf1)	array	See defined object / value type	cost / original row unit
runs[].result (anyOf1).y (anyOf1)[]	number	See defined object / value type	cost / original row unit
runs[].result (anyOf1).y (anyOf2)	null	See defined object / value type	cost / original row unit
runs[].result (anyOf1).metrics	union / composition	See defined object / value type	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
runs[].result (anyOf1).metrics (anyOf1)	object	additionalProperties=False; required=['base_objective', 'penalty_objective', 'objective', 'hard_violation', 'lower_violation', 'upper_violation', 'slack', 'objective_gap_against_lifted']	structural metadata / nested units
runs[].result (anyOf1).metrics (anyOf1).base_objective	number	See defined object / value type	model cost unit
runs[].result (anyOf1).metrics (anyOf1).penalty_objective	number	See defined object / value type	model cost unit
runs[].result (anyOf1).metrics (anyOf1).objective	number	See defined object / value type	model cost unit
runs[].result (anyOf1).metrics (anyOf1).hard_violation	number	See defined object / value type	original row units; see unit map
runs[].result (anyOf1).metrics (anyOf1).lower_violation	array	See defined object / value type	original row units; see unit map
runs[].result (anyOf1).metrics (anyOf1).lower_violation[]	number	See defined object / value type	original row units; see unit map
runs[].result (anyOf1).metrics (anyOf1).upper_violation	array	See defined object / value type	original row units; see unit map
runs[].result (anyOf1).metrics (anyOf1).upper_violation[]	number	See defined object / value type	original row units; see unit map
runs[].result (anyOf1).metrics (anyOf1).slack	array	See defined object / value type	original row units; see unit map
runs[].result (anyOf1).metrics (anyOf1).slack[]	number	See defined object / value type	original row units; see unit map
runs[].result (anyOf1).metrics (anyOf1).stationarity_residual	number	See defined object / value type	cost / decision unit; raw gradient norm
runs[].result (anyOf1).metrics (anyOf1).objective_gap_against_lifted	union / composition	See defined object / value type	model cost unit
runs[].result (anyOf1).metrics (anyOf1).objective_gap_against_lifted (anyOf1)	number	See defined object / value type	model cost unit
runs[].result (anyOf1).metrics (anyOf1).objective_gap_against_lifted (anyOf2)	null	See defined object / value type	model cost unit
runs[].result (anyOf1).metrics (anyOf2)	null	See defined object / value type	structural metadata / nested units
runs[].result (anyOf1).dimensions	object	additionalProperties=False; required=['variables', 'rows', 'nnz']	structural metadata / nested units
runs[].result (anyOf1).dimensions.variables	integer	minimum=0	count / integer identifier
runs[].result (anyOf1).dimensions.rows	integer	minimum=0	count / integer identifier
runs[].result (anyOf1).dimensions.nnz	integer	minimum=0	count / integer identifier

Field path	Type	Restrictions	Unit / role
runs[].result (anyOf1). iterations	integer	minimum=0	count / integer identifier
runs[].result (anyOf1). setup_seconds	number	minimum=0	s
runs[].result (anyOf1).solve_ seconds	number	minimum=0	s
runs[].result (anyOf1). elapsed_seconds	number	minimum=0	s
runs[].result (anyOf1). backend_settings	object	additionalProperties={'type': 'number'}	structural metadata / nested units
runs[].result (anyOf2)	null	See defined object / value type	structural metadata / nested units
runs[] (allOf1)	object	See defined object / value type	structural metadata / nested units
runs[] (allOf1) (if)	object	See defined object / value type	structural metadata / nested units
runs[] (allOf1) (if).state	string	const=available	structural metadata / nested units
runs[] (allOf1) (then)	object	See defined object / value type	structural metadata / nested units
runs[] (allOf1) (then).reason	null	See defined object / value type	structural metadata / nested units
runs[] (allOf1) (then).result	object	additionalProperties=False; required= ['method', 'status', 'status_code', 'x', 'metrics', 'dimensions', 'iterations', 'setup_seconds', 'solve_seconds', 'elapsed_seconds']	structural metadata / nested units
runs[] (allOf1) (then).result. method	string	See defined object / value type	structural metadata / nested units
runs[] (allOf1) (then).result. status	string	See defined object / value type	structural metadata / nested units
runs[] (allOf1) (then).result. status_code	integer	See defined object / value type	count / integer identifier
runs[] (allOf1) (then).result. x	union / composition	See defined object / value type	decision unit
runs[] (allOf1) (then).result. x (anyOf1)	array	See defined object / value type	decision unit
runs[] (allOf1) (then).result. x (anyOf1)[]	number	See defined object / value type	decision unit
runs[] (allOf1) (then).result. x (anyOf2)	null	See defined object / value type	decision unit
runs[] (allOf1) (then).result. y	union / composition	See defined object / value type	cost / original row unit
runs[] (allOf1) (then).result. y (anyOf1)	array	See defined object / value type	cost / original row unit
runs[] (allOf1) (then).result. y (anyOf1)[]	number	See defined object / value type	cost / original row unit
runs[] (allOf1) (then).result. y (anyOf2)	null	See defined object / value type	cost / original row unit
runs[] (allOf1) (then).result. metrics	union / composition	See defined object / value type	structural metadata / nested units
runs[] (allOf1) (then).result. metrics (anyOf1)	object	additionalProperties=False; required= ['base_objective', 'penalty_objective', 'objective', 'hard_violation', 'lower_ violation', 'upper_violation', 'slack', 'objective_gap_against_lifted']	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
runs[] (allOf1) (then).result.metrics (anyOf1).base_objective	number	See defined object / value type	model cost unit
runs[] (allOf1) (then).result.metrics (anyOf1).penalty_objective	number	See defined object / value type	model cost unit
runs[] (allOf1) (then).result.metrics (anyOf1).objective	number	See defined object / value type	model cost unit
runs[] (allOf1) (then).result.metrics (anyOf1).hard_violation	number	See defined object / value type	original row units; see unit map
runs[] (allOf1) (then).result.metrics (anyOf1).lower_violation	array	See defined object / value type	original row units; see unit map
runs[] (allOf1) (then).result.metrics (anyOf1).lower_violation[]	number	See defined object / value type	original row units; see unit map
runs[] (allOf1) (then).result.metrics (anyOf1).upper_violation	array	See defined object / value type	original row units; see unit map
runs[] (allOf1) (then).result.metrics (anyOf1).upper_violation[]	number	See defined object / value type	original row units; see unit map
runs[] (allOf1) (then).result.metrics (anyOf1).slack	array	See defined object / value type	original row units; see unit map
runs[] (allOf1) (then).result.metrics (anyOf1).slack[]	number	See defined object / value type	original row units; see unit map
runs[] (allOf1) (then).result.metrics (anyOf1).stationarity_residual	number	See defined object / value type	cost / decision unit; raw gradient norm
runs[] (allOf1) (then).result.metrics (anyOf1).objective_gap_against_lifted	union / composition	See defined object / value type	model cost unit
runs[] (allOf1) (then).result.metrics (anyOf1).objective_gap_against_lifted (anyOf1)	number	See defined object / value type	model cost unit
runs[] (allOf1) (then).result.metrics (anyOf1).objective_gap_against_lifted (anyOf2)	null	See defined object / value type	model cost unit
runs[] (allOf1) (then).result.metrics (anyOf2)	null	See defined object / value type	structural metadata / nested units
runs[] (allOf1) (then).result.dimensions	object	additionalProperties=False; required=['variables', 'rows', 'nnz']	structural metadata / nested units
runs[] (allOf1) (then).result.dimensions.variables	integer	minimum=0	count / integer identifier
runs[] (allOf1) (then).result.dimensions.rows	integer	minimum=0	count / integer identifier
runs[] (allOf1) (then).result.dimensions.nnz	integer	minimum=0	count / integer identifier
runs[] (allOf1) (then).result.iterations	integer	minimum=0	count / integer identifier
runs[] (allOf1) (then).result.setup_seconds	number	minimum=0	s

Field path	Type	Restrictions	Unit / role
runs[] (allOf1) (then).result.solve_seconds	number	minimum=0	s
runs[] (allOf1) (then).result.elapsed_seconds	number	minimum=0	s
runs[] (allOf1) (then).result.backend_settings	object	additionalProperties={'type': 'number'}	structural metadata / nested units
runs[] (allOf2)	object	See defined object / value type	structural metadata / nested units
runs[] (allOf2) (if)	object	See defined object / value type	structural metadata / nested units
runs[] (allOf2) (if).state	string	const=incompatible	structural metadata / nested units
runs[] (allOf2) (then)	object	See defined object / value type	structural metadata / nested units
runs[] (allOf2) (then).reason	string	minLength=1	structural metadata / nested units
runs[] (allOf2) (then).result	null	See defined object / value type	structural metadata / nested units
rows	array	See defined object / value type	count / integer identifier
rows[]	object	additionalProperties=False; required=['id', 'label', 'unit', 'soft']	count / integer identifier
rows[].id	string	pattern=^r[0-9]{4}$	structural metadata / nested units
rows[].label	object	additionalProperties=False; required=['en', 'es']	structural metadata / nested units
rows[].label.en	string	minLength=1	structural metadata / nested units
rows[].label.es	string	minLength=1	structural metadata / nested units
rows[].unit	string	minLength=1	structural metadata / nested units
rows[].soft	boolean	See defined object / value type	structural metadata / nested units
problem	object	additionalProperties=False; required=['P', 'q', 'A', 'lower', 'upper', 'penalty']	structural metadata / nested units
problem.P	object	additionalProperties=False; required=['shape', 'data', 'indices', 'indptr']	structural metadata / nested units
problem.P.shape	array	minItems=2; maxItems=2	count / integer identifier
problem.P.shape[0]	integer	minimum=0; maximum=20000	count / integer identifier
problem.P.shape[1]	integer	minimum=0; maximum=20000	count / integer identifier
problem.P.data	array	maxItems=2000000	cost / decision units squared
problem.P.data[]	number	See defined object / value type	cost / decision units squared
problem.P.indices	array	maxItems=2000000	count / integer identifier
problem.P.indices[]	integer	minimum=0	count / integer identifier
problem.P.indptr	array	maxItems=20001	count / integer identifier
problem.P.indptr[]	integer	minimum=0	count / integer identifier

Field path	Type	Restrictions	Unit / role
problem.q	array	maxItems=20000	cost / decision unit
problem.q[]	number	See defined object / value type	cost / decision unit
problem.A	object	additionalProperties=False; required=['shape', 'data', 'indices', 'indptr']	structural metadata / nested units
problem.A.shape	array	minItems=2; maxItems=2	count / integer identifier
problem.A.shape[0]	integer	minimum=0; maximum=20000	count / integer identifier
problem.A.shape[1]	integer	minimum=0; maximum=20000	count / integer identifier
problem.A.data	array	maxItems=2000000	row / decision unit
problem.A.data[]	number	See defined object / value type	row / decision unit
problem.A.indices	array	maxItems=2000000	count / integer identifier
problem.A.indices[]	integer	minimum=0	count / integer identifier
problem.A.indptr	array	maxItems=20001	count / integer identifier
problem.A.indptr[]	integer	minimum=0	count / integer identifier
problem.lower	array	maxItems=20000	original row unit
problem.lower[]	number / null	See defined object / value type	original row unit
problem.upper	array	maxItems=20000	original row unit
problem.upper[]	number / null	See defined object / value type	original row unit
problem.penalty	object	additionalProperties=False; required=['lower_linear', 'upper_linear', 'lower_quadratic', 'upper_quadratic', 'soft']	structural metadata / nested units
problem.penalty.lower_linear	array	maxItems=20000	cost / row unit
problem.penalty.lower_linear[]	number	minimum=0	cost / row unit
problem.penalty.upper_linear	array	maxItems=20000	cost / row unit
problem.penalty.upper_linear[]	number	minimum=0	cost / row unit
problem.penalty.lower_quadratic	array	maxItems=20000	cost / row unit squared
problem.penalty.lower_quadratic[]	number	minimum=0	cost / row unit squared
problem.penalty.upper_quadratic	array	maxItems=20000	cost / row unit squared
problem.penalty.upper_quadratic[]	number	minimum=0	cost / row unit squared
problem.penalty.soft	array	maxItems=20000	structural metadata / nested units
problem.penalty.soft[]	boolean	See defined object / value type	structural metadata / nested units

Nested fields of softqp-case-spec are enumerated in Table 30.

Table 30: Complete nested fields, value types, literal schema restrictions and physical or structural dimensions for softqp-case-spec. Required and optional structures retain the original schema and model interpretation.

Field path	Type	Restrictions	Unit / role
P	object	additionalProperties=False; required=['shape', 'data', 'indices', 'indptr']	structural metadata / nested units
P.shape	array	minItems=2; maxItems=2	count / integer identifier
P.shape[0]	integer	minimum=0; maximum=20000	count / integer identifier
P.shape[1]	integer	minimum=0; maximum=20000	count / integer identifier
P.data	array	maxItems=2000000	cost / decision units squared
P.data[]	number	See defined object / value type	cost / decision units squared
P.indices	array	maxItems=2000000	count / integer identifier
P.indices[]	integer	minimum=0	count / integer identifier
P.indptr	array	maxItems=20001	count / integer identifier
P.indptr[]	integer	minimum=0	count / integer identifier
q	array	maxItems=20000	cost / decision unit
q[]	number	See defined object / value type	cost / decision unit
A	object	additionalProperties=False; required=['shape', 'data', 'indices', 'indptr']	structural metadata / nested units
A.shape	array	minItems=2; maxItems=2	count / integer identifier
A.shape[0]	integer	minimum=0; maximum=20000	count / integer identifier
A.shape[1]	integer	minimum=0; maximum=20000	count / integer identifier
A.data	array	maxItems=2000000	row / decision unit
A.data[]	number	See defined object / value type	row / decision unit
A.indices	array	maxItems=2000000	count / integer identifier
A.indices[]	integer	minimum=0	count / integer identifier
A.indptr	array	maxItems=20001	count / integer identifier
A.indptr[]	integer	minimum=0	count / integer identifier
lower	array	maxItems=20000	original row unit
lower[]	number / null	See defined object / value type	original row unit
upper	array	maxItems=20000	original row unit
upper[]	number / null	See defined object / value type	original row unit
penalty	object	additionalProperties=False; required=['lower_linear', 'upper_linear', 'lower_quadratic', 'upper_quadratic', 'soft']	structural metadata / nested units
penalty.lower_linear	array	maxItems=20000	cost / row unit
penalty.lower_linear[]	number	minimum=0	cost / row unit

Field path	Type	Restrictions	Unit / role
penalty.upper_linear	array	maxItems=20000	cost / row unit
penalty.upper_linear[]	number	minimum=0	cost / row unit
penalty.lower_quadratic	array	maxItems=20000	cost / row unit squared
penalty.lower_quadratic[]	number	minimum=0	cost / row unit squared
penalty.upper_quadratic	array	maxItems=20000	cost / row unit squared
penalty.upper_quadratic[]	number	minimum=0	cost / row unit squared
penalty.soft	array	maxItems=20000	structural metadata / nested units
penalty.soft[]	boolean	See defined object / value type	structural metadata / nested units

Nested fields of softqp-run-request are enumerated in Table 31.

Table 31: Complete nested fields, value types, literal schema restrictions and physical or structural dimensions for softqp-run-request. Required and optional structures retain the original schema and model interpretation.

Field path	Type	Restrictions	Unit / role
schema_version	string	const=softqp.run-request/v1	structural metadata / nested units
case_id	string	pattern=^C[0-9]{2}$	structural metadata / nested units
base_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
lower_multiplier	number	minimum=0.01; maximum=100	dimensionless factor
upper_multiplier	number	minimum=0.01; maximum=100	dimensionless factor
quadratic_multiplier	number	minimum=0; maximum=100	dimensionless factor

Nested fields of softqp-local-export are enumerated in Table 32.

Table 32: Complete nested fields, value types, literal schema restrictions and physical or structural dimensions for softqp-local-export. Required and optional structures retain the original schema and model interpretation.

Field path	Type	Restrictions	Unit / role
schema_version	string	const=softqp.local-export/v1	structural metadata / nested units
request	object	additionalProperties=False; required=['schema_version', 'case_id', 'base_sha256', 'lower_multiplier', 'upper_multiplier', 'quadratic_multiplier']	structural metadata / nested units
request.schema_version	string	const=softqp.run-request/v1	structural metadata / nested units
request.case_id	string	pattern=^C[0-9]{2}$	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
request.base_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
request.lower_multiplier	number	minimum=0.01; maximum=100	dimensionless factor
request.upper_multiplier	number	minimum=0.01; maximum=100	dimensionless factor
request.quadratic_multiplier	number	minimum=0; maximum=100	dimensionless factor
result	object	additionalProperties=False; required=['schema_version', 'case_id', 'title', 'category', 'seed', 'source', 'settings', 'runs', 'rows']	structural metadata / nested units
result.schema_version	string	const=softqp.case-result/v2	structural metadata / nested units
result.case_id	string	pattern=^C[0-9]{2}$	structural metadata / nested units
result.title	object	additionalProperties=False; required=['en', 'es']	structural metadata / nested units
result.title.en	string	minLength=1	structural metadata / nested units
result.title.es	string	minLength=1	structural metadata / nested units
result.category	object	additionalProperties=False; required=['en', 'es']	structural metadata / nested units
result.category.en	string	minLength=1	structural metadata / nested units
result.category.es	string	minLength=1	structural metadata / nested units
result.seed	integer	See defined object / value type	count / integer identifier
result.source	string	minLength=1	structural metadata / nested units
result.settings	object	additionalProperties=False; required=['eps_abs', 'eps_rel', 'max_iter', 'adaptive_rho_interval']	structural metadata / nested units
result.settings.eps_abs	number	exclusiveMinimum=0	solver coordinate tolerance; model scaling applies
result.settings.eps_rel	number	exclusiveMinimum=0	dimensionless normalized diagnostic / factor
result.settings.max_iter	integer	minimum=1	count / integer identifier
result.settings.adaptive_rho_interval	integer	minimum=1	count / integer identifier
result.runs	array	minItems=1	structural metadata / nested units
result.runs[]	object	additionalProperties=False; required=['method', 'state', 'reason', 'result']	structural metadata / nested units
result.runs[].method	enumerated value	enum=['lifted', 'generalized', 'asymmetric', 'daqp', 'flex', 'elastiqp']	structural metadata / nested units
result.runs[].state	enumerated value	enum=['available', 'failed', 'incompatible']	structural metadata / nested units
result.runs[].reason	union / composition	See defined object / value type	structural metadata / nested units
result.runs[].reason (anyOf1)	string	minLength=1	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
result.runs[].reason (anyOf2)	null	See defined object / value type	structural metadata / nested units
result.runs[].result	union / composition	See defined object / value type	structural metadata / nested units
result.runs[].result (anyOf1)	object	additionalProperties=False; required=['method', 'status', 'status_code', 'x', 'metrics', 'dimensions', 'iterations', 'setup_seconds', 'solve_seconds', 'elapsed_seconds']	structural metadata / nested units
result.runs[].result (anyOf1).method	string	See defined object / value type	structural metadata / nested units
result.runs[].result (anyOf1).status	string	See defined object / value type	structural metadata / nested units
result.runs[].result (anyOf1).status_code	integer	See defined object / value type	count / integer identifier
result.runs[].result (anyOf1).x	union / composition	See defined object / value type	decision unit
result.runs[].result (anyOf1).x (anyOf1)	array	See defined object / value type	decision unit
result.runs[].result (anyOf1).x (anyOf1)[]	number	See defined object / value type	decision unit
result.runs[].result (anyOf1).x (anyOf2)	null	See defined object / value type	decision unit
result.runs[].result (anyOf1).y	union / composition	See defined object / value type	cost / original row unit
result.runs[].result (anyOf1).y (anyOf1)	array	See defined object / value type	cost / original row unit
result.runs[].result (anyOf1).y (anyOf1)[]	number	See defined object / value type	cost / original row unit
result.runs[].result (anyOf1).y (anyOf2)	null	See defined object / value type	cost / original row unit
result.runs[].result (anyOf1).metrics	union / composition	See defined object / value type	structural metadata / nested units
result.runs[].result (anyOf1).metrics (anyOf1)	object	additionalProperties=False; required=['base_objective', 'penalty_objective', 'objective', 'hard_violation', 'lower_violation', 'upper_violation', 'slack', 'objective_gap_against_lifted']	structural metadata / nested units
result.runs[].result (anyOf1).metrics (anyOf1).base_objective	number	See defined object / value type	model cost unit
result.runs[].result (anyOf1).metrics (anyOf1).penalty_objective	number	See defined object / value type	model cost unit
result.runs[].result (anyOf1).metrics (anyOf1).objective	number	See defined object / value type	model cost unit
result.runs[].result (anyOf1).metrics (anyOf1).hard_violation	number	See defined object / value type	original row units; see unit map
result.runs[].result (anyOf1).metrics (anyOf1).lower_violation	array	See defined object / value type	original row units; see unit map
result.runs[].result (anyOf1).metrics (anyOf1).lower_violation[]	number	See defined object / value type	original row units; see unit map

Field path	Type	Restrictions	Unit / role
result.runs[].result (anyOf1). metrics (anyOf1).upper_ violation	array	See defined object / value type	original row units; see unit map
result.runs[].result (anyOf1). metrics (anyOf1).upper_ violation[]	number	See defined object / value type	original row units; see unit map
result.runs[].result (anyOf1). metrics (anyOf1).slack	array	See defined object / value type	original row units; see unit map
result.runs[].result (anyOf1). metrics (anyOf1).slack[]	number	See defined object / value type	original row units; see unit map
result.runs[].result (anyOf1). metrics (anyOf1). stationarity_residual	number	See defined object / value type	cost / decision unit; raw gradient norm
result.runs[].result (anyOf1). metrics (anyOf1).objective_ gap_against_lifted	union / composition	See defined object / value type	model cost unit
result.runs[].result (anyOf1). metrics (anyOf1).objective_ gap_against_lifted (anyOf1)	number	See defined object / value type	model cost unit
result.runs[].result (anyOf1). metrics (anyOf1).objective_ gap_against_lifted (anyOf2)	null	See defined object / value type	model cost unit
result.runs[].result (anyOf1). metrics (anyOf2)	null	See defined object / value type	structural metadata / nested units
result.runs[].result (anyOf1). dimensions	object	additionalProperties=False; required= ['variables', 'rows', 'nnz']	structural metadata / nested units
result.runs[].result (anyOf1). dimensions.variables	integer	minimum=0	count / integer identifier
result.runs[].result (anyOf1). dimensions.rows	integer	minimum=0	count / integer identifier
result.runs[].result (anyOf1). dimensions.nnz	integer	minimum=0	count / integer identifier
result.runs[].result (anyOf1). iterations	integer	minimum=0	count / integer identifier
result.runs[].result (anyOf1). setup_seconds	number	minimum=0	s
result.runs[].result (anyOf1). solve_seconds	number	minimum=0	s
result.runs[].result (anyOf1). elapsed_seconds	number	minimum=0	s
result.runs[].result (anyOf1). backend_settings	object	additionalProperties={'type': 'number'}	structural metadata / nested units
result.runs[].result (anyOf2)	null	See defined object / value type	structural metadata / nested units
result.runs[] (allOf1)	object	See defined object / value type	structural metadata / nested units
result.runs[] (allOf1) (if)	object	See defined object / value type	structural metadata / nested units
result.runs[] (allOf1) (if). state	string	const=available	structural metadata / nested units
result.runs[] (allOf1) (then)	object	See defined object / value type	structural metadata / nested units
result.runs[] (allOf1) (then). reason	null	See defined object / value type	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
result.runs[] (allOf1) (then). result	object	additionalProperties=False; required= ['method', 'status', 'status_code', 'x', 'metrics', 'dimensions', 'iterations', 'setup_seconds', 'solve_seconds', 'elapsed_seconds']	structural metadata / nested units
result.runs[] (allOf1) (then). result.method	string	See defined object / value type	structural metadata / nested units
result.runs[] (allOf1) (then). result.status	string	See defined object / value type	structural metadata / nested units
result.runs[] (allOf1) (then). result.status_code	integer	See defined object / value type	count / integer identifier
result.runs[] (allOf1) (then). result.x	union / composition	See defined object / value type	decision unit
result.runs[] (allOf1) (then). result.x (anyOf1)	array	See defined object / value type	decision unit
result.runs[] (allOf1) (then). result.x (anyOf1)[]	number	See defined object / value type	decision unit
result.runs[] (allOf1) (then). result.x (anyOf2)	null	See defined object / value type	decision unit
result.runs[] (allOf1) (then). result.y	union / composition	See defined object / value type	cost / original row unit
result.runs[] (allOf1) (then). result.y (anyOf1)	array	See defined object / value type	cost / original row unit
result.runs[] (allOf1) (then). result.y (anyOf1)[]	number	See defined object / value type	cost / original row unit
result.runs[] (allOf1) (then). result.y (anyOf2)	null	See defined object / value type	cost / original row unit
result.runs[] (allOf1) (then). result.metrics	union / composition	See defined object / value type	structural metadata / nested units
result.runs[] (allOf1) (then). result.metrics (anyOf1)	object	additionalProperties=False; required= ['base_objective', 'penalty_objective', 'objective', 'hard_violation', 'lower_ violation', 'upper_violation', 'slack', 'objective_gap_against_lifted']	structural metadata / nested units
result.runs[] (allOf1) (then). result.metrics (anyOf1). base_objective	number	See defined object / value type	model cost unit
result.runs[] (allOf1) (then). result.metrics (anyOf1). penalty_objective	number	See defined object / value type	model cost unit
result.runs[] (allOf1) (then). result.metrics (anyOf1). objective	number	See defined object / value type	model cost unit
result.runs[] (allOf1) (then). result.metrics (anyOf1). hard_violation	number	See defined object / value type	original row units; see unit map
result.runs[] (allOf1) (then). result.metrics (anyOf1). lower_violation	array	See defined object / value type	original row units; see unit map
result.runs[] (allOf1) (then). result.metrics (anyOf1). lower_violation[]	number	See defined object / value type	original row units; see unit map
result.runs[] (allOf1) (then). result.metrics (anyOf1). upper_violation	array	See defined object / value type	original row units; see unit map

Field path	Type	Restrictions	Unit / role
result.runs[] (allOf1) (then). result.metrics (anyOf1). upper_violation[]	number	See defined object / value type	original row units; see unit map
result.runs[] (allOf1) (then). result.metrics (anyOf1).slack	array	See defined object / value type	original row units; see unit map
result.runs[] (allOf1) (then). result.metrics (anyOf1). slack[]	number	See defined object / value type	original row units; see unit map
result.runs[] (allOf1) (then). result.metrics (anyOf1). stationarity_residual	number	See defined object / value type	cost / decision unit; raw gradient norm
result.runs[] (allOf1) (then). result.metrics (anyOf1). objective_gap_against_ lifted	union / composition	See defined object / value type	model cost unit
result.runs[] (allOf1) (then). result.metrics (anyOf1). objective_gap_against_ lifted (anyOf1)	number	See defined object / value type	model cost unit
result.runs[] (allOf1) (then). result.metrics (anyOf1). objective_gap_against_ lifted (anyOf2)	null	See defined object / value type	model cost unit
result.runs[] (allOf1) (then). result.metrics (anyOf2)	null	See defined object / value type	structural metadata / nested units
result.runs[] (allOf1) (then). result.dimensions	object	additionalProperties=False; required= ['variables', 'rows', 'nnz']	structural metadata / nested units
result.runs[] (allOf1) (then). result.dimensions.variables	integer	minimum=0	count / integer identifier
result.runs[] (allOf1) (then). result.dimensions.rows	integer	minimum=0	count / integer identifier
result.runs[] (allOf1) (then). result.dimensions.nnz	integer	minimum=0	count / integer identifier
result.runs[] (allOf1) (then). result.iterations	integer	minimum=0	count / integer identifier
result.runs[] (allOf1) (then). result.setup_seconds	number	minimum=0	s
result.runs[] (allOf1) (then). result.solve_seconds	number	minimum=0	s
result.runs[] (allOf1) (then). result.elapsed_seconds	number	minimum=0	s
result.runs[] (allOf1) (then). result.backend_settings	object	additionalProperties={'type': 'number'}	structural metadata / nested units
result.runs[] (allOf2)	object	See defined object / value type	structural metadata / nested units
result.runs[] (allOf2) (if)	object	See defined object / value type	structural metadata / nested units
result.runs[] (allOf2) (if). state	string	const=incompatible	structural metadata / nested units
result.runs[] (allOf2) (then)	object	See defined object / value type	structural metadata / nested units
result.runs[] (allOf2) (then). reason	string	minLength=1	structural metadata / nested units
result.runs[] (allOf2) (then). result	null	See defined object / value type	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
result.rows	array	See defined object / value type	count / integer identifier
result.rows[]	object	additionalProperties=False; required=['id', 'label', 'unit', 'soft']	count / integer identifier
result.rows[].id	string	pattern=^r[0-9]{4}$	structural metadata / nested units
result.rows[].label	object	additionalProperties=False; required=['en', 'es']	structural metadata / nested units
result.rows[].label.en	string	minLength=1	structural metadata / nested units
result.rows[].label.es	string	minLength=1	structural metadata / nested units
result.rows[].unit	string	minLength=1	structural metadata / nested units
result.rows[].soft	boolean	See defined object / value type	structural metadata / nested units
result.problem	object	additionalProperties=False; required=['P', 'q', 'A', 'lower', 'upper', 'penalty']	structural metadata / nested units
result.problem.P	object	additionalProperties=False; required=['shape', 'data', 'indices', 'indptr']	structural metadata / nested units
result.problem.P.shape	array	minItems=2; maxItems=2	count / integer identifier
result.problem.P.shape[0]	integer	minimum=0; maximum=20000	count / integer identifier
result.problem.P.shape[1]	integer	minimum=0; maximum=20000	count / integer identifier
result.problem.P.data	array	maxItems=2000000	cost / decision units squared
result.problem.P.data[]	number	See defined object / value type	cost / decision units squared
result.problem.P.indices	array	maxItems=2000000	count / integer identifier
result.problem.P.indices[]	integer	minimum=0	count / integer identifier
result.problem.P.indptr	array	maxItems=20001	count / integer identifier
result.problem.P.indptr[]	integer	minimum=0	count / integer identifier
result.problem.q	array	maxItems=20000	cost / decision unit
result.problem.q[]	number	See defined object / value type	cost / decision unit
result.problem.A	object	additionalProperties=False; required=['shape', 'data', 'indices', 'indptr']	structural metadata / nested units
result.problem.A.shape	array	minItems=2; maxItems=2	count / integer identifier
result.problem.A.shape[0]	integer	minimum=0; maximum=20000	count / integer identifier
result.problem.A.shape[1]	integer	minimum=0; maximum=20000	count / integer identifier
result.problem.A.data	array	maxItems=2000000	row / decision unit
result.problem.A.data[]	number	See defined object / value type	row / decision unit
result.problem.A.indices	array	maxItems=2000000	count / integer identifier
result.problem.A.indices[]	integer	minimum=0	count / integer identifier
result.problem.A.indptr	array	maxItems=20001	count / integer identifier

Field path	Type	Restrictions	Unit / role
result.problem.A.indptr[]	integer	minimum=0	count / integer identifier
result.problem.lower	array	maxItems=20000	original row unit
result.problem.lower[]	number / null	See defined object / value type	original row unit
result.problem.upper	array	maxItems=20000	original row unit
result.problem.upper[]	number / null	See defined object / value type	original row unit
result.problem.penalty	object	additionalProperties=False; required=['lower_linear', 'upper_linear', 'lower_quadratic', 'upper_quadratic', 'soft']	structural metadata / nested units
result.problem.penalty.lower_linear	array	maxItems=20000	cost / row unit
result.problem.penalty.lower_linear[]	number	minimum=0	cost / row unit
result.problem.penalty.upper_linear	array	maxItems=20000	cost / row unit
result.problem.penalty.upper_linear[]	number	minimum=0	cost / row unit
result.problem.penalty.lower_quadratic	array	maxItems=20000	cost / row unit squared
result.problem.penalty.lower_quadratic[]	number	minimum=0	cost / row unit squared
result.problem.penalty.upper_quadratic	array	maxItems=20000	cost / row unit squared
result.problem.penalty.upper_quadratic[]	number	minimum=0	cost / row unit squared
result.problem.penalty.soft	array	maxItems=20000	structural metadata / nested units
result.problem.penalty.soft[]	boolean	See defined object / value type	structural metadata / nested units
environment	object	additionalProperties=False; required=['python', 'platform', 'packages']	structural metadata / nested units
environment.python	string	See defined object / value type	structural metadata / nested units
environment.platform	string	See defined object / value type	structural metadata / nested units
environment.packages	object	additionalProperties={'type': 'string'}	structural metadata / nested units
environment.engine_revision	string / null	pattern=^[0-9a-f]{40}$	structural metadata / nested units
environment.engine_source_dirty	boolean / null	See defined object / value type	structural metadata / nested units
environment.native_binary_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units

Nested fields of softqp-stream are enumerated in Table 33.

Table 33: Complete nested fields, value types, literal schema restrictions and physical or structural dimensions for softqp-stream. Required and optional structures retain the original schema and model interpretation.

Field path	Type	Restrictions	Unit / role
schema_version	string	const=softqp.stream/v1	structural metadata / nested units
case_id	string	const=C12	structural metadata / nested units
protocol_id	string	const=softqp-experiments-v1	structural metadata / nested units
engine_source	string	pattern=^[0-9a-f]{40}$	structural metadata / nested units
environment	object	additionalProperties=False; required=['python', 'platform', 'packages']	structural metadata / nested units
environment.python	string	See defined object / value type	structural metadata / nested units
environment.platform	string	See defined object / value type	structural metadata / nested units
environment.packages	object	additionalProperties={'type': 'string'}	structural metadata / nested units
environment.engine_revision	string / null	pattern=^[0-9a-f]{40}$	structural metadata / nested units
environment.engine_source_dirty	boolean / null	See defined object / value type	structural metadata / nested units
environment.native_binary_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
specification	object	additionalProperties=False; required=['case_id', 'frames', 'horizon', 'dt_seconds', 'initial_state', 'velocity_impulses', 'policy', 'methods', 'modes', 'timing']	structural metadata / nested units
specification.case_id	string	const=C12	structural metadata / nested units
specification.frames	integer	const=32	count / integer identifier
specification.horizon	integer	const=20	count / integer identifier
specification.dt_seconds	number	const=0.2	s
specification.initial_state	array	minItems=2; maxItems=2	components: m and m/s
specification.initial_state[0]	number	const=1.6	m
specification.initial_state[1]	number	const=0.0	m/s
specification.velocity_impulses	object	additionalProperties=False; required=['8', '20']	structural metadata / nested units
specification.velocity_impulses.8	number	const=0.8	m/s; key is frame index
specification.velocity_impulses.20	number	const=-0.6	m/s; key is frame index
specification.policy	string	const=Independent stock lifting drives the common plant trajectory. Every compared method receives identical state inputs. These are paired solver observations, not separate controller rollouts.	structural metadata / nested units
specification.methods	array	minItems=4; maxItems=4	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
specification.methods[0]	string	const=lifted	structural metadata / nested units
specification.methods[1]	string	const=generalized	structural metadata / nested units
specification.methods[2]	string	const=asymmetric	structural metadata / nested units
specification.methods[3]	string	const=daqp	structural metadata / nested units
specification.modes	array	minItems=2; maxItems=2	structural metadata / nested units
specification.modes[0]	string	const=cold	structural metadata / nested units
specification.modes[1]	string	const=warm	structural metadata / nested units
specification.timing	string	const=one documented stream observation per frame; no statistical superiority claim from it	structural metadata / nested units
frames	array	minItems=32; maxItems=32	count / integer identifier
frames[]	object	additionalProperties=False; required=['frame', 'time_seconds', 'state', 'velocity_impulse', 'oracle_objective', 'objective_constant', 'oracle_horizon_cost', 'oracle_control', 'applied_control', 'next_state', 'generation_seconds', 'runs']	count / integer identifier
frames[].frame	integer	minimum=0	count / integer identifier
frames[].time_seconds	number	minimum=0	s
frames[].state	array	minItems=2; maxItems=2	C12 components: m and m/s
frames[].state[]	number	See defined object / value type	C12 components: m and m/s
frames[].velocity_impulse	number	See defined object / value type	m/s
frames[].oracle_objective	number	See defined object / value type	model cost unit
frames[].objective_constant	number	See defined object / value type	model cost unit
frames[].oracle_horizon_cost	number	See defined object / value type	model cost unit
frames[].oracle_control	number	See defined object / value type	m/s squared
frames[].applied_control	number	See defined object / value type	m/s squared
frames[].next_state	array	minItems=2; maxItems=2	C12 components: m and m/s
frames[].next_state[]	number	See defined object / value type	C12 components: m and m/s
frames[].generation_seconds	number	minimum=0	s
frames[].runs	array	minItems=8; maxItems=8	structural metadata / nested units
frames[].runs[]	object	additionalProperties=False; required=['method', 'mode', 'state', 'reason', 'metrics', 'normalized_gap', 'elapsed_seconds', 'update_seconds', 'workspace_setup_seconds', 'result']	structural metadata / nested units
frames[].runs[].method	enumerated value	enum=['lifted', 'generalized', 'asymmetric', 'daqp', 'flex', 'elastiqp']	structural metadata / nested units
frames[].runs[].mode	enumerated value	enum=['cold', 'warm']	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
frames[].runs[].state	enumerated value	enum=['accepted', 'incompatible', 'failed', 'accuracy-rejected', 'timeout', 'resource-limit', 'oracle-failed']	structural metadata / nested units
frames[].runs[].reason	union / composition	See defined object / value type	structural metadata / nested units
frames[].runs[].reason (anyOf1)	string	See defined object / value type	structural metadata / nested units
frames[].runs[].reason (anyOf2)	null	See defined object / value type	structural metadata / nested units
frames[].runs[].metrics	union / composition	See defined object / value type	structural metadata / nested units
frames[].runs[].metrics (anyOf1)	object	additionalProperties=False; required=['base_objective', 'penalty_objective', 'objective', 'hard_violation', 'lower_violation', 'upper_violation', 'slack']	structural metadata / nested units
frames[].runs[].metrics (anyOf1).base_objective	number	See defined object / value type	model cost unit
frames[].runs[].metrics (anyOf1).penalty_objective	number	See defined object / value type	model cost unit
frames[].runs[].metrics (anyOf1).objective	number	See defined object / value type	model cost unit
frames[].runs[].metrics (anyOf1).hard_violation	number	See defined object / value type	original row units; see unit map
frames[].runs[].metrics (anyOf1).lower_violation	array	See defined object / value type	original row units; see unit map
frames[].runs[].metrics (anyOf1).lower_violation[]	number	See defined object / value type	original row units; see unit map
frames[].runs[].metrics (anyOf1).upper_violation	array	See defined object / value type	original row units; see unit map
frames[].runs[].metrics (anyOf1).upper_violation[]	number	See defined object / value type	original row units; see unit map
frames[].runs[].metrics (anyOf1).slack	array	See defined object / value type	original row units; see unit map
frames[].runs[].metrics (anyOf1).slack[]	number	See defined object / value type	original row units; see unit map
frames[].runs[].metrics (anyOf1).stationarity_residual	union / composition	See defined object / value type	cost / decision unit; raw gradient norm
frames[].runs[].metrics (anyOf1).stationarity_residual (anyOf1)	number	minimum=0	cost / decision unit; raw gradient norm
frames[].runs[].metrics (anyOf1).stationarity_residual (anyOf2)	null	See defined object / value type	cost / decision unit; raw gradient norm
frames[].runs[].metrics (anyOf1).objective_gap_against_lifted	union / composition	See defined object / value type	model cost unit
frames[].runs[].metrics (anyOf1).objective_gap_against_lifted (anyOf1)	number	See defined object / value type	model cost unit
frames[].runs[].metrics (anyOf1).objective_gap_against_lifted (anyOf2)	null	See defined object / value type	model cost unit
frames[].runs[].metrics (anyOf2)	null	See defined object / value type	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
frames[].runs[].normalized_gap	union / composition	See defined object / value type	dimensionless normalized diagnostic / factor
frames[].runs[].normalized_gap (anyOf1)	number	minimum=0	dimensionless normalized diagnostic / factor
frames[].runs[].normalized_gap (anyOf2)	null	See defined object / value type	dimensionless normalized diagnostic / factor
frames[].runs[].elapsed_seconds	union / composition	See defined object / value type	s
frames[].runs[].elapsed_seconds (anyOf1)	number	minimum=0	s
frames[].runs[].elapsed_seconds (anyOf2)	null	See defined object / value type	s
frames[].runs[].update_seconds	union / composition	See defined object / value type	s
frames[].runs[].update_seconds (anyOf1)	number	minimum=0	s
frames[].runs[].update_seconds (anyOf2)	null	See defined object / value type	s
frames[].runs[].workspace_setup_seconds	union / composition	See defined object / value type	s
frames[].runs[].workspace_setup_seconds (anyOf1)	number	minimum=0	s
frames[].runs[].workspace_setup_seconds (anyOf2)	null	See defined object / value type	s
frames[].runs[].result	union / composition	See defined object / value type	structural metadata / nested units
frames[].runs[].result (anyOf1)	object	additionalProperties=False; required=['method', 'status', 'status_code', 'x', 'metrics', 'dimensions', 'iterations', 'setup_seconds', 'solve_seconds', 'elapsed_seconds']	structural metadata / nested units
frames[].runs[].result (anyOf1).method	string	See defined object / value type	structural metadata / nested units
frames[].runs[].result (anyOf1).status	string	See defined object / value type	structural metadata / nested units
frames[].runs[].result (anyOf1).status_code	integer	See defined object / value type	count / integer identifier
frames[].runs[].result (anyOf1).x	union / composition	See defined object / value type	decision unit
frames[].runs[].result (anyOf1).x (anyOf1)	array	See defined object / value type	decision unit
frames[].runs[].result (anyOf1).x (anyOf1)[]	number	See defined object / value type	decision unit
frames[].runs[].result (anyOf1).x (anyOf2)	null	See defined object / value type	decision unit
frames[].runs[].result (anyOf1).y	union / composition	See defined object / value type	cost / original row unit
frames[].runs[].result (anyOf1).y (anyOf1)	array	See defined object / value type	cost / original row unit
frames[].runs[].result (anyOf1).y (anyOf1)[]	number	See defined object / value type	cost / original row unit
frames[].runs[].result (anyOf1).y (anyOf2)	null	See defined object / value type	cost / original row unit

Field path	Type	Restrictions	Unit / role
frames[].runs[].result (anyOf1).metrics	union / composition	See defined object / value type	structural metadata / nested units
frames[].runs[].result (anyOf1).metrics (anyOf1)	object	additionalProperties=False; required=['base_objective', 'penalty_objective', 'objective', 'hard_violation', 'lower_violation', 'upper_violation', 'slack']	structural metadata / nested units
frames[].runs[].result (anyOf1).metrics (anyOf1).base_objective	number	See defined object / value type	model cost unit
frames[].runs[].result (anyOf1).metrics (anyOf1).penalty_objective	number	See defined object / value type	model cost unit
frames[].runs[].result (anyOf1).metrics (anyOf1).objective	number	See defined object / value type	model cost unit
frames[].runs[].result (anyOf1).metrics (anyOf1).hard_violation	number	See defined object / value type	original row units; see unit map
frames[].runs[].result (anyOf1).metrics (anyOf1).lower_violation	array	See defined object / value type	original row units; see unit map
frames[].runs[].result (anyOf1).metrics (anyOf1).lower_violation[]	number	See defined object / value type	original row units; see unit map
frames[].runs[].result (anyOf1).metrics (anyOf1).upper_violation	array	See defined object / value type	original row units; see unit map
frames[].runs[].result (anyOf1).metrics (anyOf1).upper_violation[]	number	See defined object / value type	original row units; see unit map
frames[].runs[].result (anyOf1).metrics (anyOf1).slack	array	See defined object / value type	original row units; see unit map
frames[].runs[].result (anyOf1).metrics (anyOf1).slack[]	number	See defined object / value type	original row units; see unit map
frames[].runs[].result (anyOf1).metrics (anyOf1).stationarity_residual	union / composition	See defined object / value type	cost / decision unit; raw gradient norm
frames[].runs[].result (anyOf1).metrics (anyOf1).stationarity_residual (anyOf1)	number	minimum=0	cost / decision unit; raw gradient norm
frames[].runs[].result (anyOf1).metrics (anyOf1).stationarity_residual (anyOf2)	null	See defined object / value type	cost / decision unit; raw gradient norm
frames[].runs[].result (anyOf1).metrics (anyOf1).objective_gap_against_lifted	union / composition	See defined object / value type	model cost unit
frames[].runs[].result (anyOf1).metrics (anyOf1).objective_gap_against_lifted (anyOf1)	number	See defined object / value type	model cost unit

Field path	Type	Restrictions	Unit / role
frames[].runs[].result (anyOf1).metrics (anyOf1). objective_gap_against_ lifted (anyOf2)	null	See defined object / value type	model cost unit
frames[].runs[].result (anyOf1).metrics (anyOf2)	null	See defined object / value type	structural metadata / nested units
frames[].runs[].result (anyOf1).dimensions	object	additionalProperties=False; required= ['variables', 'rows', 'nnz']	structural metadata / nested units
frames[].runs[].result (anyOf1).dimensions. variables	integer	minimum=0	count / integer identifier
frames[].runs[].result (anyOf1).dimensions.rows	integer	minimum=0	count / integer identifier
frames[].runs[].result (anyOf1).dimensions.nnz	integer	minimum=0	count / integer identifier
frames[].runs[].result (anyOf1).iterations	integer	minimum=0	count / integer identifier
frames[].runs[].result (anyOf1).setup_seconds	number	minimum=0	s
frames[].runs[].result (anyOf1).solve_seconds	number	minimum=0	s
frames[].runs[].result (anyOf1).elapsed_seconds	number	minimum=0	s
frames[].runs[].result (anyOf1).backend_settings	object	additionalProperties={'type': 'number'}	structural metadata / nested units
frames[].runs[].result (anyOf2)	null	See defined object / value type	structural metadata / nested units
counts	object	additionalProperties=False; required=[]	structural metadata / nested units
counts.accepted	integer	minimum=0	count / integer identifier
counts.incompatible	integer	minimum=0	count / integer identifier
counts.failed	integer	minimum=0	count / integer identifier
counts.accuracy-rejected	integer	minimum=0	count / integer identifier
counts.timeout	integer	minimum=0	count / integer identifier
counts.resource-limit	integer	minimum=0	count / integer identifier
counts.oracle-failed	integer	minimum=0	count / integer identifier
limitations	array	See defined object / value type	structural metadata / nested units
limitations[]	string	See defined object / value type	structural metadata / nested units

Nested fields of softqp-campaign are enumerated in Table 34.

Table 34: Complete nested fields, value types, literal schema restrictions and physical or structural dimensions for softqp-campaign. Required and optional structures retain the original schema and model interpretation.

Field path	Type	Restrictions	Unit / role
schema_version	string	const=softqp.campaign/v1	structural metadata / nested units
protocol	object	additionalProperties=False; required=['protocol_id', 'phase', 'created', 'engine_source', 'operating_limits', 'accuracy', 'methods', 'backends', 'generator', 'seeds', 'repetitions', 'primary_endpoint', 'secondary_endpoints', 'robustness', 'stream', 'measurement_scope', 'failure_policy', 'claims']	structural metadata / nested units
protocol.protocol_id	string	const=softqp-experiments-v1	structural metadata / nested units
protocol.phase	string	const=registered before held-out generation	structural metadata / nested units
protocol.created	string	const=2026-10-31	structural metadata / nested units
protocol.engine_source	string	const=d0c69ed974d577ba02adbd4806332270d9425651	structural metadata / nested units
protocol.operating_limits	object	additionalProperties=False; required=['blas_threads', 'wall_seconds', 'process_memory_limit_bytes', 'parallel_workers', 'trial_timeout_seconds']	structural metadata / nested units
protocol.operating_limits.blas_threads	integer	const=1	count / integer identifier
protocol.operating_limits.wall_seconds	integer	const=7200	s
protocol.operating_limits.process_memory_limit_bytes	integer	const=4294967296	bytes
protocol.operating_limits.parallel_workers	integer	const=1	count / integer identifier
protocol.operating_limits.trial_timeout_seconds	integer	const=10	s
protocol.accuracy	object	additionalProperties=False; required=['eps_abs', 'eps_rel', 'max_iter', 'adaptive_rho_interval', 'normalized_objective_gap_limit', 'hard_row_error_limit']	structural metadata / nested units
protocol.accuracy.eps_abs	number	const=1e-07	solver coordinate tolerance; model scaling applies
protocol.accuracy.eps_rel	number	const=1e-07	dimensionless normalized diagnostic / factor
protocol.accuracy.max_iter	integer	const=200000	count / integer identifier
protocol.accuracy.adaptive_rho_interval	integer	const=25	count / integer identifier
protocol.accuracy.normalized_objective_gap_limit	number	const=1e-06	dimensionless

Field path	Type	Restrictions	Unit / role
protocol.accuracy.hard_row_error_limit	number	const=1e-06	original row unit; synthetic coordinates
protocol.methods	array	minItems=6; maxItems=6	structural metadata / nested units
protocol.methods[0]	string	const=lifted	structural metadata / nested units
protocol.methods[1]	string	const=generalized	structural metadata / nested units
protocol.methods[2]	string	const=asymmetric	structural metadata / nested units
protocol.methods[3]	string	const=daqp	structural metadata / nested units
protocol.methods[4]	string	const=flex	structural metadata / nested units
protocol.methods[5]	string	const=elastiqp	structural metadata / nested units
protocol.backends	object	additionalProperties=False; required=['lifted_timed', 'lifted_oracle', 'flex', 'elastiqp']	structural metadata / nested units
protocol.backends.lifted_timed	string	const=pinned native fork	structural metadata / nested units
protocol.backends.lifted_oracle	string	const=independent stock OSQP with eps_abs=eps_rel=1e-9	structural metadata / nested units
protocol.backends.flex	string	const=fixed-parameter paper-equation reconstruction	structural metadata / nested units
protocol.backends.elastiqp	string	const=published 0.2.0 dual active set	structural metadata / nested units
protocol.generator	object	additionalProperties=False; required=['n', 'm_per_n', 'density', 'hessian_condition', 'families', 'hard_row_fraction_elastic', 'hard_row_fraction_l1', 'row_norm', 'q_standard_deviation', 'linear_cost_range', 'quadratic_cost_range', 'asymmetric_upper_factor_range', 'lower', 'upper']	structural metadata / nested units
protocol.generator.n	array	minItems=4; maxItems=4	count / integer identifier
protocol.generator.n[0]	integer	const=32	count / integer identifier
protocol.generator.n[1]	integer	const=64	count / integer identifier
protocol.generator.n[2]	integer	const=128	count / integer identifier
protocol.generator.n[3]	integer	const=256	count / integer identifier
protocol.generator.m_per_n	integer	const=2	dimensionless
protocol.generator.density	array	minItems=2; maxItems=2	dimensionless normalized diagnostic / factor
protocol.generator.density[0]	number	const=0.05	dimensionless normalized diagnostic / factor
protocol.generator.density[1]	number	const=0.2	dimensionless normalized diagnostic / factor

Field path	Type	Restrictions	Unit / role
protocol.generator.hessian_condition	array	minItems=2; maxItems=2	dimensionless
protocol.generator.hessian_condition[0]	integer	const=1	dimensionless
protocol.generator.hessian_condition[1]	integer	const=1000	dimensionless
protocol.generator.families	array	minItems=3; maxItems=3	structural metadata / nested units
protocol.generator.families[0]	string	const=symmetric-elastic	structural metadata / nested units
protocol.generator.families[1]	string	const=asymmetric-elastic	structural metadata / nested units
protocol.generator.families[2]	string	const=asymmetric-l1	structural metadata / nested units
protocol.generator.hard_row_fraction_elastic	number	const=0.2	dimensionless
protocol.generator.hard_row_fraction_l1	integer	const=0	dimensionless
protocol.generator.row_norm	integer	const=1	row / decision unit; generator coordinates
protocol.generator.q_standard_deviation	integer	const=3	cost / decision unit
protocol.generator.linear_cost_range	array	minItems=2; maxItems=2	cost / row unit
protocol.generator.linear_cost_range[0]	number	const=0.1	cost / row unit
protocol.generator.linear_cost_range[1]	number	const=1.0	cost / row unit
protocol.generator.quadratic_cost_range	array	minItems=2; maxItems=2	cost / row unit squared
protocol.generator.quadratic_cost_range[0]	number	const=0.5	cost / row unit squared
protocol.generator.quadratic_cost_range[1]	number	const=4.0	cost / row unit squared
protocol.generator.asymmetric_upper_factor_range	array	minItems=2; maxItems=2	dimensionless
protocol.generator.asymmetric_upper_factor_range[0]	number	const=1.5	dimensionless
protocol.generator.asymmetric_upper_factor_range[1]	number	const=3.0	dimensionless
protocol.generator.lower	number	const=-0.4	original row unit
protocol.generator.upper	number	const=0.6	original row unit
protocol.seeds	object	additionalProperties=False; required=['development', 'heldout', 'robustness', 'order', 'bootstrap']	structural metadata / nested units
protocol.seeds.development	array	minItems=3; maxItems=3	integer random seed
protocol.seeds.development[0]	integer	const=1000	integer random seed
protocol.seeds.development[1]	integer	const=1001	integer random seed
protocol.seeds.development[2]	integer	const=1002	integer random seed
protocol.seeds.heldout	array	minItems=6; maxItems=6	integer random seed

Field path	Type	Restrictions	Unit / role
protocol.seeds.heldout[0]	integer	const=9000	integer random seed
protocol.seeds.heldout[1]	integer	const=9001	integer random seed
protocol.seeds.heldout[2]	integer	const=9002	integer random seed
protocol.seeds.heldout[3]	integer	const=9003	integer random seed
protocol.seeds.heldout[4]	integer	const=9004	integer random seed
protocol.seeds.heldout[5]	integer	const=9005	integer random seed
protocol.seeds.robustness	array	minItems=6; maxItems=6	integer random seed
protocol.seeds.robustness[0]	integer	const=8500	integer random seed
protocol.seeds.robustness[1]	integer	const=8501	integer random seed
protocol.seeds.robustness[2]	integer	const=8502	integer random seed
protocol.seeds.robustness[3]	integer	const=8503	integer random seed
protocol.seeds.robustness[4]	integer	const=8504	integer random seed
protocol.seeds.robustness[5]	integer	const=8505	integer random seed
protocol.seeds.order	integer	const=271828	integer random seed
protocol.seeds.bootstrap	integer	const=314159	integer random seed
protocol.repetitions	object	additionalProperties=False; required=['warmup_per_backend', 'measured', 'paired_order', 'bootstrap_seed_clusters']	structural metadata / nested units
protocol.repetitions.warmup_per_backend	integer	const=1	count / integer identifier
protocol.repetitions.measured	integer	const=7	count / integer identifier
protocol.repetitions.paired_order	string	const=seeded random method order within each instance and repetition	structural metadata / nested units
protocol.repetitions.bootstrap_seed_clusters	integer	const=2000	count / integer identifier
protocol.primary_endpoint	object	additionalProperties=False; required=['family', 'n_minimum', 'density', 'quantity', 'minimum_reduction', 'confidence', 'success_condition']	structural metadata / nested units
protocol.primary_endpoint.family	string	const=asymmetric-elastic	structural metadata / nested units
protocol.primary_endpoint.n_minimum	integer	const=128	count / integer identifier
protocol.primary_endpoint.density	number	const=0.05	dimensionless normalized diagnostic / factor
protocol.primary_endpoint.quantity	string	const=median paired relative reduction in cold end-to-end time against the faster of native compact lifting and DAQP	structural metadata / nested units
protocol.primary_endpoint.minimum_reduction	number	const=0.1	dimensionless
protocol.primary_endpoint.confidence	string	const=two-sided 95 percent seed-cluster bootstrap interval has a lower endpoint above zero	structural metadata / nested units
protocol.primary_endpoint.success_condition	string	const=all compared cells satisfy external original-space acceptance; failures are retained	structural metadata / nested units
protocol.secondary_endpoints	array	minItems=5; maxItems=5	structural metadata / nested units
protocol.secondary_endpoints[0]	string	const=native solve/setup and update time	structural metadata / nested units
protocol.secondary_endpoints[1]	string	const=iterations	structural metadata / nested units
protocol.secondary_endpoints[2]	string	const=formulation dimensions and nonzeros	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
protocol.secondary_endpoints[3]	string	const=accuracy versus conditioning	structural metadata / nested units
protocol.secondary_endpoints[4]	string	const=within-method warm/cold stream measurements	structural metadata / nested units
protocol.robustness	object	additionalProperties=False; required=['n', 'm', 'density', 'hessian_condition', 'same_A_q_and_costs_within_seed']	structural metadata / nested units
protocol.robustness.n	integer	const=32	count / integer identifier
protocol.robustness.m	integer	const=64	count / integer identifier
protocol.robustness.density	number	const=0.1	dimensionless normalized diagnostic / factor
protocol.robustness.hessian_condition	array	minItems=4; maxItems=4	dimensionless
protocol.robustness.hessian_condition[0]	integer	const=1	dimensionless
protocol.robustness.hessian_condition[1]	integer	const=100	dimensionless
protocol.robustness.hessian_condition[2]	integer	const=10000	dimensionless
protocol.robustness.hessian_condition[3]	integer	const=1000000	dimensionless
protocol.robustness.same_A_q_and_costs_within_seed	boolean	const=True	structural metadata / nested units
protocol.stream	object	additionalProperties=False; required=['case_id', 'frames', 'horizon', 'dt_seconds', 'initial_state', 'velocity_impulses', 'policy', 'methods', 'modes', 'timing']	structural metadata / nested units
protocol.stream.case_id	string	const=C12	structural metadata / nested units
protocol.stream.frames	integer	const=32	count / integer identifier
protocol.stream.horizon	integer	const=20	count / integer identifier
protocol.stream.dt_seconds	number	const=0.2	s
protocol.stream.initial_state	array	minItems=2; maxItems=2	components: m and m/s
protocol.stream.initial_state[0]	number	const=1.6	m
protocol.stream.initial_state[1]	number	const=0.0	m/s
protocol.stream.velocity_impulses	object	additionalProperties=False; required=['8', '20']	structural metadata / nested units
protocol.stream.velocity_impulses.8	number	const=0.8	m/s; key is frame index
protocol.stream.velocity_impulses.20	number	const=-0.6	m/s; key is frame index
protocol.stream.policy	string	const=Independent stock lifting drives the common plant trajectory. Every compared method receives identical state inputs. These are paired solver observations, not separate controller rollouts.	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
protocol.stream.methods	array	minItems=4; maxItems=4	structural metadata / nested units
protocol.stream.methods[0]	string	const=lifted	structural metadata / nested units
protocol.stream.methods[1]	string	const=generalized	structural metadata / nested units
protocol.stream.methods[2]	string	const=asymmetric	structural metadata / nested units
protocol.stream.methods[3]	string	const=daqp	structural metadata / nested units
protocol.stream.modes	array	minItems=2; maxItems=2	structural metadata / nested units
protocol.stream.modes[0]	string	const=cold	structural metadata / nested units
protocol.stream.modes[1]	string	const=warm	structural metadata / nested units
protocol.stream.timing	string	const=one documented stream observation per frame; no statistical superiority claim from it	structural metadata / nested units
protocol.measurement_scope	object	additionalProperties=False; required=['cold_end_to_end', 'generation', 'warm_end_to_end', 'factorization', 'memory']	structural metadata / nested units
protocol.measurement_scope.cold_end_to_end	string	const=validated original QP input to independently evaluated returned result; includes validation, formulation construction, setup and solve; excludes generator, module import and export	structural metadata / nested units
protocol.measurement_scope.generation	string	const=measured separately	structural metadata / nested units
protocol.measurement_scope.warm_end_to_end	string	const=vector validation, update, solve and original-space evaluation; one initial workspace setup is recorded separately	structural metadata / nested units
protocol.measurement_scope.factorization	string	const=backend setup currently includes linear-system preparation; isolated numeric factor time is not instrumented and cannot be inferred	structural metadata / nested units
protocol.measurement_scope.memory	string	const=process RSS and campaign high-water mark are resource observations, not per-method native allocation peaks	structural metadata / nested units
protocol.failure_policy	string	const=Preserve incompatible, maximum-iteration, timeout, numerical and accuracy-rejected cells with reasons. Never replace an objective or drop a failed evaluation seed.	structural metadata / nested units
protocol.claims	string	const=A successful implementation benchmark does not establish global algorithmic novelty, safety or universal solver superiority. Deep FlexQP trained weights are not available in this reconstruction.	structural metadata / nested units
environment	object	additionalProperties=False; required=['python', 'platform', 'packages']	structural metadata / nested units
environment.python	string	See defined object / value type	structural metadata / nested units
environment.platform	string	See defined object / value type	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
environment.packages	object	additionalProperties={'type': 'string'}	structural metadata / nested units
environment.engine_revision	string / null	pattern=^[0-9a-f]{40}$	structural metadata / nested units
environment.engine_source_dirty	boolean / null	See defined object / value type	structural metadata / nested units
environment.native_binary_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
protocol_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
source_sha256	object	additionalProperties={'type': 'string', 'pattern': '^[0-9a-f]{64}$'}	structural metadata / nested units
observed_wall_seconds	number	minimum=0	s
worker_peak_rss_bytes	integer	minimum=0	bytes
worker_restarts	integer	minimum=0	count / integer identifier
planned_instances	integer	minimum=0	count / integer identifier
completed_instances	integer	minimum=0	count / integer identifier
cells	array	See defined object / value type	structural metadata / nested units
cells[]	object	additionalProperties=False; required=['split', 'n', 'density', 'condition', 'family', 'method', 'counts', 'median_seconds', 'quantile_25_seconds', 'quantile_75_seconds', 'max_normalized_gap', 'max_hard_violation']	structural metadata / nested units
cells[].split	enumerated value	enum=['development', 'heldout', 'robustness']	structural metadata / nested units
cells[].n	integer	minimum=0	count / integer identifier
cells[].density	number	minimum=0	dimensionless normalized diagnostic / factor
cells[].condition	number	minimum=0	dimensionless normalized diagnostic / factor
cells[].family	enumerated value	enum=['symmetric-elastic', 'asymmetric-elastic', 'asymmetric-l1']	structural metadata / nested units
cells[].method	enumerated value	enum=['lifted', 'generalized', 'asymmetric', 'daqp', 'flex', 'elastiqp']	structural metadata / nested units
cells[].counts	object	additionalProperties=False; required=[]	structural metadata / nested units
cells[].counts.accepted	integer	minimum=0	count / integer identifier
cells[].counts.incompatible	integer	minimum=0	count / integer identifier
cells[].counts.failed	integer	minimum=0	count / integer identifier
cells[].counts.accuracy-rejected	integer	minimum=0	count / integer identifier
cells[].counts.timeout	integer	minimum=0	count / integer identifier
cells[].counts.resource-limit	integer	minimum=0	count / integer identifier

Field path	Type	Restrictions	Unit / role
cells[].counts.oracle-failed	integer	minimum=0	count / integer identifier
cells[].median_seconds	union / composition	See defined object / value type	s
cells[].median_seconds (anyOf1)	number	minimum=0	s
cells[].median_seconds (anyOf2)	null	See defined object / value type	s
cells[].quantile_25_seconds	union / composition	See defined object / value type	s
cells[].quantile_25_seconds (anyOf1)	number	minimum=0	s
cells[].quantile_25_seconds (anyOf2)	null	See defined object / value type	s
cells[].quantile_75_seconds	union / composition	See defined object / value type	s
cells[].quantile_75_seconds (anyOf1)	number	minimum=0	s
cells[].quantile_75_seconds (anyOf2)	null	See defined object / value type	s
cells[].max_normalized_gap	union / composition	See defined object / value type	dimensionless normalized diagnostic / factor
cells[].max_normalized_gap (anyOf1)	number	minimum=0	dimensionless normalized diagnostic / factor
cells[].max_normalized_gap (anyOf2)	null	See defined object / value type	dimensionless normalized diagnostic / factor
cells[].max_hard_violation	union / composition	See defined object / value type	original row units; see unit map
cells[].max_hard_violation (anyOf1)	number	minimum=0	original row units; see unit map
cells[].max_hard_violation (anyOf2)	null	See defined object / value type	original row units; see unit map
primary_endpoint	object	additionalProperties=False; required=['state', 'median_relative_reduction', 'confidence_interval_95', 'paired_samples', 'rejected_pairs', 'seed_clusters']	structural metadata / nested units
primary_endpoint.state	enumerated value	enum=['met', 'not-established']	structural metadata / nested units
primary_endpoint.median_relative_reduction	union / composition	See defined object / value type	dimensionless normalized diagnostic / factor
primary_endpoint.median_relative_reduction (anyOf1)	number	See defined object / value type	dimensionless normalized diagnostic / factor
primary_endpoint.median_relative_reduction (anyOf2)	null	See defined object / value type	dimensionless normalized diagnostic / factor
primary_endpoint.confidence_interval_95	union / composition	See defined object / value type	dimensionless normalized diagnostic / factor
primary_endpoint.confidence_interval_95 (anyOf1)	array	minItems=2; maxItems=2	dimensionless normalized diagnostic / factor

Field path	Type	Restrictions	Unit / role
primary_endpoint.confidence_interval_95 (anyOf1)[]	number	See defined object / value type	dimensionless normalized diagnostic / factor
primary_endpoint.confidence_interval_95 (anyOf2)	null	See defined object / value type	dimensionless normalized diagnostic / factor
primary_endpoint.paired_samples	integer	minimum=0	count / integer identifier
primary_endpoint.rejected_pairs	integer	minimum=0	count / integer identifier
primary_endpoint.seed_clusters	integer	minimum=0	count / integer identifier
counts	object	additionalProperties=False; required=[]	structural metadata / nested units
counts.accepted	integer	minimum=0	count / integer identifier
counts.incompatible	integer	minimum=0	count / integer identifier
counts.failed	integer	minimum=0	count / integer identifier
counts.accuracy-rejected	integer	minimum=0	count / integer identifier
counts.timeout	integer	minimum=0	count / integer identifier
counts.resource-limit	integer	minimum=0	count / integer identifier
counts.oracle-failed	integer	minimum=0	count / integer identifier
artifacts	array	See defined object / value type	structural metadata / nested units
artifacts[]	object	additionalProperties=False; required=['path', 'bytes', 'sha256']	structural metadata / nested units
artifacts[].path	string	pattern=^[a-z][a-z0-9-]*\.(json|csv|jsonl)$	structural metadata / nested units
artifacts[].bytes	integer	minimum=0	bytes
artifacts[].sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units

Nested fields of softqp-datasets are enumerated in Table 35.

Table 35: Complete nested fields, value types, literal schema restrictions and physical or structural dimensions for softqp-datasets. Required and optional structures retain the original schema and model interpretation.

Field path	Type	Restrictions	Unit / role
id	string	pattern=^(development|heldout|robustness)-[0-9]{4}$	structural metadata / nested units
split	enumerated value	enum=['development', 'heldout', 'robustness']	structural metadata / nested units
n	integer	minimum=0	count / integer identifier
density	number	minimum=0	dimensionless normalized diagnostic / factor

Field path	Type	Restrictions	Unit / role
condition	number	minimum=0	dimensionless normalized diagnostic / factor
family	enumerated value	enum=['symmetric-elastic', 'asymmetric-elastic', 'asymmetric-l1']	structural metadata / nested units
seed	integer	minimum=0	count / integer identifier
generation_seconds	number	minimum=0	s
problem_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
oracle_state	enumerated value	enum=['accepted', 'oracle-failed']	structural metadata / nested units
oracle_reason	union / composition	See defined object / value type	structural metadata / nested units
oracle_reason (anyOf1)	string	See defined object / value type	structural metadata / nested units
oracle_reason (anyOf2)	null	See defined object / value type	structural metadata / nested units
oracle_objective	union / composition	See defined object / value type	model cost unit
oracle_objective (anyOf1)	number	See defined object / value type	model cost unit
oracle_objective (anyOf2)	null	See defined object / value type	model cost unit

Nested fields of softqp-trial are enumerated in Table 36.

Table 36: Complete nested fields, value types, literal schema restrictions and physical or structural dimensions for softqp-trial. Required and optional structures retain the original schema and model interpretation.

Field path	Type	Restrictions	Unit / role
id	string	pattern=^(development|heldout|robustness)-[0-9]{4}$	structural metadata / nested units
split	enumerated value	enum=['development', 'heldout', 'robustness']	structural metadata / nested units
n	integer	minimum=0	count / integer identifier
density	number	minimum=0	dimensionless normalized diagnostic / factor
condition	number	minimum=0	dimensionless normalized diagnostic / factor
family	enumerated value	enum=['symmetric-elastic', 'asymmetric-elastic', 'asymmetric-l1']	structural metadata / nested units
seed	integer	minimum=0	count / integer identifier
method	enumerated value	enum=['lifted', 'generalized', 'asymmetric', 'daqp', 'flex', 'elastiqp']	structural metadata / nested units
repetition	integer	minimum=-1	count / integer identifier
warmup	boolean	See defined object / value type	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
state	enumerated value	enum=['accepted', 'incompatible', 'failed', 'accuracy-rejected', 'timeout', 'resource-limit', 'oracle-failed']	structural metadata / nested units
reason	union / composition	See defined object / value type	structural metadata / nested units
reason (anyOf1)	string	See defined object / value type	structural metadata / nested units
reason (anyOf2)	null	See defined object / value type	structural metadata / nested units
metrics	union / composition	See defined object / value type	structural metadata / nested units
metrics (anyOf1)	object	additionalProperties=False; required=['objective', 'base_objective', 'penalty_objective', 'hard_violation', 'stationarity_residual', 'total_lower_violation', 'total_upper_violation']	structural metadata / nested units
metrics (anyOf1).objective	number	See defined object / value type	model cost unit
metrics (anyOf1).base_objective	number	See defined object / value type	model cost unit
metrics (anyOf1).penalty_objective	number	See defined object / value type	model cost unit
metrics (anyOf1).hard_violation	number	See defined object / value type	original row units; see unit map
metrics (anyOf1).stationarity_residual	union / composition	See defined object / value type	cost / decision unit; raw gradient norm
metrics (anyOf1).stationarity_residual (anyOf1)	number	minimum=0	cost / decision unit; raw gradient norm
metrics (anyOf1).stationarity_residual (anyOf2)	null	See defined object / value type	cost / decision unit; raw gradient norm
metrics (anyOf1).total_lower_violation	number	See defined object / value type	original row units; see unit map
metrics (anyOf1).total_upper_violation	number	See defined object / value type	original row units; see unit map
metrics (anyOf2)	null	See defined object / value type	structural metadata / nested units
normalized_gap	union / composition	See defined object / value type	dimensionless normalized diagnostic / factor
normalized_gap (anyOf1)	number	minimum=0	dimensionless normalized diagnostic / factor
normalized_gap (anyOf2)	null	See defined object / value type	dimensionless normalized diagnostic / factor
elapsed_seconds	union / composition	See defined object / value type	s
elapsed_seconds (anyOf1)	number	minimum=0	s
elapsed_seconds (anyOf2)	null	See defined object / value type	s
update_seconds	union / composition	See defined object / value type	s
update_seconds (anyOf1)	number	minimum=0	s
update_seconds (anyOf2)	null	See defined object / value type	s
workspace_setup_seconds	union / composition	See defined object / value type	s

Field path	Type	Restrictions	Unit / role
workspace_setup_seconds (anyOf1)	number	minimum=0	s
workspace_setup_seconds (anyOf2)	null	See defined object / value type	s
result	union / composition	See defined object / value type	structural metadata / nested units
result (anyOf1)	object	additionalProperties=False; required=['method', 'status', 'status_code', 'x', 'metrics', 'dimensions', 'iterations', 'setup_seconds', 'solve_seconds', 'elapsed_seconds']	structural metadata / nested units
result (anyOf1).method	string	See defined object / value type	structural metadata / nested units
result (anyOf1).status	string	See defined object / value type	structural metadata / nested units
result (anyOf1).status_code	integer	See defined object / value type	count / integer identifier
result (anyOf1).x	union / composition	See defined object / value type	decision unit
result (anyOf1).x (anyOf1)	array	See defined object / value type	decision unit
result (anyOf1).x (anyOf1)[]	number	See defined object / value type	decision unit
result (anyOf1).x (anyOf2)	null	See defined object / value type	decision unit
result (anyOf1).y	union / composition	See defined object / value type	cost / original row unit
result (anyOf1).y (anyOf1)	array	See defined object / value type	cost / original row unit
result (anyOf1).y (anyOf1)[]	number	See defined object / value type	cost / original row unit
result (anyOf1).y (anyOf2)	null	See defined object / value type	cost / original row unit
result (anyOf1).metrics	union / composition	See defined object / value type	structural metadata / nested units
result (anyOf1).metrics (anyOf1)	object	additionalProperties=False; required=['base_objective', 'penalty_objective', 'objective', 'hard_violation', 'lower_violation', 'upper_violation', 'slack']	structural metadata / nested units
result (anyOf1).metrics (anyOf1).base_objective	number	See defined object / value type	model cost unit
result (anyOf1).metrics (anyOf1).penalty_objective	number	See defined object / value type	model cost unit
result (anyOf1).metrics (anyOf1).objective	number	See defined object / value type	model cost unit
result (anyOf1).metrics (anyOf1).hard_violation	number	See defined object / value type	original row units; see unit map
result (anyOf1).metrics (anyOf1).lower_violation	array	See defined object / value type	original row units; see unit map
result (anyOf1).metrics (anyOf1).lower_violation[]	number	See defined object / value type	original row units; see unit map
result (anyOf1).metrics (anyOf1).upper_violation	array	See defined object / value type	original row units; see unit map
result (anyOf1).metrics (anyOf1).upper_violation[]	number	See defined object / value type	original row units; see unit map
result (anyOf1).metrics (anyOf1).slack	array	See defined object / value type	original row units; see unit map
result (anyOf1).metrics (anyOf1).slack[]	number	See defined object / value type	original row units; see unit map

Field path	Type	Restrictions	Unit / role
result (anyOf1).metrics (anyOf1).stationarity_residual	union / composition	See defined object / value type	cost / decision unit; raw gradient norm
result (anyOf1).metrics (anyOf1).stationarity_residual (anyOf1)	number	minimum=0	cost / decision unit; raw gradient norm
result (anyOf1).metrics (anyOf1).stationarity_residual (anyOf2)	null	See defined object / value type	cost / decision unit; raw gradient norm
result (anyOf1).metrics (anyOf1).objective_gap_against_lifted	union / composition	See defined object / value type	model cost unit
result (anyOf1).metrics (anyOf1).objective_gap_against_lifted (anyOf1)	number	See defined object / value type	model cost unit
result (anyOf1).metrics (anyOf1).objective_gap_against_lifted (anyOf2)	null	See defined object / value type	model cost unit
result (anyOf1).metrics (anyOf2)	null	See defined object / value type	structural metadata / nested units
result (anyOf1).dimensions	object	additionalProperties=False; required=['variables', 'rows', 'nnz']	structural metadata / nested units
result (anyOf1).dimensions.variables	integer	minimum=0	count / integer identifier
result (anyOf1).dimensions.rows	integer	minimum=0	count / integer identifier
result (anyOf1).dimensions.nnz	integer	minimum=0	count / integer identifier
result (anyOf1).iterations	integer	minimum=0	count / integer identifier
result (anyOf1).setup_seconds	number	minimum=0	s
result (anyOf1).solve_seconds	number	minimum=0	s
result (anyOf1).elapsed_seconds	number	minimum=0	s
result (anyOf1).backend_settings	object	additionalProperties={'type': 'number'}	structural metadata / nested units
result (anyOf2)	null	See defined object / value type	structural metadata / nested units
iterations	union / composition	See defined object / value type	count / integer identifier
iterations (anyOf1)	integer	minimum=0	count / integer identifier
iterations (anyOf2)	null	See defined object / value type	count / integer identifier
dimensions	union / composition	See defined object / value type	structural metadata / nested units
dimensions (anyOf1)	object	additionalProperties=False; required=['variables', 'rows', 'nnz']	structural metadata / nested units
dimensions (anyOf1).variables	integer	minimum=0	count / integer identifier
dimensions (anyOf1).rows	integer	minimum=0	count / integer identifier
dimensions (anyOf1).nnz	integer	minimum=0	count / integer identifier

Field path	Type	Restrictions	Unit / role
dimensions (anyOf2)	null	See defined object / value type	structural metadata / nested units
native_setup_seconds	union / composition	See defined object / value type	s
native_setup_seconds (anyOf1)	number	minimum=0	s
native_setup_seconds (anyOf2)	null	See defined object / value type	s
native_solve_seconds	union / composition	See defined object / value type	s
native_solve_seconds (anyOf1)	number	minimum=0	s
native_solve_seconds (anyOf2)	null	See defined object / value type	s
status	union / composition	See defined object / value type	structural metadata / nested units
status (anyOf1)	string	See defined object / value type	structural metadata / nested units
status (anyOf2)	null	See defined object / value type	structural metadata / nested units
status_code	union / composition	See defined object / value type	count / integer identifier
status_code (anyOf1)	integer	See defined object / value type	count / integer identifier
status_code (anyOf2)	null	See defined object / value type	count / integer identifier

Nested fields of softqp-experiment-index are enumerated in Table 37.

Table 37: Complete nested fields, value types, literal schema restrictions and physical or structural dimensions for softqp-experiment-index. Required and optional structures retain the original schema and model interpretation.

Field path	Type	Restrictions	Unit / role
schema_version	string	const=softqp.experiments/v1	structural metadata / nested units
protocol_id	string	const=softqp-experiments-v1	structural metadata / nested units
artifacts	array	See defined object / value type	structural metadata / nested units
artifacts[]	object	additionalProperties=False; required=['path', 'bytes', 'sha256', 'schema']	structural metadata / nested units
artifacts[].path	string	pattern=^[a-z][a-z0-9-]*\.(json|csv|jsonl)$	structural metadata / nested units
artifacts[].bytes	integer	minimum=0	bytes
artifacts[].sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
artifacts[].schema	enumerated value	enum=['softqp-stream', 'softqp-campaign', 'softqp-datasets', 'softqp-trial', 'csv']	structural metadata / nested units
state	enumerated value	enum=['development', 'measured']	structural metadata / nested units

Nested fields of softqp-coupled-batch are enumerated in Table 38.

Table 38: Complete nested fields, value types, literal schema restrictions and physical or structural dimensions for softqp-coupled-batch. Required and optional structures retain the original schema and model interpretation.

Field path	Type	Restrictions	Unit / role
schema_version	string	const=1.0.0	structural metadata / nested units
kind	string	const=directional-coupled-peak-batch	structural metadata / nested units
description	string	minLength=1	structural metadata / nested units
problem	object	additionalProperties=False; required=['P', 'A', 'q', 'lower', 'upper', 'groups']	structural metadata / nested units
problem.P	object	additionalProperties=False; required=['shape', 'data', 'indices', 'indptr']	structural metadata / nested units
problem.P.shape	array	minItems=2; maxItems=2	count / integer identifier
problem.P.shape[]	integer	minimum=1; maximum=16384	count / integer identifier
problem.P.data	array	maxItems=4194304	cost / decision units squared
problem.P.data[]	number	See defined object / value type	cost / decision units squared
problem.P.indices	array	maxItems=4194304	count / integer identifier
problem.P.indices[]	integer	minimum=0	count / integer identifier
problem.P.indptr	array	minItems=2; maxItems=16385	count / integer identifier
problem.P.indptr[]	integer	minimum=0; maximum=4194304	count / integer identifier
problem.A	object	additionalProperties=False; required=['shape', 'data', 'indices', 'indptr']	structural metadata / nested units
problem.A.shape	array	minItems=2; maxItems=2	count / integer identifier
problem.A.shape[]	integer	minimum=1; maximum=16384	count / integer identifier
problem.A.data	array	maxItems=4194304	row / decision unit
problem.A.data[]	number	See defined object / value type	row / decision unit
problem.A.indices	array	maxItems=4194304	count / integer identifier
problem.A.indices[]	integer	minimum=0	count / integer identifier
problem.A.indptr	array	minItems=2; maxItems=16385	count / integer identifier
problem.A.indptr[]	integer	minimum=0; maximum=4194304	count / integer identifier
problem.q	array	minItems=1; maxItems=16384	cost / decision unit
problem.q[]	number	See defined object / value type	cost / decision unit
problem.lower	array	minItems=1; maxItems=16384	original row unit
problem.lower[]	number / null	See defined object / value type	original row unit
problem.upper	array	minItems=1; maxItems=16384	original row unit
problem.upper[]	number / null	See defined object / value type	original row unit

Field path	Type	Restrictions	Unit / role
problem.groups	array	maxItems=16384	structural metadata / nested units
problem.groups[]	object	additionalProperties=False; required=['rows', 'lower_scale', 'upper_scale', 'linear', 'quadratic']	structural metadata / nested units
problem.groups[].rows	array	minItems=1; maxItems=16384; uniqueItems=True	count / integer identifier
problem.groups[].rows[]	integer	minimum=0	count / integer identifier
problem.groups[].lower_scale	array	minItems=1; maxItems=16384	peak unit / row unit; model map
problem.groups[].lower_scale[]	number	exclusiveMinimum=0	peak unit / row unit; model map
problem.groups[].upper_scale	array	minItems=1; maxItems=16384	peak unit / row unit; model map
problem.groups[].upper_scale[]	number	exclusiveMinimum=0	peak unit / row unit; model map
problem.groups[].linear	number	minimum=0	cost / peak unit
problem.groups[].quadratic	number	minimum=0	cost / peak unit squared
updates	array	minItems=1; maxItems=512	structural metadata / nested units
updates[]	object	additionalProperties=False; required=['q', 'lower', 'upper']	structural metadata / nested units
updates[].q	array	minItems=1; maxItems=16384	cost / decision unit
updates[].q[]	number	See defined object / value type	cost / decision unit
updates[].lower	array	minItems=1; maxItems=16384	original row unit
updates[].lower[]	number / null	See defined object / value type	original row unit
updates[].upper	array	minItems=1; maxItems=16384	original row unit
updates[].upper[]	number / null	See defined object / value type	original row unit
units	object	additionalProperties={'type': 'string', 'minLength': 1}; required=['variable', 'row', 'q', 'P_diagonal', 'objective']	structural metadata / nested units
units.variable	array	minItems=1; maxItems=16384	structural metadata / nested units
units.variable[]	string	minLength=1	structural metadata / nested units
units.row	array	minItems=1; maxItems=16384	structural metadata / nested units
units.row[]	string	minLength=1	structural metadata / nested units
units.q	array	minItems=1; maxItems=16384	cost / decision unit
units.q[]	string	minLength=1	cost / decision unit
units.P_diagonal	array	minItems=1; maxItems=16384	structural metadata / nested units
units.P_diagonal[]	string	minLength=1	structural metadata / nested units
units.objective	string	minLength=1	model cost unit

Nested fields of softqp-coupled-observation are enumerated in Table 39.

Table 39: Complete nested fields, value types, literal schema restrictions and physical or structural dimensions for softqp-coupled-observation. Required and optional structures retain the original schema and model interpretation.

Field path	Type	Restrictions	Unit / role
schema_version	string	const=1.0.0	structural metadata / nested units
kind	string	const=directional-coupled-observation	structural metadata / nested units
role	enumerated value	enum=['oracle', 'method']	structural metadata / nested units
id	string	pattern=^(development|heldout|robustness)-[0-9]{4}$	structural metadata / nested units
split	enumerated value	enum=['development', 'heldout', 'robustness']	structural metadata / nested units
seed	integer	minimum=0	count / integer identifier
family	enumerated value	enum=['sparse', 'thermal']	structural metadata / nested units
curvature	enumerated value	enum=['positive-definite', 'semidefinite', 'zero-hessian']	structural metadata / nested units
quadratic	number	minimum=0	cost / peak unit squared
method	enumerated value	enum=['__oracle__', 'native', 'epigraph-default', 'epigraph-fixed', 'row-split-default', 'row-split-fixed', 'daqp-direct', 'daqp-prox-auto', 'daqp-prox-forced', 'daqp-lp-model']	structural metadata / nested units
repetition	integer / null	minimum=-1; maximum=2	count / integer identifier
warmup	boolean	See defined object / value type	structural metadata / nested units
state	enumerated value	enum=['accepted', 'failed', 'accuracy-rejected', 'incompatible', 'oracle-failed', 'resource-limit', 'timeout']	structural metadata / nested units
reason	string / null	See defined object / value type	structural metadata / nested units
elapsed_seconds	number / null	exclusiveMinimum=0	s
batch_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
predeclared_incompatible	boolean	See defined object / value type	structural metadata / nested units
frames	array	maxItems=32	count / integer identifier
frames[]	object	required=['state', 'reason', 'metrics']	count / integer identifier
frames[].state	enumerated value	enum=['accepted', 'failed', 'accuracy-rejected']	structural metadata / nested units
frames[].reason	string / null	See defined object / value type	structural metadata / nested units
frames[].metrics	object / null	See defined object / value type	structural metadata / nested units
frames[].metrics.objective	number	See defined object / value type	model cost unit

Field path	Type	Restrictions	Unit / role
frames[].metrics.peaks	array	See defined object / value type	group peak unit; model map
frames[].metrics.peaks[]	number	minimum=0	group peak unit; model map
frames[].metrics.hard_violation	number	minimum=0	original row units; see unit map
frames[].metrics.stationarity	number	minimum=0	dimensionless normalized diagnostic / factor
frames[].metrics.dual_domain	number	minimum=0	dimensionless normalized diagnostic / factor
frames[].metrics.dual_repair	number	minimum=0	dimensionless normalized diagnostic / factor
frames[].metrics.fenchel	number	minimum=0	dimensionless normalized diagnostic / factor
frames[].metrics.aggregate_fenchel	number	minimum=0	dimensionless normalized diagnostic / factor
frames[].metrics.objective_gap	number	minimum=0	dimensionless normalized diagnostic / factor
frames[] (allOf1)	object	See defined object / value type	count / integer identifier
frames[] (allOf1) (if)	object	See defined object / value type	count / integer identifier
frames[] (allOf1) (if).state	string	const=accepted	structural metadata / nested units
frames[] (allOf1) (then)	object	See defined object / value type	count / integer identifier
frames[] (allOf1) (then).reason	null	See defined object / value type	structural metadata / nested units
frames[] (allOf1) (then).metrics	object	required=['objective', 'hard_violation', 'stationarity', 'dual_domain', 'dual_repair', 'fenchel', 'objective_gap']	structural metadata / nested units
backend_results	array	maxItems=32	structural metadata / nested units
backend_results[]	object	required=['method', 'status', 'status_code', 'x', 'y', 'dimensions']	structural metadata / nested units
backend_results[].method	string	See defined object / value type	structural metadata / nested units
backend_results[].status	string	See defined object / value type	structural metadata / nested units
backend_results[].status_code	integer	See defined object / value type	count / integer identifier
backend_results[].x	array / null	See defined object / value type	decision unit
backend_results[].x[]	number	See defined object / value type	decision unit
backend_results[].y	array / null	See defined object / value type	cost / original row unit
backend_results[].y[]	number	See defined object / value type	cost / original row unit
backend_results[].dimensions	object	required=['variables', 'rows']	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
backend_results[].dimensions.variables	integer	minimum=1	count / integer identifier
backend_results[].dimensions.rows	integer	minimum=1	count / integer identifier
objectives	array / null	maxItems=32	model cost unit
objectives[]	number	See defined object / value type	model cost unit
n	integer	minimum=1	count / integer identifier
group_size	integer	minimum=1	count / integer identifier
condition	number	minimum=1	dimensionless normalized diagnostic / factor
horizon	integer	minimum=1	count / integer identifier
zones	integer	minimum=1	count / integer identifier
(allOf1)	object	See defined object / value type	structural metadata / nested units
(allOf1) (if)	object	See defined object / value type	structural metadata / nested units
(allOf1) (if).role	string	const=oracle	structural metadata / nested units
(allOf1) (then)	object	See defined object / value type	structural metadata / nested units
(allOf1) (then).method	string	const=___oracle___	structural metadata / nested units
(allOf1) (then).repetition	null	See defined object / value type	count / integer identifier
(allOf1) (then).warmup	boolean	const=False	structural metadata / nested units
(allOf1) (else)	object	required=['elapsed_seconds']	structural metadata / nested units
(allOf1) (else).repetition	integer	See defined object / value type	count / integer identifier
(allOf1) (else).method	object	See defined object / value type	structural metadata / nested units
(allOf1) (else).method (not)	string	const=___oracle___	structural metadata / nested units
(allOf2)	object	See defined object / value type	structural metadata / nested units
(allOf2) (if)	object	See defined object / value type	structural metadata / nested units
(allOf2) (if).state	string	const=accepted	structural metadata / nested units
(allOf2) (then)	object	required=['batch_sha256', 'frames', 'backend_results']	structural metadata / nested units
(allOf2) (then).reason	null	See defined object / value type	structural metadata / nested units
(allOf2) (then).frames	object	minItems=32	count / integer identifier
(allOf2) (then).frames[]	object	See defined object / value type	count / integer identifier
(allOf2) (then).frames[].state	string	const=accepted	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
(allOf2) (then).backend_results	object	minItems=32	structural metadata / nested units
(allOf3)	object	See defined object / value type	structural metadata / nested units
(allOf3) (if)	object	See defined object / value type	structural metadata / nested units
(allOf3) (if).role	string	const=oracle	structural metadata / nested units
(allOf3) (if).state	string	const=accepted	structural metadata / nested units
(allOf3) (then)	object	required=['objectives']	structural metadata / nested units
(allOf3) (then).objectives	array	minItems=32	model cost unit
(allOf4)	object	See defined object / value type	structural metadata / nested units
(allOf4) (if)	object	See defined object / value type	structural metadata / nested units
(allOf4) (if).role	string	const=method	structural metadata / nested units
(allOf4) (if).state	string	const=accepted	structural metadata / nested units
(allOf4) (then)	object	See defined object / value type	structural metadata / nested units
(allOf4) (then).elapsed_seconds	number	exclusiveMinimum=0	s

Nested fields of softqp-coupled-campaign are enumerated in Table 40.

Table 40: Complete nested fields, value types, literal schema restrictions and physical or structural dimensions for softqp-coupled-campaign. Required and optional structures retain the original schema and model interpretation.

Field path	Type	Restrictions	Unit / role
schema_version	string	const=1.0.0	structural metadata / nested units
kind	string	const=directional-coupled-campaign	structural metadata / nested units
protocol_id	string	const=softqp-directional-coupled-v1	structural metadata / nested units
split	enumerated value	enum=['development', 'heldout', 'robustness']	structural metadata / nested units
environment	object	required=['packages', 'native_binary_sha256', 'registration', 'sdk_python_sha256', 'hardware']	structural metadata / nested units
environment.packages	object	additionalProperties={'type': 'string'}	structural metadata / nested units
environment.native_binary_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
environment.registration	object	required=['commit', 'protocol_sha256', 'review_sha256']	structural metadata / nested units
environment.registration.commit	string	pattern=^[0-9a-f]{40}$	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
environment.registration.protocol_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
environment.registration.review_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
environment.sdk_python_sha256	object	additionalProperties={'$ref': '#/$defs/hash'}	structural metadata / nested units
environment.hardware	object	required=['machine', 'processor', 'logical_cpu_count']	structural metadata / nested units
source_sha256	object	additionalProperties={'$ref': '#/$defs/hash'}	structural metadata / nested units
inventory	array	See defined object / value type	structural metadata / nested units
inventory[]	object	required=['id', 'split', 'seed', 'family', 'curvature', 'quadratic']	structural metadata / nested units
datasets	array	See defined object / value type	structural metadata / nested units
datasets[]	object	required=['id', 'oracle_state', 'oracle_reason', 'oracle_objectives', 'batch_sha256']	structural metadata / nested units
datasets[].id	string	See defined object / value type	structural metadata / nested units
datasets[].oracle_state	enumerated value	enum=['accepted', 'oracle-failed', 'failed', 'timeout', 'resource-limit']	structural metadata / nested units
datasets[].oracle_reason	string / null	See defined object / value type	structural metadata / nested units
datasets[].oracle_objectives	array / null	See defined object / value type	model cost unit
datasets[].oracle_objectives[]	number	See defined object / value type	model cost unit
datasets[].batch_sha256	string / null	pattern=^[0-9a-f]{64}$	structural metadata / nested units
counts	object	additionalProperties={'type': 'integer', 'minimum': 0}	structural metadata / nested units
primary	object	required=['established', 'pairs']	structural metadata / nested units
primary.established	boolean	See defined object / value type	structural metadata / nested units
primary.pairs	array	See defined object / value type	structural metadata / nested units
primary.pairs[]	object	required=['id', 'seed', 'reduction', 'fastest_baseline', 'failure_reasons', 'median_seconds']	structural metadata / nested units
primary.pairs[].reduction	number	See defined object / value type	dimensionless normalized diagnostic / factor
primary.pairs[].seed	integer	See defined object / value type	count / integer identifier
primary.pairs[].failure_reasons	array	See defined object / value type	structural metadata / nested units
primary.pairs[].failure_reasons[]	string	See defined object / value type	structural metadata / nested units
primary.pairs[].median_seconds	object	additionalProperties={'type': 'number', 'exclusiveMinimum': 0}	s
primary.estimate	number	See defined object / value type	dimensionless normalized diagnostic / factor

Field path	Type	Restrictions	Unit / role
primary.confidence_interval	array	minItems=2; maxItems=2	dimensionless normalized diagnostic / factor
primary.confidence_ interval[]	number	See defined object / value type	dimensionless normalized diagnostic / factor
primary.denominator	integer	minimum=0	count / integer identifier
primary.seed_clusters	integer	minimum=0	count / integer identifier
primary.failed_pairs	integer	minimum=0	count / integer identifier
peak_worker_rss	integer	minimum=0	bytes
worker_restarts	integer	minimum=0	count / integer identifier
raw_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
csv_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units

Nested fields of softqp-coupled-lesson are enumerated in Table 41.

Table 41: Complete nested fields, value types, literal schema restrictions and physical or structural dimensions for softqp-coupled-lesson. Required and optional structures retain the original schema and model interpretation.

Field path	Type	Restrictions	Unit / role
schema_version	string	const=softqp.coupled-lesson/v1	structural metadata / nested units
sdk_version	string	const=0.2.0	structural metadata / nested units
worker_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
input_sha256	string	pattern=^[0-9a-f]{64}$	structural metadata / nested units
input	object	additionalProperties=False; required= ['linear', 'quadratic', 'upper_scale', 'row_ scale']	structural metadata / nested units
input.linear	number	minimum=0; maximum=10	cost / peak unit
input.quadratic	number	minimum=0; maximum=10	cost / peak unit squared
input.upper_scale	number	minimum=0.25; maximum=4	dimensionless policy multiplier
input.row_scale	number	minimum=0.01; maximum=100	dimensionless factor
records	array	minItems=3; maxItems=3	structural metadata / nested units
records[]	object	additionalProperties=False; required= ['method', 'state', 'reason', 'objective', 'x', 'peak', 'hard_violation', 'kkt_max', 'dimensions']	structural metadata / nested units
records[].method	enumerated value	enum=['native', 'epigraph', 'row-split-gsp']	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
records[].state	enumerated value	enum=['accepted', 'incompatible', 'failed', 'accuracy-rejected']	structural metadata / nested units
records[].reason	union / composition	See defined object / value type	structural metadata / nested units
records[].reason (anyOf1)	string	minLength=1	structural metadata / nested units
records[].reason (anyOf2)	null	See defined object / value type	structural metadata / nested units
records[].objective	union / composition	See defined object / value type	model cost unit
records[].objective (anyOf1)	number	See defined object / value type	model cost unit
records[].objective (anyOf2)	null	See defined object / value type	model cost unit
records[].x	union / composition	See defined object / value type	decision unit
records[].x (anyOf1)	array	minItems=2; maxItems=2	decision unit
records[].x (anyOf1)[]	number	See defined object / value type	decision unit
records[].x (anyOf2)	null	See defined object / value type	decision unit
records[].peak	union / composition	See defined object / value type	group peak unit; model map
records[].peak (anyOf1)	number	minimum=0	group peak unit; model map
records[].peak (anyOf2)	null	See defined object / value type	group peak unit; model map
records[].hard_violation	union / composition	See defined object / value type	original row units; see unit map
records[].hard_violation (anyOf1)	number	minimum=0	original row units; see unit map
records[].hard_violation (anyOf2)	null	See defined object / value type	original row units; see unit map
records[].kkt_max	union / composition	See defined object / value type	dimensionless normalized diagnostic / factor
records[].kkt_max (anyOf1)	number	minimum=0	dimensionless normalized diagnostic / factor
records[].kkt_max (anyOf2)	null	See defined object / value type	dimensionless normalized diagnostic / factor
records[].dimensions	union / composition	See defined object / value type	structural metadata / nested units
records[].dimensions (anyOf1)	object	additionalProperties=False; required=['variables', 'rows', 'nnz']	structural metadata / nested units
records[].dimensions (anyOf1).variables	integer	minimum=1	count / integer identifier
records[].dimensions (anyOf1).rows	integer	minimum=1	count / integer identifier
records[].dimensions (anyOf1).nnz	integer	minimum=0	count / integer identifier
records[].dimensions (anyOf2)	null	See defined object / value type	structural metadata / nested units
records[] (allOf1)	object	See defined object / value type	structural metadata / nested units
records[] (allOf1) (if)	object	See defined object / value type	structural metadata / nested units
records[] (allOf1) (if).state	enumerated value	enum=['accepted', 'accuracy-rejected']	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
records[] (allOf1) (then)	object	See defined object / value type	structural metadata / nested units
records[] (allOf1) (then). objective	number	See defined object / value type	model cost unit
records[] (allOf1) (then).x	array	minItems=2; maxItems=2	decision unit
records[] (allOf1) (then).x[]	number	See defined object / value type	decision unit
records[] (allOf1) (then). peak	number	minimum=0	group peak unit; model map
records[] (allOf1) (then). hard_violation	number	minimum=0	original row units; see unit map
records[] (allOf1) (then). kkt_max	number	minimum=0	dimensionless normalized diagnostic / factor
records[] (allOf1) (then). dimensions	object	additionalProperties=False; required=['variables', 'rows', 'nnz']	structural metadata / nested units
records[] (allOf1) (then). dimensions.variables	integer	minimum=1	count / integer identifier
records[] (allOf1) (then). dimensions.rows	integer	minimum=1	count / integer identifier
records[] (allOf1) (then). dimensions.nnz	integer	minimum=0	count / integer identifier
records[] (allOf2)	object	See defined object / value type	structural metadata / nested units
records[] (allOf2) (if)	object	See defined object / value type	structural metadata / nested units
records[] (allOf2) (if).state	string	const=accepted	structural metadata / nested units
records[] (allOf2) (then)	object	See defined object / value type	structural metadata / nested units
records[] (allOf2) (then). reason	null	See defined object / value type	structural metadata / nested units
records[] (allOf2) (else)	object	See defined object / value type	structural metadata / nested units
records[] (allOf2) (else). reason	string	minLength=1	structural metadata / nested units
records[] (allOf3)	object	See defined object / value type	structural metadata / nested units
records[] (allOf3) (if)	object	See defined object / value type	structural metadata / nested units
records[] (allOf3) (if).state	enumerated value	enum=['incompatible', 'failed']	structural metadata / nested units
records[] (allOf3) (then)	object	See defined object / value type	structural metadata / nested units
records[] (allOf3) (then). objective	null	See defined object / value type	model cost unit
records[] (allOf3) (then).x	null	See defined object / value type	decision unit
records[] (allOf3) (then). peak	null	See defined object / value type	group peak unit; model map
records[] (allOf3) (then). hard_violation	null	See defined object / value type	original row units; see unit map
records[] (allOf3) (then). kkt_max	null	See defined object / value type	dimensionless normalized diagnostic / factor
records[] (allOf3) (then). dimensions	null	See defined object / value type	structural metadata / nested units

Field path	Type	Restrictions	Unit / role
(allOf1)	object	See defined object / value type	structural metadata / nested units
(allOf1).records	object	minContains=1; maxContains=1	structural metadata / nested units
(allOf1).records (contains)	object	See defined object / value type	structural metadata / nested units
(allOf1).records (contains). method	string	const=native	structural metadata / nested units
(allOf2)	object	See defined object / value type	structural metadata / nested units
(allOf2).records	object	minContains=1; maxContains=1	structural metadata / nested units
(allOf2).records (contains)	object	See defined object / value type	structural metadata / nested units
(allOf2).records (contains). method	string	const=epigraph	structural metadata / nested units
(allOf3)	object	See defined object / value type	structural metadata / nested units
(allOf3).records	object	minContains=1; maxContains=1	structural metadata / nested units
(allOf3).records (contains)	object	See defined object / value type	structural metadata / nested units
(allOf3).records (contains). method	string	const=row-split-gsp	structural metadata / nested units

References

[1] Daniel Arnström, Alberto Bemporad, and Daniel Axehill. A Dual Active-Set Solver for Embedded Quadratic Programming Using Recursive LDL^T Updates. *IEEE Transactions on Automatic Control*, 67(8):4362–4369, 2022.

[2] Daniel Arnström, Alberto Bemporad, and Daniel Axehill. A Linear Programming Method Based on Proximal-Point Iterations With Applications to Multi-Parametric Programming. *IEEE Control Systems Letters*, 6:2066–2071, 2022.

[3] Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, and Jonathan Eckstein. Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers. *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011.

[4] A. Colin Cameron, Jonah B. Gelbach, and Douglas L. Miller. Bootstrap-Based Improvements for Inference with Clustered Errors. *Review of Economics and Statistics*, 90(3):414–427, 2008.

[5] Jean-Philippe Chancelier and Michel de Lara. Orthant-Strictly Monotonic Norms, Generalized Top-k and k-Support Norms and the ℓ_0 Pseudonorm, 2022. Version 3.

[6] Laurent Condat. Fast projection onto the simplex and the ℓ_1 ball. *Mathematical Programming*, 158(1-2):575–585, 2016.

[7] A. C. Davison and D. V. Hinkley. *Bootstrap Methods and their Application*. Cambridge University Press, 1997.

[8] Elizabeth D. Dolan and Jorge J. Moré. Benchmarking optimization software with performance profiles. *Mathematical Programming*, 91(2):201–213, 2002.

[9] John Duchi, Shai Shalev-Shwartz, Yoram Singer, and Tushar Chandra. Efficient projections onto the ℓ_1-ball for learning in high dimensions. In *Proceedings of the 25th international conference on Machine learning - ICML '08*, pages 272–279, 2008.

[10] Jonathan Eckstein and Dimitri P. Bertsekas. On the Douglas–Rachford splitting method and the proximal point algorithm for maximal monotone operators. *Mathematical Programming*, 55(1-3):293–318, 1992.

[11] B. Efron. Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

[12] Hans Joachim Ferreau, Christian Kirches, Andreas Potschka, Hans Georg Bock, and Moritz Diehl. qpOASES: a parametric active-set algorithm for quadratic programming. *Mathematical Programming Computation*, 6(4):327–363, 2014.

[13] D. Goldfarb and A. Idnani. A numerically stable dual method for solving strictly convex quadratic programs. *Mathematical Programming*, 27(1):1–33, 1983.

[14] Nicholas I. M. Gould, Dominique Orban, and Philippe L. Toint. CUTEst: a Constrained and Unconstrained Testing Environment with safe threads for mathematical optimization. *Computational Optimization and Applications*, 60(3):545–557, 2015.

[15] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, 2 edition, 2002.

[16] Eric C. Kerrigan and Jan M. Maciejowski. Soft Constraints and Exact Penalty Functions in Model Predictive Control. In *UKACC International Conference on Control*. Institution of Electrical Engineers, 2000.

[17] Philip A. Knight, Daniel Ruiz, and Bora Uçar. A Symmetry Preserving Algorithm for Matrix Scaling. *SIAM Journal on Matrix Analysis and Applications*, 35(3):931–955, 2014.

[18] Thomas Lew, Marcus Greiff, John Subosits, and Brian Plancher. Solving quadratic programs with slack variables via ADMM without increasing the problem size. *European Journal of Control*, page 101557, 2026.

[19] D.Q. Mayne, J.B. Rawlings, C.V. Rao, and P.O.M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, 2000.

[20] J.J. Moreau. Proximité et dualité dans un espace hilbertien. *Bulletin de la Société Mathématique de France*, 93:273–299, 1965.

[21] Daniel Morton, Jon Arrizabalaga, Zachary Manchester, and Marco Pavone. ElastiQP: An Always-Feasible QP Solver for Constrained Robot Control, 2026. Version 1.

[22] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer New York, 2 edition, 2006.

[23] Alex Oshin, Rahul Ghosh, Augustinos Saravanos, and Evangelos Theodorou. Deep FlexQP: Accelerated Nonlinear Programming via Deep Unfolding. In *International Conference on Learning Representations*, 2026.

[24] Neal Parikh and Stephen Boyd. Proximal Algorithms. *Foundations and Trends in Optimization*, 1(3):123–231, 2014.

[25] Guillaume Perez, Sebastian Ament, Carla Gomes, and Michel Barlaud. Efficient projection algorithms onto the weighted ℓ_1 ball. *Artificial Intelligence*, 306:103683, 2022.

[26] James B. Rawlings, David Q. Mayne, and Moritz M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2 edition, 2026. Current sixth printing identified by the author site.

[27] R. Tyrrell Rockafellar. Monotone Operators and the Proximal Point Algorithm. *SIAM Journal on Control and Optimization*, 14(5):877–898, 1976.

[28] Roland Schwan, Yuning Jiang, Daniel Kuhn, and Colin N. Jones. PIQP: A Proximal Interior-Point Quadratic Programming Solver, 2023. Version 2.

[29] Roland Schwan, Thomas Lew, and Brian Plancher. Efficient ADMM for Soft-Constrained Quadratic Programs via Generalized Slack Projections, 2026. Version 1.

[30] Bartolomeo Stellato, Goran Banjac, Paul Goulart, Alberto Bemporad, and Stephen Boyd. OSQP: an operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 12(4):637–672, 2020.

[31] Robert J. Vanderbei. Symmetric Quasidefinite Matrices. *SIAM Journal on Optimization*, 5(1):100–113, 1995.